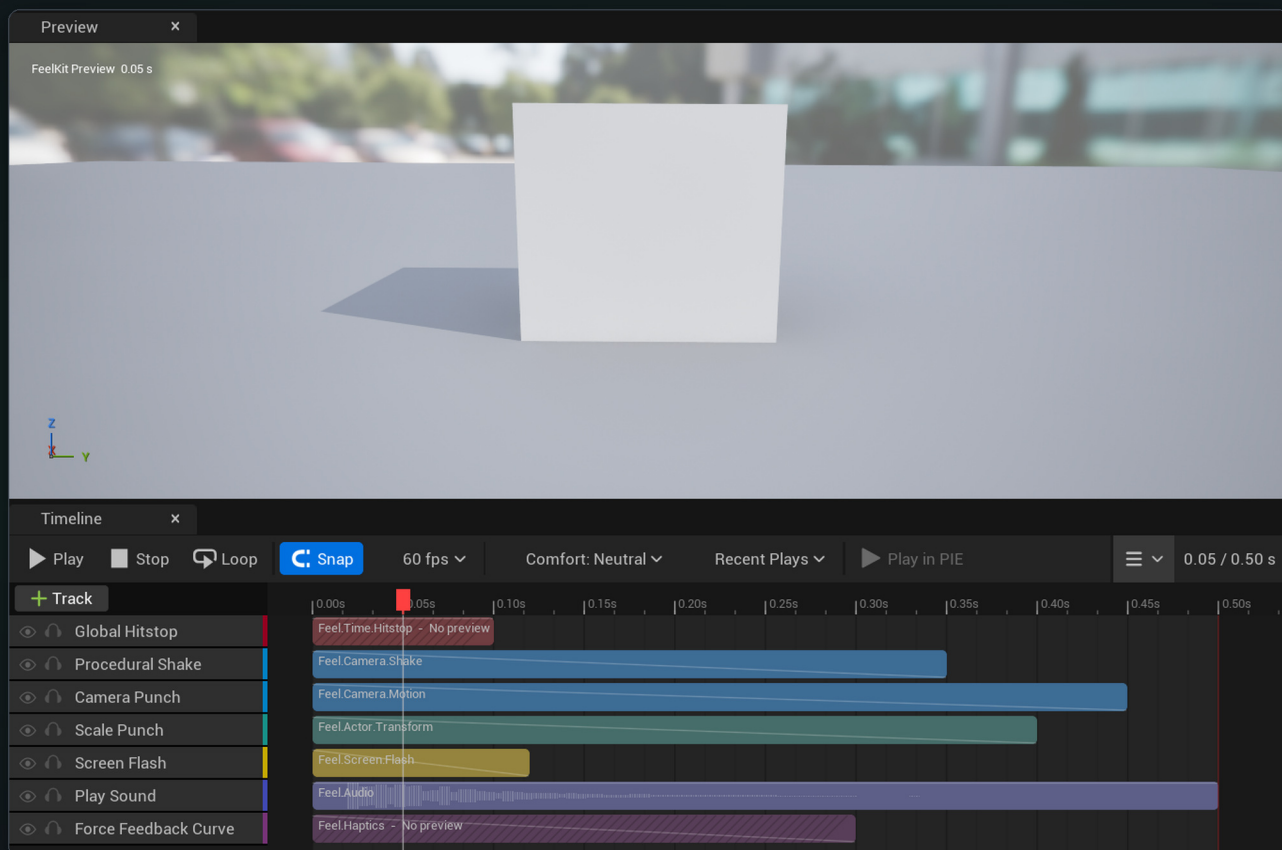


UNREAL ENGINE PLUGIN

FeelKit

Manual

Build game feel on a timeline. Play it with one node.
Let every player choose how much they feel.



TIMELINE

Build and preview a hit without pressing Play.

ONE NODE

Play recipes from Blueprint or C++, or send an event.

COMFORT

Every player chooses how much shake, flash and rumble they get.

Contents

1	Welcome.....	10
1.1	What FeelKit does	10
1.2	What is in the package.....	11
1.3	How to use this manual	12
1.4	Getting help.....	13
2	Core ideas.....	14
2.1	Recipes, tracks and steps	14
2.2	Channels, and what happens when effects overlap	15
2.3	Intensity: how strong each effect plays	17
2.4	Targets: what a recipe plays on	18
2.5	Parameters, accumulators and sustained recipes, in brief PRO	19
2.6	Events and Feel Maps, in brief PRO	20
2.7	Comfort on one page.....	22
2.8	Why the editor preview matches the game	23
3	Installing FeelKit.....	24
3.1	Requirements.....	24
3.2	Installing from Fab and enabling the plugin.....	25
3.3	What FeelKit switches on for you PRO	26
3.4	Showing FeelKit content in the Content Browser	26
3.5	Adding the GAS add-on PRO	27
3.6	Blueprint-only projects and packaging	27
3.7	Moving a project from Lite to Pro	28
4	Quick start: your first recipe	29
4.1	Pick a recipe from the library	29
4.2	Preview it without pressing Play	30
4.3	Play it from a Blueprint	31
4.4	Compare with the feel off.....	33

4.5	Where to go next	34
5	The recipe editor	35
5.1	Creating and opening recipes.....	35
5.2	The window: Preview, Timeline, Details, Intensity	36
5.3	The toolbar	37
5.4	Adding, moving and resizing tracks	37
5.5	Shaping intensity with curve keys	38
5.6	Recipe and track settings in Details.....	39
5.7	The preview: play, loop, scrub, preview mesh, parameter sliders	40
5.8	Sound tracks: waveforms, snapping, tracks made from a sound PRO	41
5.9	Checking comfort, and the intensity graph.....	42
5.10	Play in PIE, and editing while the game runs.....	43
5.11	Copy, paste and keyboard shortcuts	43
5.12	Validation messages on save	44
6	Playing recipes in your game	45
6.1	Play Feel, and choosing a target.....	46
6.2	Passing context with Play Feel with Context PRO	47
6.3	Handles: stop, release, change a parameter.....	48
6.4	Play Feel and Wait PRO	49
6.5	Events and Feel Maps PRO	49
6.6	Animation notifies PRO	52
6.7	The Feel Trigger component PRO	52
6.8	Enhanced Input: the Feel Input component PRO	54
6.9	Gameplay Ability System cues PRO	55
6.10	Using FeelKit from C++	55
6.11	The Feel Switch: feel off and on while playing.....	56
7	Parameters, context and variation.....	58
7.1	Parameters and track mappings	58
7.2	The Distance parameter	60
7.3	Accumulators: effects that build up with repeated plays.....	60
7.4	Sustained recipes	61

7.5	The instigator, and the Applies To setting	62
7.6	Directions and locations.....	62
7.7	Randomness, conditions and Random Choice.....	62
7.8	Recipes inside recipes	63
8	Comfort and accessibility	65
8.1	Comfort groups, and how the scales apply	65
8.2	Presets	66
8.3	Reading and changing a player's settings	67
8.4	Essential tracks and substitutes.....	68
8.5	The flash limiter	69
8.6	Motion comfort: camera roll and field of view speed	69
8.7	Engine camera shakes and controller vibration.....	69
8.8	Saving settings, or using your own save system	70
8.9	The comfort menu for players	70
8.10	Restyling the comfort menu.....	71
8.11	The Comfort Audit PRO	72
9	The recipe library and browser	74
9.1	The Recipe Browser PRO	74
9.2	Filters, search and live preview PRO	75
9.3	Copying a library recipe into your project	76
9.4	Why library recipes are read-only	76
9.5	Content Browser tiles and tooltips.....	77
9.6	The 38 library recipes at a glance	77
9.7	Tagging your own recipes PRO	79
10	Debugging and tuning	81
10.1	The FeelKit Debugger PRO	81
10.2	Replaying a recent play in the editor PRO	83
10.3	Track labels that explain why a track did not play	84
10.4	showdebug feel and stat Feel PRO	85
10.5	Log messages.....	85
10.6	Capture GIF PRO	86

11	Multiplayer	87
11.1	What travels over the network	87
11.2	The Feel Replication component and its modes PRO	88
11.3	Relevancy distance PRO	89
11.4	Hitstop and slow motion in networked games	89
11.5	Dedicated servers	90
12	Performance.....	91
12.1	When FeelKit runs.....	91
12.2	What a play costs.....	92
12.3	Measuring it.....	92
12.4	Keeping many plays in check.....	93
12.5	Assets and memory	93
13	Demo levels.....	94
13.1	Getting the demos	94
13.2	Action/RPG: Weight Class	95
13.3	Platformer: Bounce Feel	96
13.4	Shooter: Every Bullet Has an Opinion.....	98
13.5	Horror: Heartbeat	100
14	Troubleshooting and questions.....	102
14.1	Nothing plays.....	102
14.2	Controllers and vibration.....	103
14.3	Camera and screen effects.....	104
14.4	Materials and decals PRO	104
14.5	Blueprints, animations and GAS	105
14.6	Recipes and the library	105
14.7	Installing, packaging and engine versions.....	106
14.8	Getting help	106
15	Lite and Pro	107
15.1	What each edition contains.....	107
15.2	What Lite leaves out.....	108
15.3	The Lite library	109

15.4	Recipes and data in Lite	110
15.5	Moving from Lite to Pro.....	110
15.6	Licenses.....	110
16	Step reference	112
16.1	Recipe fields.....	112
16.2	Track fields.....	114
16.3	Reading the step entries.....	116
16.4	Shared motion settings.....	117
16.5	Camera steps.....	118
16.6	Time steps.....	121
16.7	Screen steps.....	123
16.8	Actor steps.....	126
16.9	Audio steps	130
16.10	Haptics steps.....	133
16.11	UI steps.....	135
16.12	Spawn steps.....	137
16.13	Meta steps.....	139
17	Blueprint nodes and components.....	142
17.1	Playing and stopping recipes.....	142
17.2	Targets	147
17.3	Play context.....	149
17.4	Parameters and accumulators	150
17.5	Events and Feel Maps	152
17.6	Comfort.....	153
17.7	Feel Switch.....	158
17.8	Animation notifies	161
17.9	Feel Trigger component.....	163
17.10	Feel Input component.....	164
17.11	Feel Replication component	166
17.12	Gameplay Cue notifies (GAS add-on)	168
17.13	Editor scripting	169

17.14 C++ access	171
18 Project settings and editor preferences.....	173
18.1 Where the settings are kept.....	173
18.2 Camera.....	176
18.3 Playback.....	177
18.4 Events PRO	178
18.5 Comfort.....	178
18.6 Comfort scale fields and preset values	179
18.7 Comfort storage.....	181
18.8 Editor preferences	181
19 Console commands	182
19.1 feel.Enabled.....	182
19.2 feel.GlobalScale PRO	183
19.3 stat Feel PRO	183
19.4 showdebug feel PRO	183
20 Writing your own steps in Blueprint and C++	184
20.1 How a step works	184
20.2 A step in Blueprint.....	185
20.3 A step in C++	186
20.4 Channels and comfort	188
21 Recipe JSON files and editor scripting	189
21.1 Exporting and importing in the Content Browser	189
21.2 The file format.....	190
21.3 Editor scripting.....	190
22 Credits and licenses	192
22.1 FeelKit's license.....	192
22.2 Sample sounds	192
22.3 Materials.....	195
22.4 Demo projects.....	195
22.5 Your own project	195
23 Version history	196

23.1 1.0.0.....	196
Appendix A: Glossary.....	198

PRO In FeelKit Pro only. Everything without this label is in FeelKit Lite and FeelKit Pro.

Draft, 26 September 2026

PART ONE

Using FeelKit

- 1 Welcome
- 2 Core ideas
- 3 Installing FeelKit
- 4 Quick start: your first recipe
- 5 The recipe editor
- 6 Playing recipes in your game
- 7 Parameters, context and variation
- 8 Comfort and accessibility
- 9 The recipe library and browser
- 10 Debugging and tuning
- 11 Multiplayer
- 12 Performance
- 13 Demo kits
- 14 Troubleshooting and questions
- 15 Lite and Pro

1 Welcome

IN THIS CHAPTER

- 1.1 What FeelKit does
- 1.2 What is in the package
- 1.3 How to use this manual
- 1.4 Getting help

FeelKit is an Unreal Engine plugin for game feel: the short, precise feedback that makes an action read as heavy, fast, dangerous or rewarding. A hit that freezes the game for a few frames, kicks the camera, flashes the screen, punches the character's scale, plays a sound and rumbles the controller is one such piece of feedback. FeelKit lets these effects be designed, previewed and tuned together, and played from the game with a single node.

1.1 What FeelKit does

In a game that uses FeelKit, gameplay code states what happened, and FeelKit decides how it feels. The design rests on four ideas, each described in [Chapter 2, Core ideas](#):

- **Recipes.** A recipe is an asset that places effects on a timeline: a camera punch at 0 s, a hitstop at 0.02 s, a sound, a flash, a rumble. The game plays the whole recipe with **Play Feel**, so the timing of the effects is designed once, in one place.
- **A preview that matches the game.** The recipe editor plays the recipe while it is being edited, using the same code as the game, so most of a recipe is tuned without starting the game. Scrubbing the timeline shows any moment exactly.
- **Rules for overlapping effects.** When several recipes play at once, FeelKit decides which camera shake, flash or time scale wins, and always restores the camera and game time afterwards.
- **Comfort for players.** Every effect belongs to a comfort group, such as camera shake, flashes or controller vibration. Players can turn each group down in a ready-made menu, and FeelKit

applies their choice to every recipe the game plays. The recipe editor can preview a recipe with the same settings.

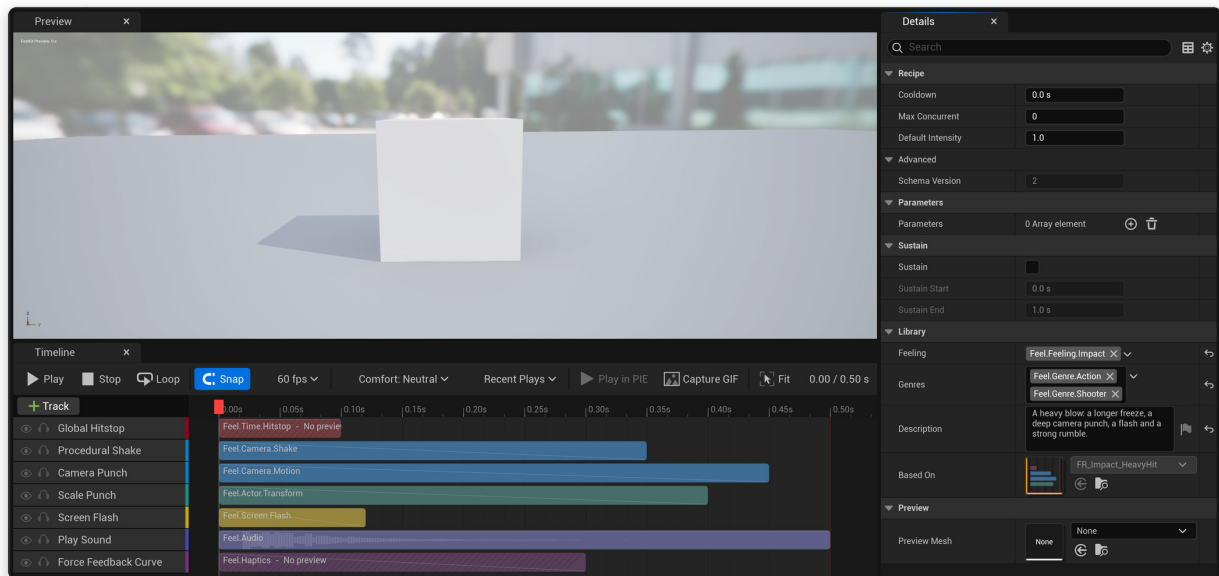


Figure 1.1 The recipe editor with the HeavyHit recipe: the preview above, the timeline with its seven tracks below, the settings on the right

1.2 What is in the package

Part	Contents
Plugin modules	FeelCore (runtime), FeelEditor (the recipe editor and editor tools), FeelNiagara (the particle step) and FeelEnhancedInput (the Feel Input component). Full C++ source is included.
Steps	37 effects for the camera, the screen, actors, game time, audio, controller vibration, UI and spawned effects (Chapter 16, Step reference).
Recipe library	38 ready-made recipes, grouped by feeling: Impact, Weight, Power, Speed, Reward, Danger, Dread, Denial and Interface (Chapter 9, The recipe library and browser).
Demo recipes	The recipes of the Shooter (20), Horror (8) and Platformer (5) demo levels (Chapter 13, Demo levels).
Samples	43 sounds, one sound attenuation asset and three materials, used by the library and the demos.
Comfort menu	A player menu for the comfort settings, ready to use and to restyle (Chapter 8, Comfort and accessibility).
GAS add-on	PRO A separate plugin in the Extras folder that plays recipes from Gameplay Cues (3.5 Adding the GAS add-on).

FeelKit is sold in two editions. **FeelKit Lite** is free and contains the recipe editor, twelve steps, the comfort layer with its menu, the Feel Switch and a selection of library recipes. **FeelKit Pro** contains everything described in this manual. [Chapter 15, Lite and Pro](#) compares the two in full.

Playable builds of the four demo levels show FeelKit in finished scenes, with a key that turns FeelKit off and on for comparison:

- [Action/RPG](#)
- [Platformer](#)
- [Shooter](#)
- [Horror](#)

1.3 How to use this manual

The manual has two parts:

- **Part One, the guide** (chapters 1 to 15), explains how to work with FeelKit, task by task, in the order in which a project usually uses it. Chapters 2 to 4 are worth reading in full; the others can be read when their topic comes up.
- **Part Two, the reference** (chapters 16 onwards), lists every step, node, component, setting and console command with its default values. It is meant for looking things up, not for reading from start to end. Its entries are generated from FeelKit's source code, so they always match the version they describe.

Finding a topic. The table below points to the section for the most common tasks. In the PDF, the bookmarks panel lists every section, and all references in the text are links.

To do this	Read
Install FeelKit and enable it in a project	see 3.2 Installing from Fab and enabling the plugin
Get a first result in five minutes	see chapter 4, Quick start: your first recipe
Build or change a recipe	see chapter 5, The recipe editor
Play a recipe from a Blueprint or from C++	6.1 Play Feel, and choosing a target
Play recipes from animations, collisions, damage or input without Blueprint wiring	Chapter 6, Playing recipes in your game
Make one recipe serve light and heavy hits	7.1 Parameters and track mappings
Let players reduce shake, flashes or vibration	Chapter 8, Comfort and accessibility
Find out why a recipe did not play	Chapter 10, Debugging and tuning
Use FeelKit in a multiplayer game	Chapter 11, Multiplayer

To do this	Read
Look up what a step, node or setting does	see chapter 16, Step reference, see chapter 17, Blueprint nodes and components, see chapter 18, Project settings and editor preferences
Solve a known problem	Chapter 14, Troubleshooting and questions

Conventions.

- Names shown in the Unreal editor, such as buttons, menus and settings, are printed in **bold**: **Play Feel**, **Project Settings** > **Plugins** > **FeelKit**.
- Names that are code, assets or gameplay tags are printed in a code font: `FR_Impact_HeavyHit`, `Feel.Camera.Shake`, `UFeelRecipe`.
- Features that only FeelKit Pro contains carry a green **Pro** label on their heading, or the word Pro in a table. Everything without the label is in both editions.
- Times are in seconds and distances in centimeters, as in Unreal.

1.4 Getting help

- Discord:** discord.gg/AtJ6RdwaxA, for questions, problems and suggestions.
- Email:** billoue4@gmail.com, which is also the support address in the plugin's details in **Edit** > **Plugins**.

A report is easiest to act on when it names the engine version, the FeelKit version (shown in **Edit** > **Plugins**), what was expected and what happened, and any `LogFeel` lines from the Output Log. [Chapter 14, Troubleshooting and questions](#) answers the most common questions, and [Chapter 10, Debugging and tuning](#) describes the tools that show what FeelKit is doing while the game runs.

2 Core ideas

IN THIS CHAPTER

- 2.1 Recipes, tracks and steps
- 2.2 Channels, and what happens when effects overlap
- 2.3 Intensity: how strong each effect plays
- 2.4 Targets: what a recipe plays on
- 2.5 Parameters, accumulators and sustained recipes, in brief
- 2.6 Events and Feel Maps, in brief
- 2.7 Comfort on one page
- 2.8 Why the editor preview matches the game

2.1 Recipes, tracks and steps

A **recipe** is an asset that defines the feedback for a single moment of play: a hit landing, a heavy landing after a fall, a pickup, a menu button being pressed. A recipe consists of **tracks** arranged on a timeline, in the same manner that a Level Sequence arranges tracks in Sequencer. Each track plays one **step**, which is the effect itself: a camera shake, a screen flash, a sound, a controller vibration or a brief freeze of time. FeelKit Pro has 37 steps and FeelKit Lite has 12 of them; [chapter 16, Step reference](#) describes each step and names its edition.

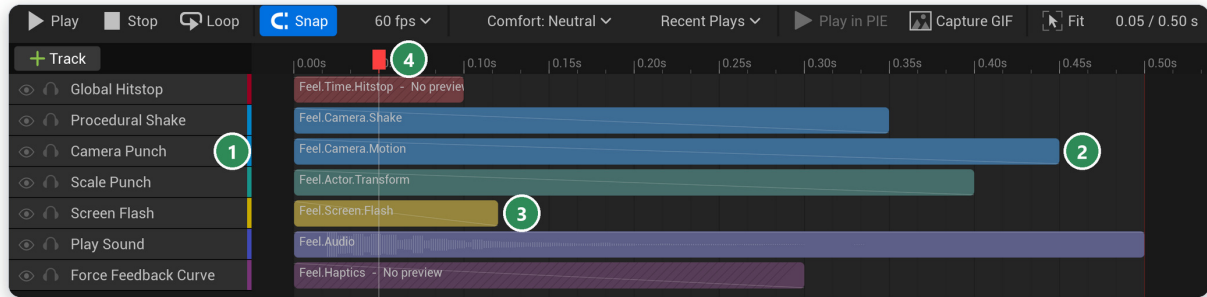


Figure 2.1 The recipe editor on a copy of `FR_Impact_HeavyHit`. 1 the steps, one per track; 2 a track on the timeline; 3 its channel, by name and color; 4 the playhead

When a recipe plays, each track begins at its **Start Time** and runs its step for its **Duration**, both measured in seconds. A new track lasts 0.5 s. A track with a length of 0 is an instant track: steps that act once, such as Play Sound, Spawn Decal or Blueprint Event, can sit on one. The play ends when its last track ends.

A few settings belong to the recipe as a whole:

- **Cooldown** (default 0 s): how long the same recipe must wait before it can play again on the same target.
- **Max Concurrent** (default 0, which means no limit): how many plays of the recipe can run at once on one target. When the limit is reached, a new play does not start and the running ones continue.
- **Default Intensity** (default 1): a multiplier on every play of the recipe.

The full list of recipe and track fields is in [16.1 Recipe fields](#) and [16.2 Track fields](#).

A step computes its output for a given moment from three inputs only: the time within its track, its intensity at that time, and a seed. It retains no state from earlier frames. This property allows the recipe editor to scrub the timeline in either direction and show exactly the frame the game would show at that moment, and it guarantees that the same seed always produces the same shake. Steps that act once, such as playing a sound or calling a Blueprint event, do so when their track starts, and some act again when it stops.

Recipes are timed in real time, which Unreal refers to as unscaled time. Consequently, a hitstop that slows the world does not slow the recipe that caused it: a shake continues to move during the freeze, and the recipe ends when its timeline specifies. FeelKit's runtime performs work only while a recipe plays or while a change it made still has to be restored; when nothing is playing, it does not tick at all.

2.2 Channels, and what happens when effects overlap

Every track belongs to a **channel**, a gameplay tag under `Feel`. A new track receives its step's default channel, for example `Feel.Camera.Motion` for Camera Punch, and you can change it on the track. The

channel colors the track on the timeline, groups the recipe editor's **Intensity** tab, and decides which comfort group scales the track (see [2.7 Comfort on one page](#)).

Channel	Used by	Comfort group
<code>Feel.Camera.Shake</code>	Procedural Shake	Camera Shake
<code>Feel.Camera.Motion</code>	Camera Punch, FOV Kick, Camera Roll, Camera Zoom, Look-at Nudge	Camera Motion
<code>Feel.Time.Hitstop</code>	Global Hitstop, Actor Hitstop	Hitstop and Slow-mo
<code>Feel.Time.SlowMo</code>	Slow-mo Ramp	Hitstop and Slow-mo
<code>Feel.Screen.Flash</code>	Screen Flash	Flashes
<code>Feel.Screen.Distortion</code>	Vignette Pulse, Chromatic Aberration, Post Process Material Pulse	Screen Distortion
<code>Feel.Screen.Color</code>	Desaturate, Color Tint	Screen Distortion
<code>Feel.Screen.Fade</code>	Screen Fade	none (Master only)
<code>Feel.Actor.Transform</code>	Scale Punch, Squash and Stretch, Mesh Wobble	none (Master only)
<code>Feel.Actor.Material</code>	Material Parameter Pulse, Hit Flash	none (Master only)
<code>Feel.Actor.Light</code>	Light Flash	Flashes
<code>Feel.Audio</code>	Play Sound	none (Master only)
<code>Feel.Audio.Mix</code>	Sound Class Duck, Pitch Bend, Low-pass Sweep	none (Master only)
<code>Feel.Haptics</code>	Force Feedback Curve, Haptic Pattern	Haptics
<code>Feel.UI</code>	Widget Punch, Widget Shake, Widget Flash, Number Pop	none (Master only)
<code>Feel.Spawn</code>	Spawn Decal, Spawn Particle	none (Master only)
<code>Feel.Meta.Event</code>	Blueprint Event	none (Master only)
<code>Feel.Meta.Recipe</code>	Play Recipe, Random Choice	none (Master only)

The comfort groups in this table are the project defaults. You can change the mapping in the project settings (see [18.5 Comfort](#)).

2.2.1 Inside one play

The tracks of a single play are combined before any output reaches the screen. Movement adds up: camera location and rotation offsets, field of view changes, and the scale and motion of the target are summed across the play's tracks, so two camera tracks placed on top of each other move the camera by both. Everything else takes the strongest track: the strongest flash, fade and tint, the strongest weight for each post-process value, and the strongest value for each controller motor.

2.2.2 Between separate plays

When several plays run concurrently, for example two hits a few frames apart, FeelKit resolves their results separately for each kind of output:

Output	Rule
Camera location, rotation and field of view of one player	Strongest Wins by default: for each of the three, the strongest play wins. With Additive Capped the plays add up and the sum is limited by caps (30 cm, 8° and 15° by default). Set in the project settings (see 18.2 Camera).
Time dilation of the world or of one actor	One request wins per clock: the highest Priority , and among equal priorities the slowest dilation. When no request remains, the clock gets back the dilation it had before FeelKit changed it.
Flash, fade, tint and each post-process value	The strongest wins.
Controller vibration of one player	The strongest value wins for each motor.
Scale and motion of one component	Changes add up.
Material parameters	Scalar changes add up; for colors the strongest wins.
Sound classes	The lowest volume, the pitch farthest from normal and the lowest filter cutoff win.
Widgets	Movement, scale and rotation add up; for the tint the strongest wins.

Under Strongest Wins, two plays that push the camera in different directions do not blend: whichever is stronger in a given frame determines that frame. If two camera recipes often overlap and aim different ways, either give one of them less camera motion or switch to Additive Capped.

2.3 Intensity: how strong each effect plays

At every moment, each track has an intensity, and its step scales the effect by that value. A Camera Punch with a **Location Punch** of 10 cm moves the camera 10 cm at intensity 1 and 5 cm at intensity 0.5. Intensity above 1 is allowed and makes most effects larger; values that cannot exceed a limit, such as a flash's opacity or a blend weight, are clamped.

A track's intensity is the product of the following factors:

Part	Where it comes from	Default
Call intensity	The Intensity pin of Play Feel and the other play nodes. A Feel Map row multiplies it by its Intensity Scale .	1
Default intensity	The recipe's Default Intensity .	1

Part	Where it comes from	Default
Intensity curve	The track's Intensity Curve , read over the track from 0 at its start to 1 at its end.	A new track fades from 1 to 0; tracks of steps that shape their own motion start flat at 1. An empty curve counts as 1.
Parameter mappings	One multiplier for each of the track's parameter mappings (see 2.5 Parameters, accumulators and sustained recipes, in brief).	none
Random intensity	A value between the track's Random Intensity Min and Max, picked once per play.	1
Comfort	The player's scale for the track's comfort group, times their Master scale (see 2.7 Comfort on one page).	1
Global scale	The console variable <code>feel.GlobalScale</code> (see 19.2 <code>feel.GlobalScale</code>).	1

Two more factors apply only at times: during a blend out after **Stop Feel** the intensity fades to 0 over **Blend Out Time**, and a flash above the rate the flash limiter allows is softened or suppressed.

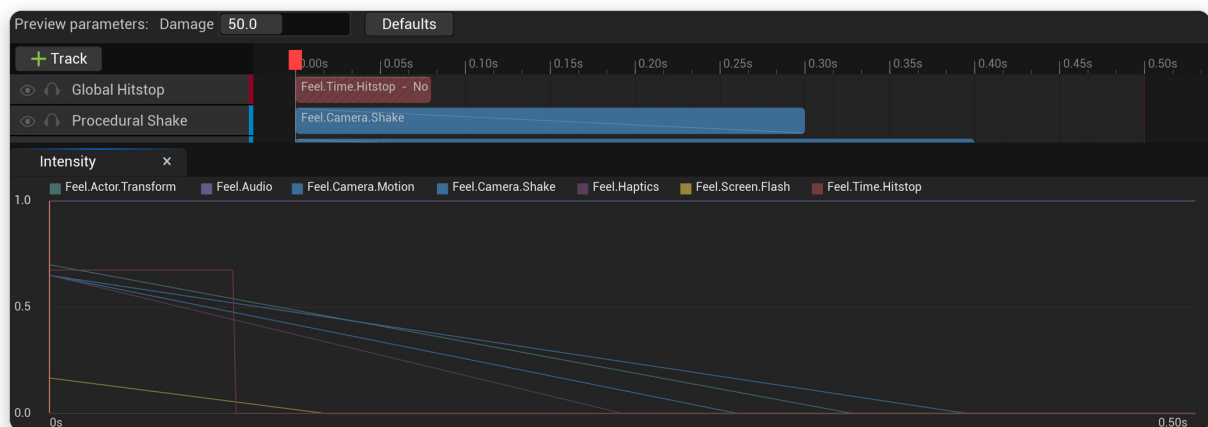


Figure 2.2 The Intensity tab of `FR_Impact_ScalableHit`, one line per channel

The recipe editor's **Intensity** tab draws the combined intensity of each channel across the recipe, with the preview's comfort setting applied.

2.4 Targets: what a recipe plays on

Every play has a **target**, created with one of the Make Feel Target nodes (see 17.2 Targets):

Target	Made with	What it is
Actor	Make Feel Target from Actor	An actor in the level, such as a character, an enemy or a prop.

Target	Made with	What it is
Scene Component	Make Feel Target from Component	One component, such as a weapon mesh.
World Location	Make Feel Target at Location	A point in the world, for effects with no actor, such as an explosion.
Local Player Camera	Make Feel Target from Local Player Camera	The camera of a local player, by player index.
Widget	Make Feel Target from Widget	A UMG widget, for UI steps.
None	the default Feel Target	Nothing in particular.

The target determines where each kind of effect is applied:

- **Camera, screen and controller effects** go to the local player the target belongs to: that player's own pawn, controller, or something they own. A target that belongs to no player, such as an enemy or a prop, sends them to the first local player. A target that belongs to a player on another machine in a networked game sends them nowhere on this machine; that player's own machine shows them.
- **Actor effects** (scale, motion, materials, lights, actor time, attached sounds and Blueprint events) apply to the target actor or component. On a character, whose root is a collision capsule, motion and scale go to the visible mesh instead, so collision and movement never change.
- **Widget steps** apply only to a widget target.
- **Spawned effects** appear at the target's location, or at a location passed with the play.

In FeelKit Pro, a play can also carry a second actor, the **instigator**, for example the player whose attack caused the hit. Tracks whose **Applies To** is set to Instigator play their actor effects on that actor, and send their camera, screen and controller effects to that actor's player. One recipe can then shake the enemy that was hit and kick the camera of the player who hit it. Tracks set to Instigator are skipped when the play has no instigator. The instigator is passed in the play context (see [17.3.1 Feel Play Context structure](#)).

2.5 Parameters, accumulators and sustained recipes, in brief PRO

These three features allow a single recipe to serve many situations. [Chapter 7, Parameters, context and variation](#) explains them in full.

Parameters are named numbers a recipe accepts each time it plays, such as Damage or Speed. The recipe declares each one with a **Default Value**, a **Min Value** and a **Max Value**; the game passes values with **Play Feel with Context**. A track follows a parameter through a **parameter mapping**: a curve that turns the parameter, read from 0 at its minimum to 1 at its maximum, into an intensity multiplier. The library recipe `FR_Impact_ScalableHit` declares Damage from 0 to 100 (default 50).

Most of its tracks play at 30 to 40% of their strength at Damage 0 and at full strength at Damage 100, while its flash stays faint below Damage 60. One recipe then covers light and heavy hits. A parameter named **Distance** is special: when the game does not pass it, FeelKit fills in the distance in centimeters from the target to the nearest local player camera. **Set Feel Parameter** changes a parameter while the recipe plays.

Accumulators are named values that build up each time the game adds to them and fall back toward 0 over time, defined in the project settings (see 18.3 [Playback](#)) with a **Max Value**, a **Decay Per Second** and a **Decay Delay**. A recipe parameter can read an accumulator whenever the game does not pass the parameter itself: FeelKit looks for the value stored for the play's target, then for its instigator, then for the global value. Repeated plays then grow stronger while they come quickly and calm down when they stop. The library recipe `FR_Reward_ComboStep` has a Combo parameter from 0 to 10 that reads an accumulator named Combo. FeelKit defines no accumulators, so add Combo to the project settings before using that recipe.

A **sustained** recipe keeps playing while a condition lasts. It has a looping region between **Sustain Start** and **Sustain End**: time loops there until the play is released, then the rest of the recipe plays as its ending. The library recipe `FR_Power_ChargeUp` loops between 0.35 s and 0.95 s while a charge is held. **Release Feel** releases a play, and so do the end of a **Play Feel (Window)** notify and a Feel Input binding set to end with its input. **Stop Feel** stops a play whether it loops or not.

2.6 Events and Feel Maps, in brief PRO

Gameplay code need not refer to recipes directly. It can instead send a **Feel Event**: a gameplay tag that says what happened, such as `Feel.Event.Hit.Landed`, with a target and a play context. A **Feel Map** asset then decides which recipe plays for that event. Changing how a hit feels therefore becomes a change to a single Feel Map row, while the gameplay code remains unchanged.

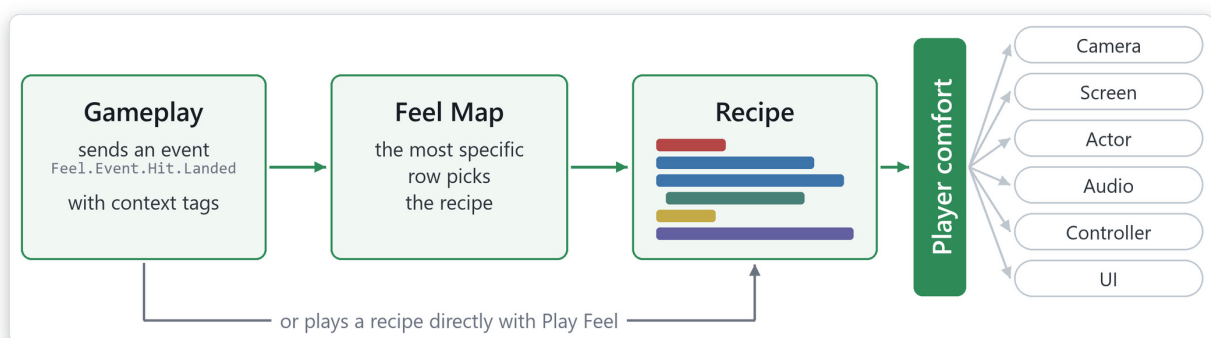


Figure 2.3 Diagram: a Feel Event goes to a Feel Map, the map picks a recipe, and the recipe's tracks reach the camera, the screen, the actor, audio, the controller and the UI

Each row of a Feel Map names an **Event**, optional **Required Tags**, the **Recipe** to play, an **Intensity Scale** and a **Priority**. A row answers its event and every event below it, and applies only when the play context's **Context Tags** contain all its required tags. When several rows match, the row with the

more specific event wins, then the one with more required tags, then the higher priority, then the row listed first. Two rows for the same event can then pick a heavy or a light variant from a context tag, with no branching in Blueprint.

FeelKit reads the Feel Maps listed in the project settings (see [18.4 Events](#)). A Feel Trigger component on the target can hold more maps, which are checked first. FeelKit defines four starter events: `Feel.Event.Hit.Landed`, `Feel.Event.Hit.Received`, `Feel.Event.Hurt` and `Feel.Event.Death`; projects add their own tags as needed. When no row matches an event, nothing plays and a warning in the log names the event.

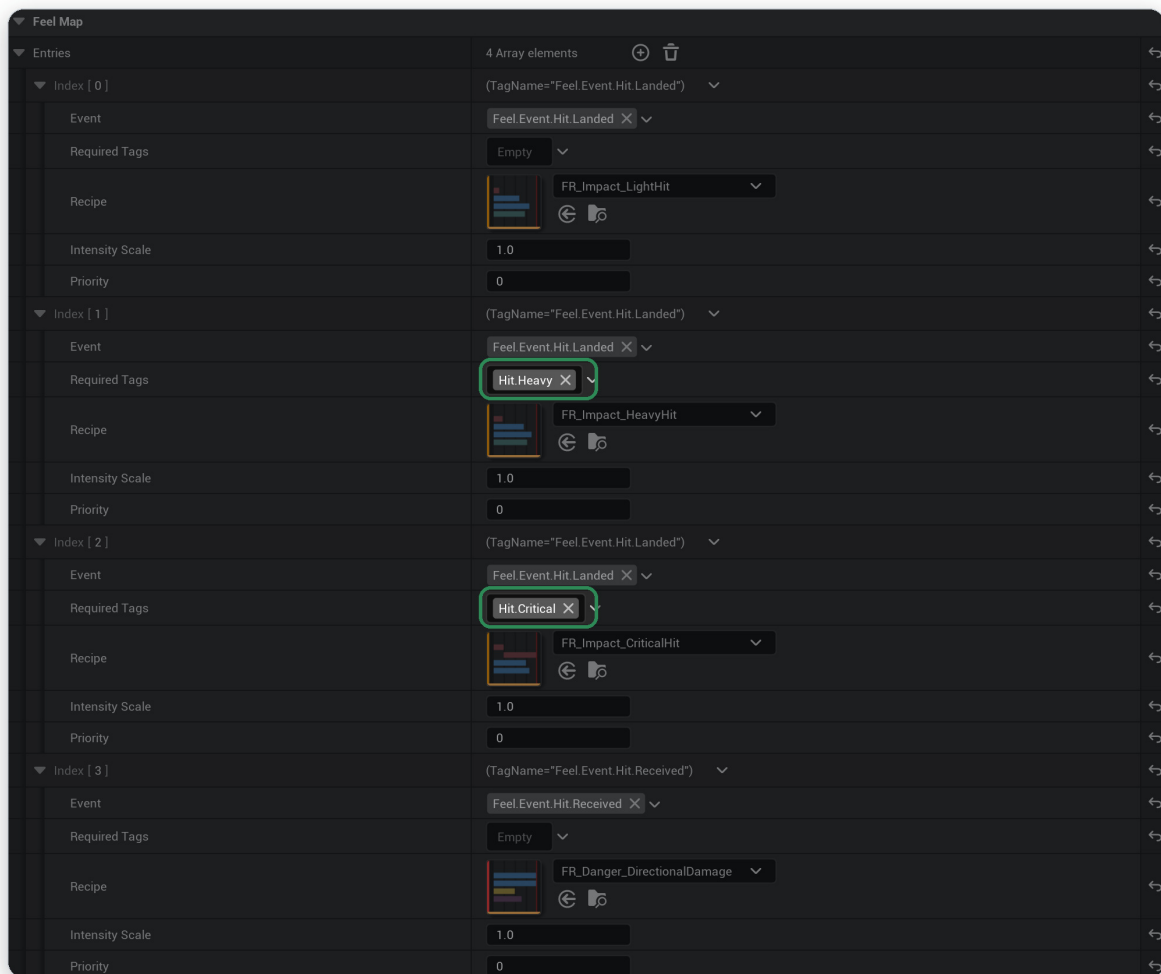


Figure 2.4 A Feel Map in its editor with four rows for `Feel.Event.Hit.Landed` and `Feel.Event.Hit.Received`, two of them with Required Tags

Events are sent by the **Send Feel Event** node, and can also come from animation notifies, the Feel Trigger component, the Feel Input component and gameplay cues. [6.5 Events and Feel Maps](#) shows each of these.

2.7 Comfort on one page

Screen shake, flashes and slowed time can cause discomfort or nausea for some players, and some players cannot use controller vibration. FeelKit therefore maintains comfort settings for each local player and applies them to every recipe, both in the game and in the editor preview.

Scales. Each player has a Master scale and six group scales, each from 0 (off) to 1 (full strength):

Camera Shake, Camera Motion, Flashes, Hitstop and Slow-mo, Screen Distortion and Haptics.

A track's comfort scale is its group's scale times Master. The project settings map channels to groups (see [18.5 Comfort](#)); channels that are not mapped, such as audio and actor effects, are scaled by Master only. A track whose comfort scale is 0 does not play at all, unless it is essential (below).

Presets. FeelKit has three built-in presets besides the project defaults, with values you can change in the project settings:

Preset	What it changes
Reduced Motion	Camera Shake 0.25, Camera Motion 0.25, Screen Distortion 0.5, camera roll off, field of view changes limited to 40° per second
Reduced Flashing	Flashes 0.2, at most 1 flash per second, extra flashes suppressed
No Haptics	Haptics 0

Custom presets are **Feel Comfort Preset** assets.

Essential tracks. A track marked **Essential** carries information the player needs, for example a flash that says the player was hit. When the player turns its group off, its **Substitute Step** plays instead, scaled by the substitute's own group; the library recipe [FR_Dread_JumpScare](#) has an essential Screen Flash with a Vignette Pulse as its substitute. An essential track without a substitute never drops below its **Essential Floor**. The floor never applies on Camera Shake and Camera Motion: there the player's setting always wins, and at 0 those tracks play no motion.

Flash limiter. Each player's settings limit how many flash tracks may start per second: 3 by default, with extra flashes softened to 0.3 of their strength. The limiter reduces risk but does not certify a game as safe for photosensitive players; games should also be tested with a dedicated analysis tool.

Motion comfort. A player can turn camera roll off for every recipe, and can limit how fast FeelKit changes the field of view, in degrees per second.

Engine effects. By default a player's Camera Shake scale (times Master) also applies to camera shakes the game plays through the engine, and their Haptics scale (times Master) to all controller vibration. Two project settings turn this off.

Saving. Settings load when a local player is created, before any recipe plays, and save automatically when they change: one save game slot per local player, named [FeelComfort_0](#), [FeelComfort_1](#) and so on. You can route loading and saving to your own save system instead.

Players change their settings through your game's options menu, which calls the comfort nodes ([see 17.6 Comfort](#)). In the recipe editor, the **Comfort** dropdown previews a recipe with a preset applied. [Chapter 8, Comfort and accessibility](#) covers comfort in full.

2.8 Why the editor preview matches the game

The recipe editor does not imitate the game; both run the same code. The steps compute their output, and a shared evaluator applies the recipe's timing, intensity and comfort rules to it. Only the final stage, which delivers the output, differs. In the game, a camera modifier on each local player's camera manager applies camera output, a view extension draws flashes and fades, and actor, audio and controller output goes to the real objects. In the editor, the same output goes to the preview's camera, preview mesh and post-process settings. Actor effects are applied and put back by the same code in both.

Because steps retain no state between frames, the preview can jump to any point in time and show the frame the game would show there. Scrubbing backwards is therefore as exact as playing forwards.

Some effects need a running game and cannot be shown in the preview: time dilation (Global Hitstop, Actor Hitstop, Slow-mo Ramp), controller vibration (Force Feedback Curve, Haptic Pattern), Blueprint Event, Light Flash, the widget steps and Number Pop. Their tracks carry a **No preview** label; the recipe's other tracks still preview normally. To judge them, start Play In Editor and press **Play in PIE** in the recipe editor, which plays the recipe on the player's pawn. Changes made in the editor apply to the next play without restarting Play In Editor.

3 Installing FeelKit

IN THIS CHAPTER

- 3.1 Requirements
- 3.2 Installing from Fab and enabling the plugin
- 3.3 What FeelKit switches on for you
- 3.4 Showing FeelKit content in the Content Browser
- 3.5 Adding the GAS add-on
- 3.6 Blueprint-only projects and packaging
- 3.7 Moving a project from Lite to Pro

FeelKit is an engine plugin: it is installed once per engine version and then enabled in each project that uses it. Two tasks need Visual Studio: adding the GAS add-on and packaging a Blueprint-only project.

► **Video: Install FeelKit and play your first recipe (YouTube, about 6 minutes)**

The link is added before release.

3.1 Requirements

Item	Requirement
Unreal Engine	5.6, 5.7 or 5.8. Fab provides a separate download for each version.
Platforms	Windows 64-bit is built and tested. The runtime modules also list Mac, Linux, Android and iOS, and the editor module Mac and Linux, but these platforms have not been tested.
Project type	Blueprint and C++ projects.
Visual Studio	Not needed to install FeelKit or to work with it in the editor, since Fab supplies ready-built binaries. Needed to package a Blueprint-only project (see 3.6 Blueprint-only projects and packaging), to use the GAS add-on (see 3.5 Adding the GAS add-on) and, as always, to build a C++ project.

Item	Requirement
Other plugins	Pro: Niagara and Enhanced Input, both part of the engine; FeelKit enables them itself (see 3.3 What FeelKit switches on for you). Lite needs no other plugin.

3.2 Installing from Fab and enabling the plugin

1. In the Epic Games Launcher, open **Unreal Engine > Library** and scroll to **Fab Library**.
2. Search for FeelKit and click **Install to Engine**. Choose the engine version and click **Install**. The launcher places the plugin in the engine's `Engine/Plugins/Marketplace` folder, where every project that uses this engine version can find it.
3. Open the project and choose **Edit > Plugins**. Type **FeelKit** in the search box. The entry is called **FeelKit** for Pro and **FeelKit Lite** for Lite.
4. Tick **Enabled** on the entry and click **Restart Now**.

Install one edition per engine version. Both editions are the same plugin, FeelKit, as far as Unreal is concerned, so an engine with both installed finds two plugins of one name (see 3.7 Moving a project from Lite to Pro).

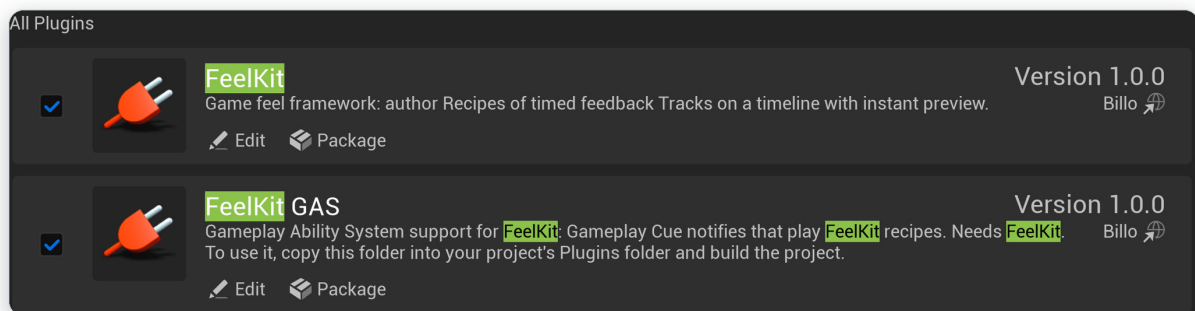


Figure 3.1 Edit > Plugins with FeelKit found by the search box. FeelKit GAS appears only in a project that has the add-on

After the restart, the **Tools** menu contains the FeelKit windows, the Content Browser can create recipes (right-click > **FeelKit > Feel Recipe**), and the Blueprint action menu lists the **Feel** nodes. Enabling adds FeelKit to the `Plugins` list of the project's `.uproject` file; nothing else in the project changes.

An update from Fab replaces the installed plugin in the engine folder. Anything edited inside that folder, such as FeelKit's own comfort menu, is replaced as well; Chapter 8, Comfort and accessibility describes how to restyle the comfort menu safely.

3.3 What FeelKit switches on for you PRO

FeelKit Pro depends on two plugins that ship with the engine, and enabling FeelKit enables them too. FeelKit Lite has neither feature and enables no other plugin.

Plugin	Used for
Niagara	The Spawn Particle step.
Enhanced Input	The Feel Input component, which plays recipes from input actions.

A project that already uses both notices no change. A project that has disabled either one will find it enabled again, because FeelKit cannot load without it.

3.4 Showing FeelKit content in the Content Browser

The Content Browser hides plugin content until it is asked to show it:

1. In the Content Browser, click **Settings**, the gear icon at the right end of the Content Browser's toolbar, and tick **Show Plugin Content**.
2. In the folder tree, open **All > Plugins > FeelKit Content**.

FeelKit Content (**FeelKit Lite Content** in Lite) holds these folders:

Folder	Contents
Library	The ready-made recipes, one folder per feeling (Chapter 9, The recipe library and browser): 38 in Pro, 11 in Lite. Read-only; a project uses its own copies.
Demos	Pro. The recipes of the Shooter, Horror and Platformer demo levels (Chapter 13, Demo levels).
Samples	The sounds and materials the library and demo recipes use. Pro: 43 sounds, one sound attenuation asset and three materials. Lite: the 7 sounds its library recipes use.
UI	The comfort menu <code>WBP_FeelComfortMenu</code> and the short recipes its sliders play as previews (Chapter 8, Comfort and accessibility).

`Credits.md` in the plugin folder lists every sample sound with its source and license.

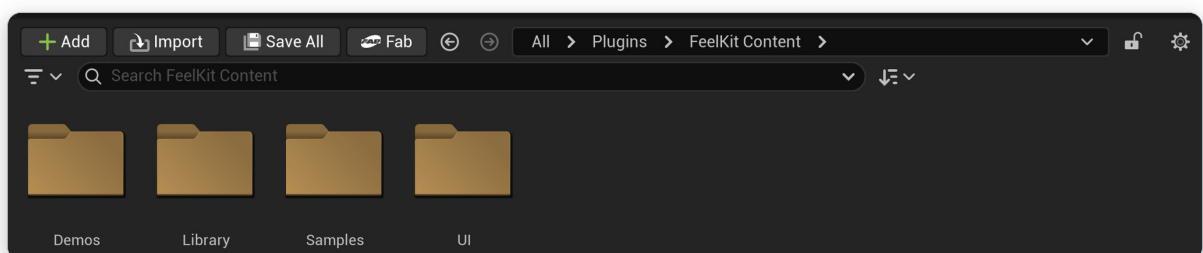


Figure 3.2 FeelKit Content in the Content Browser (Pro): Demos, Library, Samples and UI

3.5 Adding the GAS add-on PRO

Support for the Gameplay Ability System is a separate plugin, **FeelKit GAS**, inside FeelKit's **Extras** folder. It stays out of projects that do not use GAS, so they never load the Gameplay Abilities plugin because of FeelKit. It is added per project, and it compiles with the project, so Visual Studio 2022 with the C++ workload is required.

1. Close the Unreal Editor.
2. In File Explorer, open the engine's **Engine/Plugins/Marketplace** folder, then the FeelKit folder inside it (the one that contains **FeelKit.uplugin**), then **Extras**.
3. Copy the **FeelKitGAS** folder into the project's **Plugins** folder, creating **Plugins** if it does not exist. The result is **<Project>/Plugins/FeelKitGAS/FeelKitGAS.uplugin**.
4. Open the project. Unreal reports that the FeelKit GAS module is missing or was built with a different engine version and offers to rebuild it; click **Yes**. In a C++ project, building the project from Visual Studio does the same.

Location	Path
Copy from	<Engine>/Engine/Plugins/Marketplace/<FeelKit folder>/Extras/FeelKitGAS
Copy to	<Project>/Plugins/FeelKitGAS

FeelKit GAS is enabled as soon as it is in the project, and it enables Gameplay Abilities. The Gameplay Cue notifies it adds are described in [6.9 Gameplay Ability System cues](#). To remove it, delete the **Plugins/FeelKitGAS** folder while the editor is closed; FeelKit itself is unaffected.

3.6 Blueprint-only projects and packaging

A Blueprint-only project can use FeelKit in the editor without any extra software. Packaging is different: when a project enables a code plugin that the engine does not enable by default, Unreal builds a game executable for that project instead of using the ready-made one, and treats the project as a code project for packaging. Every code plugin from Fab has this effect, FeelKit included.

To package a Blueprint-only project that uses FeelKit:

1. Install Visual Studio 2022 with the **Game development with C++** workload, as described in Epic's guide to setting up Visual Studio for Unreal Engine.
2. Package the project as usual (**Platforms > Windows > Package Project**). The project stays a Blueprint project; no C++ files are added to it.

The packaged game contains FeelKit's runtime modules only. The editor module, the Recipe Browser and the other editor tools are not included.

3.7 Moving a project from Lite to Pro

FeelKit Lite and FeelKit Pro use the same plugin name, the same modules and the same class names, and Lite's recipes and assets keep the same paths in Pro. A project made with Lite therefore opens in Pro unchanged: every recipe keeps its tracks, steps and values, and every Blueprint node stays connected.

1. Close the Unreal Editor.
2. In the Epic Games Launcher, open **Unreal Engine > Library**, find FeelKit Lite under the engine version's installed plugins and remove it.
3. Install FeelKit Pro to the same engine version ([see 3.2 Installing from Fab and enabling the plugin](#)).
4. Open the project. It uses Pro from now on; the Pro features appear in the editor, and the Pro fields of existing recipes show their default values.

The other direction is limited. Lite keeps the Pro fields of a recipe, such as parameters and random ranges, in the asset without showing them, so those values survive a round trip. A track whose effect exists only in Pro, however, has no step in Lite; saving such a recipe in Lite loses that step for good.

4 Quick start: your first recipe

IN THIS CHAPTER

- 4.1 Pick a recipe from the library
- 4.2 Preview it without pressing Play
- 4.3 Play it from a Blueprint
- 4.4 Compare with the feel off
- 4.5 Where to go next

This chapter takes one recipe from the FeelKit library into a project, previews it in the recipe editor and plays it on the player's character. It requires no C++ and takes about five minutes. The steps assume that FeelKit is enabled ([see 3.2 Installing from Fab and enabling the plugin](#)) and that the project was created from Unreal's Third Person template; any project with a playable character works in the same way.

► **Video: Install FeelKit and play your first recipe (YouTube, about 6 minutes)**

The link is added before release.

4.1 Pick a recipe from the library

FeelKit includes a library of ready-made recipes, grouped by the feeling they produce: Impact, Weight, Power, Speed, Reward, Danger, Dread, Denial and Interface. Library recipes are read-only, so that an update of FeelKit can replace them without affecting a project; a project always works with its own copy. This chapter uses `FR_Impact_HeavyHit`, a heavy blow that combines a short freeze, a camera shake, a camera punch, a scale punch, a flash, a sound and a controller rumble.

1. In the Content Browser, click **Settings** (the gear icon at the right end of its toolbar) and turn on **Show Plugin Content**. FeelKit's content then appears under **Plugins > FeelKit Content**.
2. Open **FeelKit Content > Library > Impact** and select `FR_Impact_HeavyHit`.

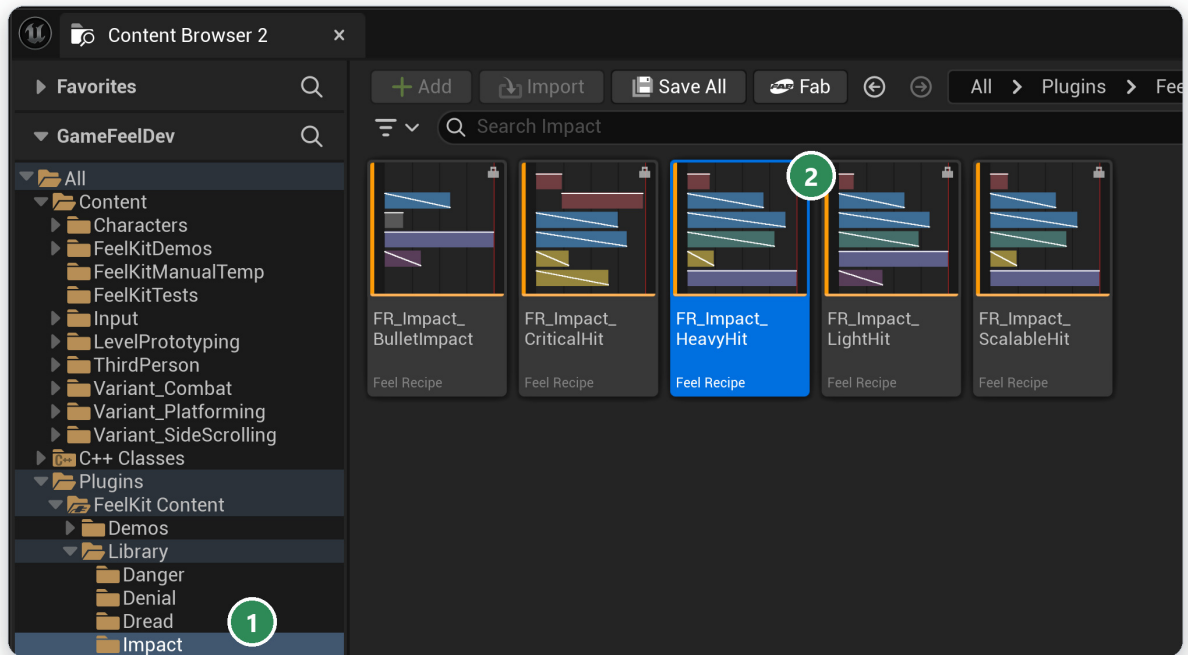


Figure 4.1 The Impact folder of the FeelKit library in the Content Browser. 1 the folder; 2 `FR_Impact_HeavyHit`

3. Double-click the recipe. The recipe editor opens it with a banner across the timeline: "Library recipe (read-only). Copy it to your project to edit."
4. Click **Copy to Project**. In the save dialog, choose a folder of the project, keep the suggested name `HeavyHit` and click **Save**. The copy opens in the recipe editor and can be changed freely; its **Based On** field records the library recipe it came from.

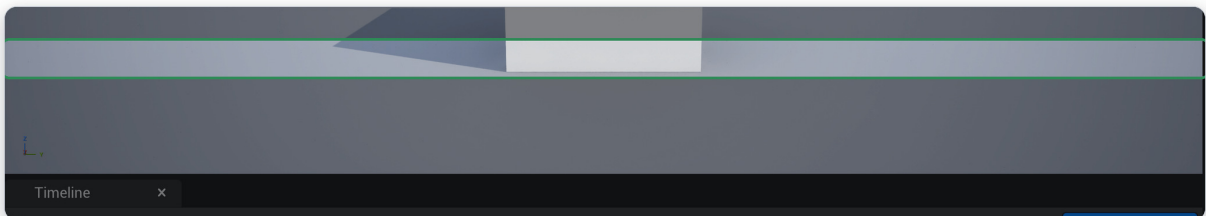


Figure 4.2 A library recipe opens read-only. Copy to Project creates an editable copy in the project

Note In FeelKit Pro, **Tools > FeelKit Recipe Browser** presents the same library with filters by feeling, genre and channel, and plays each recipe when the pointer rests on it. [Chapter 9, The recipe library and browser](#) describes it.

4.2 Preview it without pressing Play

The recipe editor plays a recipe on its own preview camera and mesh, using the same code as the game (see [2.8 Why the editor preview matches the game](#)). Most of a recipe can therefore be judged without starting the game.

1. In the **Timeline** toolbar, click **Play**, or press Space while the timeline has focus. The preview shows the camera shake, the camera punch, the scale punch and the flash, and the sound plays.
2. Click **Loop** to repeat the recipe while observing it.
3. Drag the red playhead along the ruler to step through the recipe. Because steps retain no state between frames, scrubbing backwards shows exactly the same frames as playing forwards.

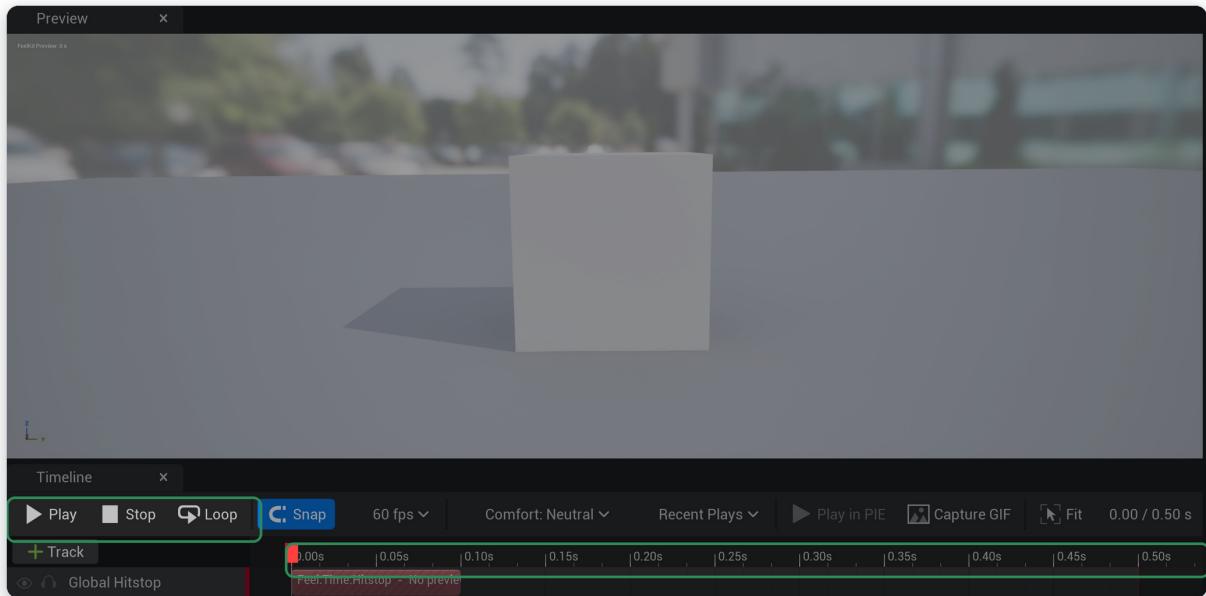


Figure 4.3 The recipe editor's preview above the Timeline toolbar. Play, Stop and Loop on the left; the ruler for scrubbing on the right

Two tracks of **HeavyHit** carry the label **No preview**: Global Hitstop, which slows the game's time, and Force Feedback Curve, which drives the controller. Both act only in a running game, which the next section starts.

4.3 Play it from a Blueprint

A recipe plays when the game calls the **Play Feel** node. The following steps play **HeavyHit** on the player's character whenever the 1 key is pressed.

1. Open the character Blueprint, for example **BP_ThirdPersonCharacter**, and its **Event Graph**.
2. Right-click an empty area of the graph, type **1** and choose **Input > Keyboard Events > 1**.
3. Drag from the **Pressed** pin, type **Play Feel** and choose **Play Feel** from the **Feel** category.
4. On the Play Feel node, set **Recipe** to **HeavyHit**.
5. Drag from the **Target** pin and choose **Make Feel Target from Actor**. Right-click the graph, choose **Get a reference to self**, and connect it to the **Actor** pin.
6. Compile and save the Blueprint.

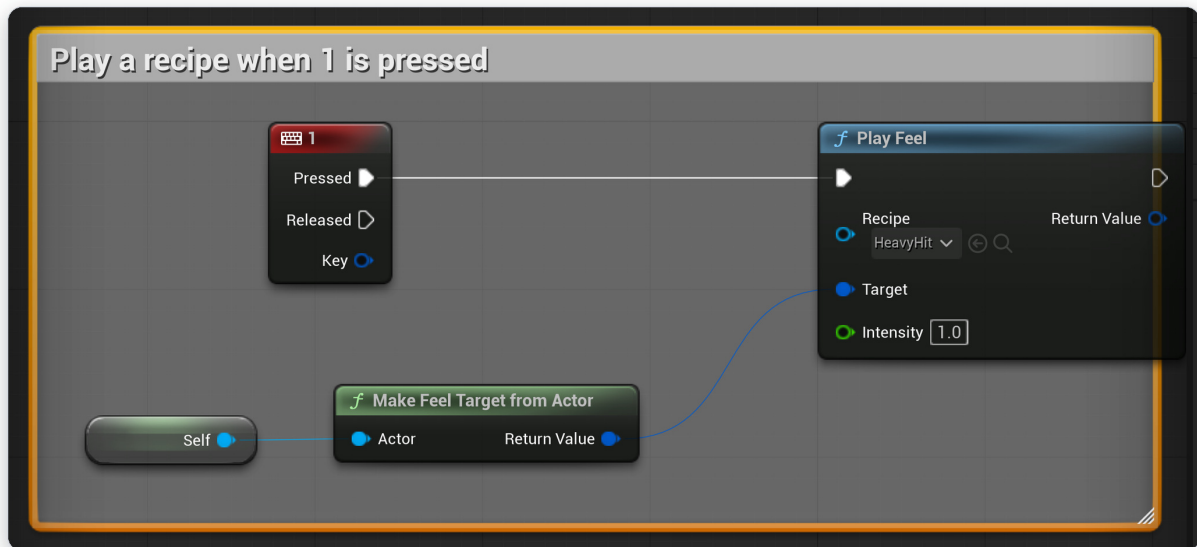


Figure 4.4 The Event Graph after step 5: the 1 key plays HeavyHit, with the character itself as the target

Start Play In Editor and press 1. The recipe plays on the character: the camera kicks and shakes, the screen flashes, the character's mesh punches in scale, time freezes briefly and a connected controller rumbles.

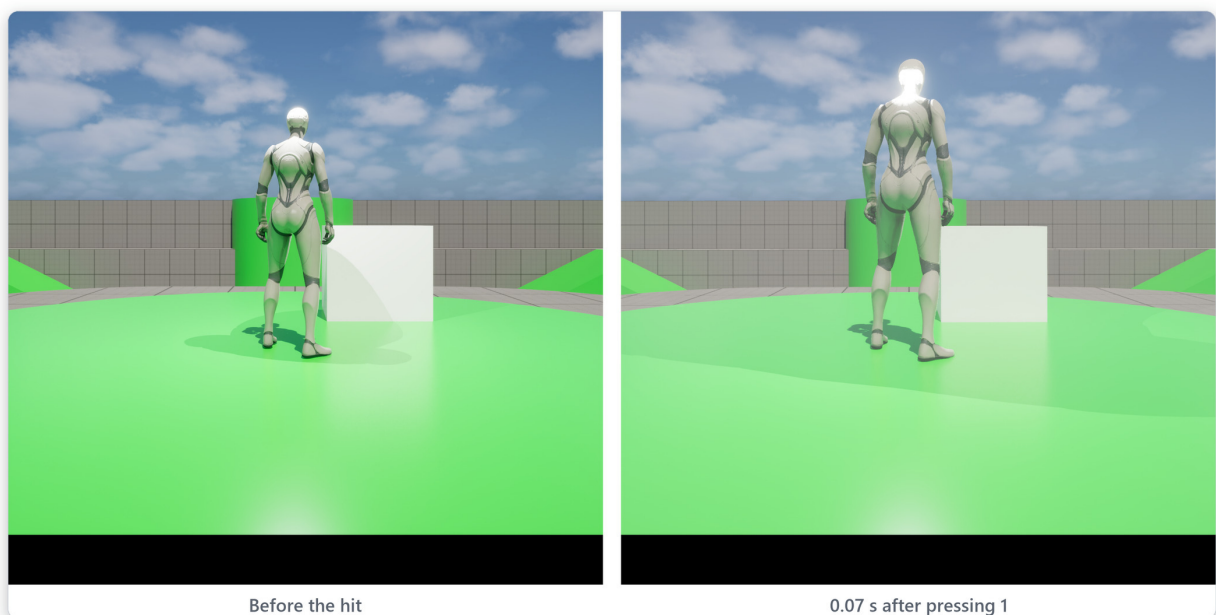


Figure 4.5 The same view before the hit and 0.07 s after pressing 1, when the camera punch and the scale punch are at their strongest. Motion blur was turned off for this picture

The target decides where each effect is applied (see [2.4 Targets: what a recipe plays on](#)). Camera, screen and controller effects reach the player who owns the target, here the player controlling the character. Actor effects apply to the character; because a character's root is its collision capsule, FeelKit moves and scales the visible mesh instead, so collision and movement are never affected.

A keyboard event in the character Blueprint is a quick way to try a recipe. In a finished game, recipes are played from the game's own events, such as a hit being registered or a landing; [Chapter 6, Playing recipes in your game](#) describes the options, including triggers that need no Blueprint at all.

4.4 Compare with the feel off

The **Feel Switch** turns FeelKit off and on while the game runs, so that the difference can be seen and felt directly. It is useful for tuning, for playtests and for showing a game to others.

1. Open **Window > Place Actors**, search for **Feel Switch** and drag it into the level.
2. Start Play In Editor. A start card explains the switch, and a **FEEL: ON** badge appears in the top right corner, with the key for the comfort settings below it.



Figure 4.6 The Feel Switch at the start of play: the start card and the FEEL: ON badge

3. Press 1 to play the recipe, then press Tab (View / Share on a controller). The badge changes to **FEEL: OFF**, and pressing 1 now shows the game without FeelKit. Press Tab again to turn FeelKit back on.



Figure 4.7 The badge while FeelKit is turned off

Turning the feel off stops only FeelKit: effects that the game itself plays are unaffected. Every play session starts with FeelKit on, and the choice is not saved. Pressing O (Menu / Options on a controller) opens the comfort settings menu, which [Chapter 8, Comfort and accessibility](#) describes. The switch keys, the start card and the controls list are set in the Feel Switch's details.

4.5 Where to go next

- [Chapter 5, The recipe editor](#) explains every part of the recipe editor, including how to build a recipe from an empty timeline.
- [Chapter 6, Playing recipes in your game](#) shows how to play recipes from gameplay events, animations and triggers.
- [Chapter 7, Parameters, context and variation](#) covers parameters, context and variation, which let one recipe serve light and heavy hits alike.
- [Chapter 8, Comfort and accessibility](#) covers the comfort settings and the comfort menu for players.
- [Chapter 16, Step reference](#) lists every step with its settings.

5 The recipe editor

IN THIS CHAPTER

- 5.1 Creating and opening recipes
- 5.2 The window: Preview, Timeline, Details, Intensity
- 5.3 The toolbar
- 5.4 Adding, moving and resizing tracks
- 5.5 Shaping intensity with curve keys
- 5.6 Recipe and track settings in Details
- 5.7 The preview: play, loop, scrub, preview mesh, parameter sliders
- 5.8 Sound tracks: waveforms, snapping, tracks made from a sound
- 5.9 Checking comfort, and the intensity graph
- 5.10 Play in PIE, and editing while the game runs
- 5.11 Copy, paste and keyboard shortcuts
- 5.12 Validation messages on save

The recipe editor is where recipes are built and tuned. It shows a recipe's tracks on a timeline, plays the recipe in its own preview while it is being edited, and uses the same evaluation code as the game (see 2.8 [Why the editor preview matches the game](#)), so that what the preview shows is what the player sees. Its parts are described below in the order in which a recipe is usually built.

► **Video: Build a recipe on the timeline (YouTube, about 9 minutes)**

The link is added before release.

5.1 Creating and opening recipes

A recipe is an asset of type **Feel Recipe**. There are three ways to obtain one:

- **A new, empty recipe.** Right-click an empty area of the Content Browser and choose **FeelKit > Feel Recipe**, then name the asset. FeelKit's own recipes begin with **FR_**; any name works.

- **A copy of a library recipe.** Open a recipe under **FeelKit Content > Library** and click **Copy to Project** in the banner above the timeline, as described in [4.1 Pick a recipe from the library](#).
- **A recipe from a template (Pro).** Right-click a folder and choose **Recipe from Template...**, which lists the library recipes and places an editable copy in that folder ([Chapter 9, The recipe library and browser](#)).

Double-click a recipe to open it. The editor opens in its own window, or as a tab of the main window when that is how the editor was last arranged.

5.2 The window: Preview, Timeline, Details, Intensity

The editor has four tabs. Like every Unreal editor tab, each can be moved, docked elsewhere or closed, and **Window** reopens a closed one.

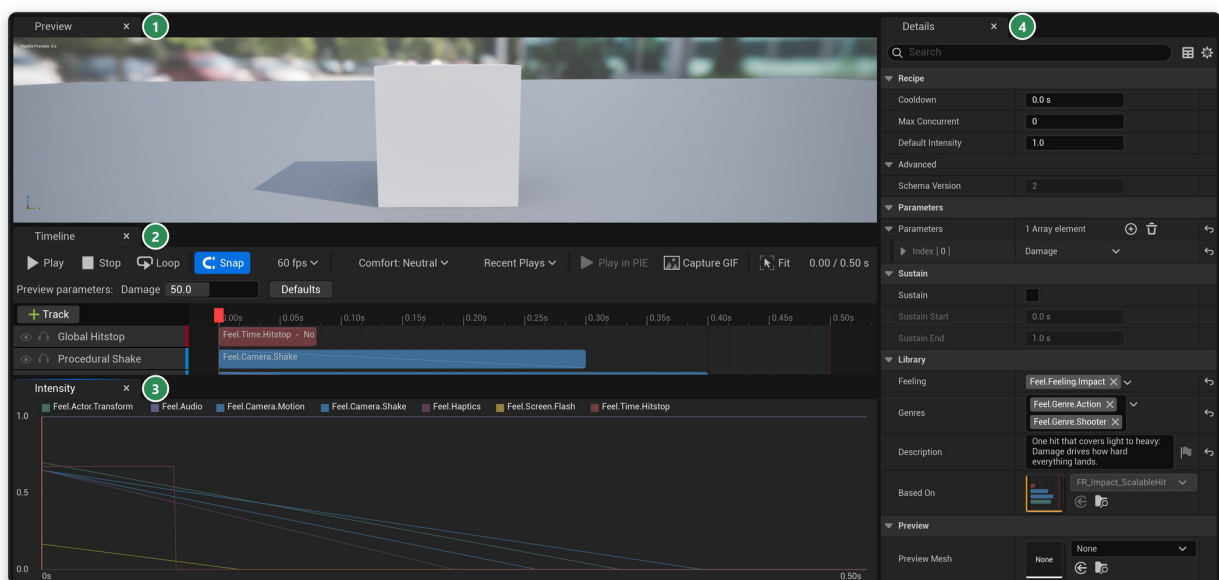


Figure 5.1 The recipe editor. 1 Preview; 2 Timeline; 3 Intensity; 4 Details

Tab	What it shows
Preview	A small scene with a camera and a preview mesh, on which the recipe plays. The line in its top-left corner shows the playhead time.
Timeline	The toolbar, the ruler and one row per track. Selecting a track opens its intensity curve below it.
Intensity	The combined intensity of each channel over the length of the recipe (see 5.9 Checking comfort, and the intensity graph).
Details	The recipe's settings, or the selected track's settings when a track is selected (see 5.6 Recipe and track settings in Details).

The preview uses the standard viewport navigation of the Unreal editor: hold the right mouse button to look around and use W, A, S and D to move.

5.3 The toolbar

The toolbar at the top of the **Timeline** tab controls playback and the preview.

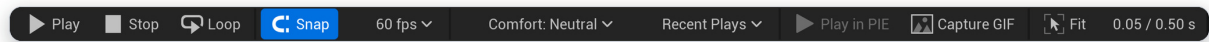


Figure 5.2 The Timeline toolbar

Control	What it does
Play / Pause	Plays or pauses the preview. Space does the same while the timeline has keyboard focus.
Stop	Stops and returns the playhead to the start.
Release	Ends the sustain loop of a sustained recipe, as Release Feel does in the game. Shown only for recipes with Sustain on, and enabled while the loop plays (see 5.7 The preview: play, loop, scrub, preview mesh, parameter sliders).
Loop	Repeats the preview.
Snap	Snaps track edits to frames. Holding Shift while dragging ignores snapping.
Frame rate	The frame rate that snapping uses: 60 fps by default, with common rates and a custom value in the menu. The choice is a personal editor preference (see chapter 18, Project settings and editor preferences).
Comfort	The comfort settings the preview applies (see 5.9 Checking comfort, and the intensity graph).
Recent Plays	PRO Replays a play of this recipe that was recorded during Play In Editor (Chapter 10, Debugging and tuning).
Play in PIE	Plays the recipe in the running Play In Editor session (see 5.10 Play in PIE, and editing while the game runs).
Capture GIF	PRO Records the preview without and with the recipe and saves both side by side as an animated GIF (Chapter 10, Debugging and tuning).
Fit	Fits the whole recipe into the timeline. F does the same.
Time	The playhead time and the length of the recipe, in seconds.

5.4 Adding, moving and resizing tracks

Each row of the timeline is a track. Its header on the left shows the step's name, a mute toggle and a solo toggle; its bar on the right spans the track's start time and length and carries the track's channel. The bar's color follows the channel's family: camera, screen, actor, time, audio, haptics and so on.

Adding a track. Click **+ Track** in the top-left corner of the timeline and choose a step. The menu lists every step by name, including steps written in Blueprint (Chapter 20, Writing your own steps in Blueprint and C++), and has a search box. The new track starts at the playhead, or at 0 s after **Stop**,

and takes the step's default channel and intensity curve. [Chapter 16, Step reference](#) describes every step.

Moving and resizing. Drag a bar to move the track in time. Drag either end of the bar to change its start or its length. With **Snap** on, edits snap to frames at the chosen frame rate, and edges are also drawn to the loud moments of any sound played by another track of the recipe, which makes it easy to align a flash or a hitstop with a sound. Hold Shift while dragging to ignore both kinds of snapping.

The track menu. Right-click a track for these commands:

Command	Shortcut	What it does
Copy	Ctrl+C	Copies the track, to paste into this recipe or any other open recipe.
Copy All Tracks		Copies every track of the recipe.
Paste	Ctrl+V	Pastes copied tracks at the playhead, keeping their timing relative to each other.
Duplicate	Ctrl+D	Adds a copy of the track.
Delete	Delete	Deletes the track.
Move Up, Move Down		Changes the track's row. The order has no effect on the result.
Mute / Unmute		Turns the track off in the preview and in the game (see 5.6 Recipe and track settings in Details).
Solo / Unsolo		Plays only soloed tracks, in the preview. Soloing is an editing aid and does not affect the game.

A Play Sound track adds two commands to this menu, described in [see 5.8 Sound tracks: waveforms, snapping, tracks made from a sound](#).

Every edit can be undone with Ctrl+Z.

Labels on bars. Some bars carry a short label after the channel name. **No preview** marks steps that act only in a running game, such as a hitstop or controller vibration: they are skipped in the preview and play normally in the game. Other labels explain why a track is silent or changed in the current preview, for example **Muted**, **Removed by comfort settings** or **Substitute plays (comfort)**; [Chapter 10, Debugging and tuning](#) lists them all.

5.5 Shaping intensity with curve keys

Each track has an intensity curve that sets how strongly the step plays across the length of the track: 1 is full strength and 0 is silence. The horizontal axis is the track's own time, from its start to its end, so the curve stretches with the track when the track is resized.

Select a track to open its curve in a lane directly below it. The lane repeats its controls as hints:

- Double-click the lane to add a key; double-click a key to delete it.
- Drag a key to move it. Holding Shift ignores snapping.
- Right-click a key to choose how it leads to the next key: **Linear** (a straight line), **Smooth** (an eased curve) or **Constant** (the value is held until the next key). **Delete Key** is in the same menu.

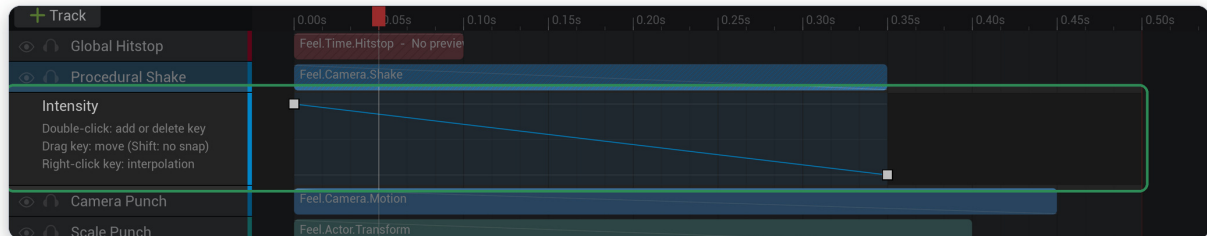


Figure 5.3 A selected Procedural Shake track with its intensity curve in the lane below it

A curve without keys plays at full intensity throughout, and the lane says so. New tracks of most steps start with a curve that falls from 1 to 0, which suits a hit that fades out. Steps that shape their own motion, such as Camera Punch or Slow-mo Ramp, start with a flat curve at 1, because their motion already rises and settles.

The curve can also be edited in **Details**: the **Intensity Curve** field shows a small preview and an **Edit...** button that opens it in Unreal's curve editor. A track can use a Curve Float asset instead of its own keys; the lane then says so, and the asset is edited in its own editor.

5.6 Recipe and track settings in Details

With no track selected, **Details** shows the recipe's settings. With a track selected, it shows that track under a category named after it (**Track 1**, **Track 2** and so on), followed by the track's other categories; the recipe's own category is folded away. Click an empty area of the timeline to return to the recipe.

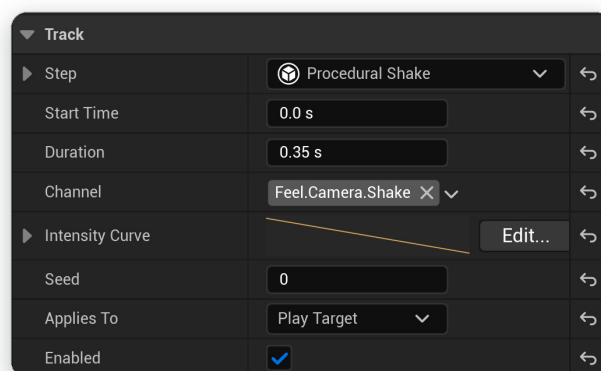


Figure 5.4 Details for a selected track: the track's timing and channel, then Step with the step's own settings

Recipe settings:

Setting	Default	Purpose
Cooldown	0 s	Time before the recipe can play again on the same target.
Max Concurrent	0	How many plays of the recipe may run at once on the same target. 0 means no limit.
Default Intensity	1	Multiplies every play of the recipe.
Parameters	empty	PRO Named values that the game passes when it plays the recipe (Chapter 7, Parameters, context and variation).
Sustain, Sustain Start, Sustain End	Off, 0 s, 1 s	PRO A region that loops until the play is released (7.4 Sustained recipes).
Library		PRO Feeling, genres and description, used by the Recipe Browser and in Content Browser tooltips (9.7 Tagging your own recipes).
Preview Mesh	None	The mesh shown in the preview: any static or skeletal mesh, or a cube when empty. It is saved with the recipe but used only by the editor.

Track settings. The **Track N** category holds **Step**, **Start Time**, **Duration**, **Channel**, **Intensity Curve**, **Seed**, **Applies To** and **Enabled**. **Step** unfolds to the settings of the step itself, such as a shake's amplitude and frequency; these are documented per step in [Chapter 16, Step reference](#). The remaining categories are:

Category	Contents
Parameter Mappings	PRO Recipe parameters that scale this track's intensity (7.1 Parameters and track mappings).
Randomness	PRO A random intensity and a random length for each play.
Comfort	Essential , Substitute Step and Essential Floor , which decide what happens when a player turns this kind of effect down (8.4 Essential tracks and substitutes).
Conditions	PRO Local Player Only , Max Distance , Chance and Platforms (7.7 Randomness, conditions and Random Choice).

A **Duration** of 0 makes an instant track, for steps that act once when the track starts, such as Play Sound or Blueprint Event. **Seed** changes the pattern of steps that use noise, such as Procedural Shake, so that two tracks of the same step do not move in step with each other. **Enabled** is the same switch as the mute toggle in the track header.

5.7 The preview: play, loop, scrub, preview mesh, parameter sliders

Playing and scrubbing. **Play** plays the recipe once and **Loop** repeats it. Click or drag in the ruler to move the playhead; the preview shows that exact moment. Because steps keep no state between

frames, scrubbing backwards shows the same frames as playing forwards, so a fast effect can be examined frame by frame.

What the preview shows. Camera effects move the preview camera, screen effects color the preview, actor effects move, scale and recolor the preview mesh, and sounds play. Effects on game time, controller vibration, UI widgets and Blueprint events act only in a running game; their tracks carry **No preview**.

The preview mesh. Set **Preview Mesh** in the recipe's Details to see actor effects on a representative object, for example a character for a squash and stretch, or a sword for a material flash.

Preview parameters (Pro). When the recipe has parameters, a row of sliders appears below the toolbar, one per parameter, under **Preview parameters:**. They play the recipe as if the game had passed those values, so the whole range of a parameter can be checked without starting the game. The values are not saved; **Defaults** returns every slider to the parameter's default.

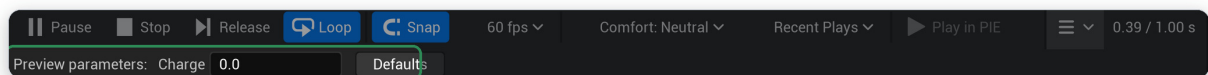


Figure 5.5 The preview parameter slider of a copy of `FR_Power_ChargeUp`, which has one parameter, Charge

Sustained recipes (Pro). When **Sustain** is on, the ruler marks the sustain region, and playback loops inside it as it would while the game holds the play. Click **Release** to leave the loop and play the rest of the recipe, as **Release Feel** does in the game.

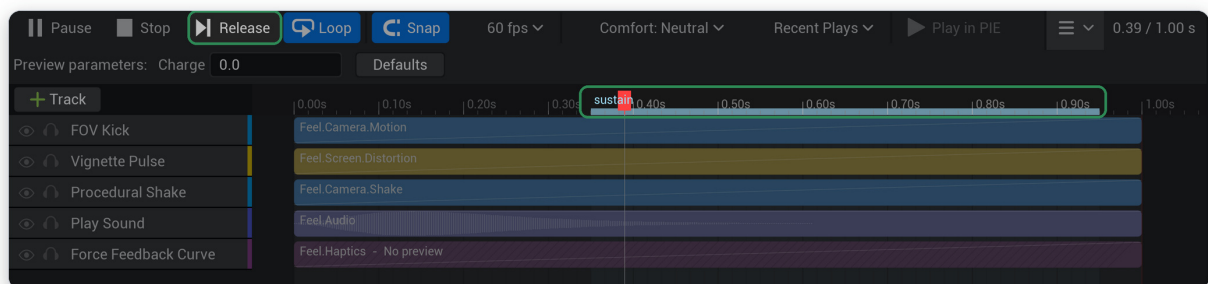


Figure 5.6 A copy of `FR_Power_ChargeUp` looping in its sustain region (0.35 s to 0.95 s, marked on the ruler), with Release enabled

5.8 Sound tracks: waveforms, snapping, tracks made from a sound PRO

A Play Sound track draws the waveform of its sound on its bar, aligned with the timeline, so that the loud moments of the sound are visible. As described in [see 5.4 Adding, moving and resizing tracks](#), other tracks snap to those moments when they are dragged.

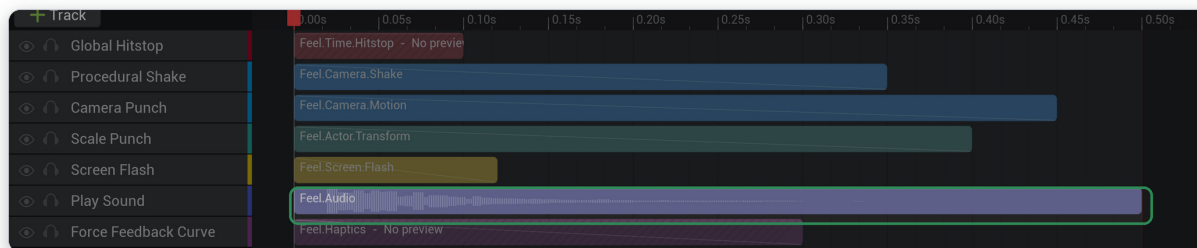


Figure 5.7 The waveform of the Play Sound track of HeavyHit

Right-clicking a Play Sound track adds two commands to the track menu:

- **Create Force Feedback Track From Sound** adds a Force Feedback Curve track below the sound, whose rumble follows the sound's loudness.
- **Create Shake Track From Sound** adds a Procedural Shake track below the sound, whose strength follows the sound's loudness.

Both commands read the sound's audio data, so they need a Sound Wave, or a Sound Cue that plays one. MetaSounds and procedural sounds cannot be read: they show no waveform, and the two commands are unavailable for them.

5.9 Checking comfort, and the intensity graph

Players can turn down effects that they find uncomfortable ([Chapter 8, Comfort and accessibility](#)).

The **Comfort** menu in the toolbar shows how the recipe plays for them:

Choice	The preview plays with
Neutral	No comfort scaling: every effect at full strength. The default.
Project Defaults	The comfort scales that new players start with.
Reduced Motion, Reduced Flashing, No Haptics	The built-in presets, with the values set in the project settings.

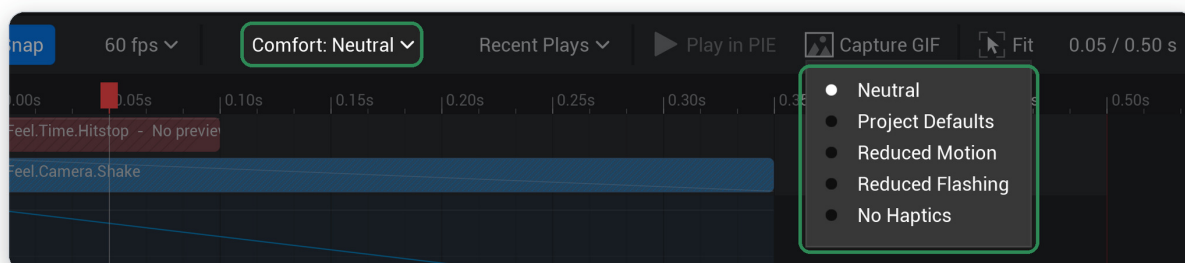


Figure 5.8 The Comfort menu of the preview

With a preset chosen, tracks that comfort changes say so on their bar, for example **Comfort x0.25** for a shake that plays at a quarter of its strength, **Removed by comfort settings**, or **Substitute**

plays (comfort) for an essential track whose substitute plays instead. Flash tracks that the flash limiter softens say **Flash softened by the flash limiter**.

The **Intensity** tab draws the combined intensity of each channel over the length of the recipe, one line per channel, with the preview parameters and the comfort choice applied. It shows at a glance where effects pile up, and how a parameter or a preset changes the recipe as a whole. [2.3 Intensity: how strong each effect plays](#) shows an example.

5.10 Play in PIE, and editing while the game runs

Effects on game time, controller vibration and UI can only be judged in a running game. **Play in PIE** plays the open recipe in the running Play In Editor session:

1. Start Play In Editor.
2. In the recipe editor, click **Play in PIE**. The recipe plays on the player's pawn; on a character, actor effects apply to its visible mesh.

The recipe can be edited while the game runs. Each change applies to the next play of the recipe, whether it comes from **Play in PIE** or from the game itself, so a value can be adjusted and tried again without restarting Play In Editor.

5.11 Copy, paste and keyboard shortcuts

Copied tracks stay available while the editor is open, so tracks can be copied from one recipe and pasted into another. A pasted track is an independent copy with its own step settings.

The shortcuts below work while the timeline has keyboard focus; click the timeline once to give it focus.

Key	Action
Space	Play or pause
F	Fit the whole recipe in view
Ctrl+Mouse Wheel	Zoom the timeline
Shift+Mouse Wheel, or drag with the middle mouse button	Scroll the timeline
Ctrl+C, Ctrl+V	Copy the selected track; paste at the playhead
Ctrl+D	Duplicate the selected track
Delete	Delete the selected track
Shift while dragging	Ignore snapping
Ctrl+Z, Ctrl+Y	Undo, redo

5.12 Validation messages on save

When a recipe is saved, FeelKit checks it through Unreal's data validation and reports problems on the **Asset Check** page of the Message Log. An error means that part of the recipe cannot work as set up; a warning points out a setting that probably does not do what was intended. The recipe is saved in both cases. Messages about a track begin with its number and step, for example "Track 2 (Play Sound): Play Sound has no sound".

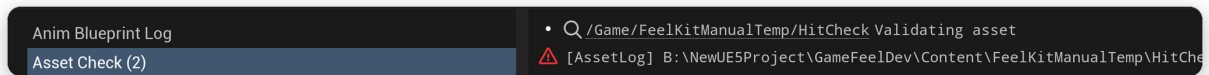


Figure 5.9 The Asset Check page after saving a recipe whose Play Sound track has no sound

Message	Meaning
Recipe has no tracks.	The recipe plays nothing.
Sustain is on, but Sustain End is not after Sustain Start, so nothing loops.	The sustain region is empty.
Sustain End is after the last track ends, so part of the loop is silent.	The loop includes time in which no track plays.
... has no step, so it is skipped.	A track without a step.
... has a length of 0, but this step needs a length to produce output.	Only steps that act once can be instant.
... intensity curve has keys outside 0 to 1.	The curve is read over the track's time from 0 to 1, so those keys never play.
... intensity curve never rises above 0, so the track produces no output.	The track is silent.
... is essential but has no substitute step and an essential floor of 0, so comfort settings can remove it completely.	See 8.4 Essential tracks and substitutes .
... has a substitute step but is not essential, so the substitute never plays.	Turn on Essential , or remove the substitute.
... has a chance of 0, so it never plays.	The track's Chance condition is 0.
... maps parameter N, which the recipe does not declare, so the mapping is ignored.	A parameter mapping names a parameter the recipe does not have.
Parameter N reads accumulator A, which is not defined in Project Settings > Plugins > FeelKit, so it uses its default value.	Add the accumulator to the project settings (7.3 Accumulators: effects that build up with repeated plays).

Steps add checks of their own, such as a Play Sound track without a sound; [Chapter 16, Step reference](#) lists them under each step.

6 Playing recipes in your game

IN THIS CHAPTER

- 6.1 Play Feel, and choosing a target
- 6.2 Passing context with Play Feel with Context
- 6.3 Handles: stop, release, change a parameter
- 6.4 Play Feel and Wait
- 6.5 Events and Feel Maps
- 6.6 Animation notifies
- 6.7 The Feel Trigger component
- 6.8 Enhanced Input: the Feel Input component
- 6.9 Gameplay Ability System cues
- 6.10 Using FeelKit from C++
- 6.11 The Feel Switch: feel off and on while playing

A recipe plays when something in the game asks for it. The simplest way is the **Play Feel** node. For larger games, FeelKit adds ways that keep recipe choices out of gameplay code: events that a Feel Map turns into recipes, markers in animations, and components that react to damage, landings, collisions or input without any Blueprint wiring. They can be combined freely in one project.

► **Video: Trigger recipes from your game (YouTube, about 9 minutes)**

The link is added before release.

Method	Suits	Edition
Play Feel and its variants (see 6.1 Play Feel, and choosing a target)	Any moment the game already handles in Blueprint or C++.	Lite and Pro
Send Feel Event with a Feel Map (see 6.5 Events and Feel Maps)	Games with many kinds of hits or actions, where designers choose the recipes.	Pro

Method	Suits	Edition
Animation notifies (see 6.6 Animation notifies)	Feedback tied to a frame of an animation, such as a sword swing or a footstep.	Pro
Feel Trigger component (see 6.7 The Feel Trigger component)	Damage, landings, jumps, collisions and overlaps, with no wiring.	Pro
Feel Input component (see 6.8 Enhanced Input: the Feel Input component)	Feedback on button presses, such as firing or charging.	Pro
Gameplay Cue notifies (see 6.9 Gameplay Ability System cues)	Projects that use the Gameplay Ability System.	Pro

6.1 Play Feel, and choosing a target

Play Feel plays a recipe on a target, with an intensity that multiplies every track of that play (1 by default). It returns a handle that identifies this play ([see 6.3 Handles: stop, release, change a parameter](#)).

The target decides where each kind of effect goes ([see 2.4 Targets: what a recipe plays on](#)). Five nodes make one:

Node	Plays on	Typical use
Make Feel Target from Actor	An actor. On a character, actor effects move and scale its visible mesh, never its collision capsule.	The player's character, an enemy, a door.
Make Feel Target from Component	One scene component, such as a weapon mesh.	Effects on part of an actor.
Make Feel Target at Location	A point in the world.	An explosion; sounds and distance checks use the point.
Make Feel Target from Local Player Camera	A local player's camera, by player index.	Camera and screen feedback that belongs to no actor.
Make Feel Target from Widget	A UMG widget. Widget steps apply to it.	Interface feedback such as a score counter punch.

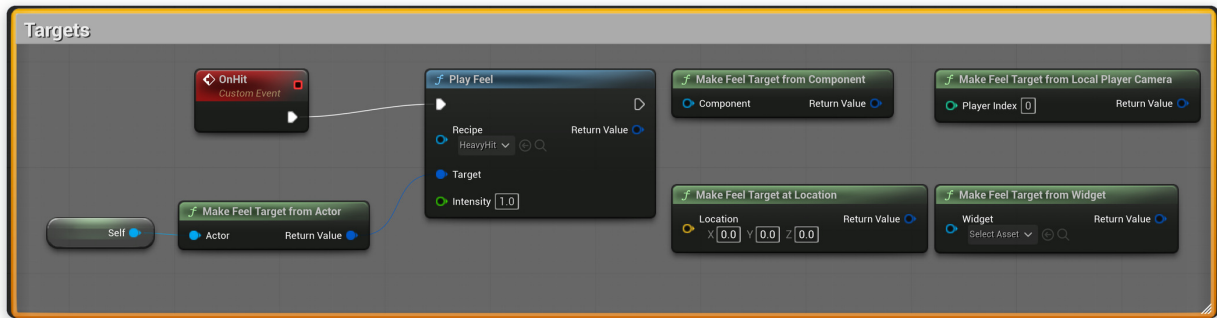


Figure 6.1 The five target nodes. Make Feel Target from Actor, given Self, feeds a Play Feel node

Camera, screen and controller effects always go to the local player who owns the target: the player controlling the character, or the owner of the widget. A target that belongs to a player on another machine receives none of them on this machine; that player's own machine shows them. With no target at all, they go to the first local player.

When a play does not start. Play Feel returns an invalid handle, and nothing plays, when:

- no recipe is connected;
- the recipe's **Cooldown** has not passed, or its **Max Concurrent** limit is reached on this target;
- FeelKit is turned off (the Feel Switch, **Set Feel Enabled** or `feel.Enabled 0`);
- the game runs as a dedicated server, which has no players to show feedback to;
- the target actor or component no longer exists (this case also logs a warning).

6.2 Passing context with Play Feel with Context PRO

Play Feel with Context plays a recipe like Play Feel, with extra information about this play in a **Feel Play Context**. Build the context with **Make FeelPlayContext**; every field is optional, and an empty context plays exactly like Play Feel.

Field	Used for
Parameters	Values for the recipe's parameters, by name, for example <code>Damage 50</code> . Tracks with parameter mappings scale by them (7.1 Parameters and track mappings).
Instigator	A second actor, such as the attacker. Tracks set to Applies To Instigator play their actor effects on it (7.5 The instigator, and the Applies To setting).
Direction, Location, Normal	Details of the event, such as the direction of a hit. Steps such as Camera Punch can take their direction from it (7.6 Directions and locations).
Context Tags	Gameplay tags that describe the circumstances. Feel Maps use them to choose a recipe (see 6.5 Events and Feel Maps).

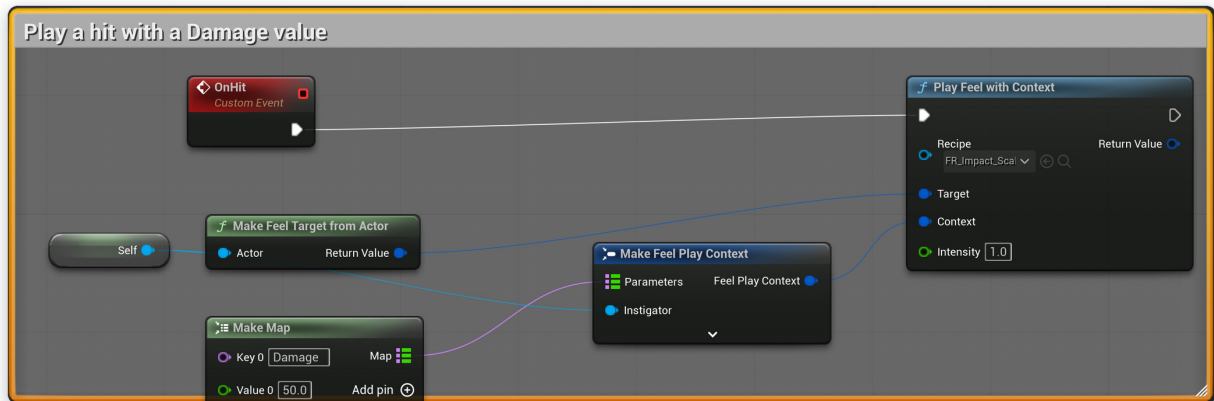


Figure 6.2 Play Feel with Context. Make FeelPlayContext passes the parameter Damage 50 (from a Make Map node) and the actor itself as Instigator

The library recipe `FR_Impact_ScalableHit` is built for this: its tracks are mapped to a `Damage` parameter from 0 to 100, so one recipe covers light and heavy hits.

6.3 Handles: stop, release, change a parameter

The handle that Play Feel returns refers to one play of a recipe. Store it in a variable when the play needs to be controlled later.

Node	What it does	Edition
Stop Feel	Stops the play. With Blend Out on (the default), its effects fade over the project's Blend Out Time (0.2 s).	Lite and Pro
Stop All Feel	Stops every play, or only the plays on one actor.	Lite and Pro
Release Feel	Ends the sustain loop of a sustained recipe, so that it plays its ending (7.4 Sustained recipes).	Pro.
Set Feel Parameter	Changes a parameter of the play while it runs, for example a charge that grows while a button is held.	Pro.
Is Feel Playing	Whether the play is still running, including while it fades out.	Lite and Pro
Is Valid (Feel Handle)	Whether the handle came from a play that started. The play may have finished since.	Lite and Pro

The **Parameter Name** pin of **Set Feel Parameter** is a dropdown. When its handle comes from a Play Feel node in the same graph, it lists the parameters of that node's recipe; otherwise it lists every recipe parameter in the project.

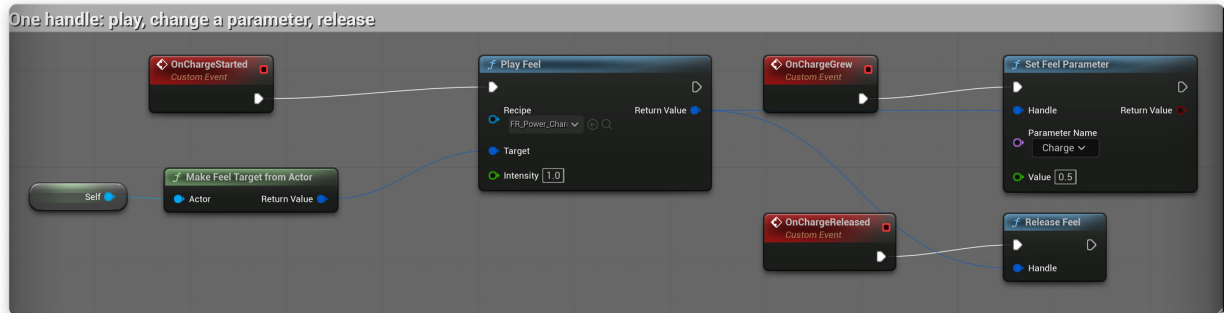


Figure 6.3 One handle from Play Feel used by Set Feel Parameter (the dropdown shows the recipe's Charge parameter) and by Release Feel

A handle stays safe to use after its play ends: the nodes then do nothing, and **Is Feel Playing** returns false.

6.4 Play Feel and Wait PRO

Play Feel and Wait plays a recipe and continues when it ends, which suits sequences such as a door that opens only once its feedback has played. It has two output execution pins:

- **On Finished** fires when the recipe plays to its end.
- **On Cancelled** fires when the play is stopped early, its target is destroyed, or it could not start.

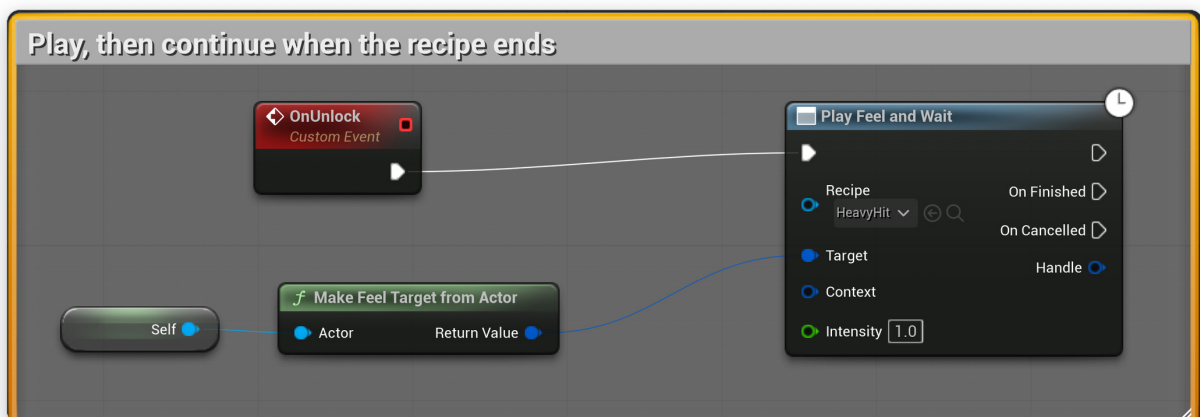


Figure 6.4 Play Feel and Wait with its On Finished and On Cancelled pins

A sustained recipe finishes only after it is released, so pair it with **Release Feel** or use a recipe without sustain.

6.5 Events and Feel Maps PRO

With **Send Feel Event**, gameplay code sends what happened, as a gameplay tag, and a **Feel Map** decides which recipe plays. Recipes can then be changed, and variants added, without touching gameplay code. [2.6 Events and Feel Maps, in brief](#) introduces the idea.

Creating a Feel Map.

1. In the Content Browser, right-click and choose **FeelKit** > **Feel Map**. Name it and open it.
2. Under **Entries**, add one row per event. Each row has an **Event**, the **Recipe** to play, and optionally **Required Tags**, an **Intensity Scale** (1) and a **Priority** (0).
3. Open **Edit** > **Project Settings** > **Plugins** > **FeelKit**, and under **Events** add the map to **Feel Maps**.

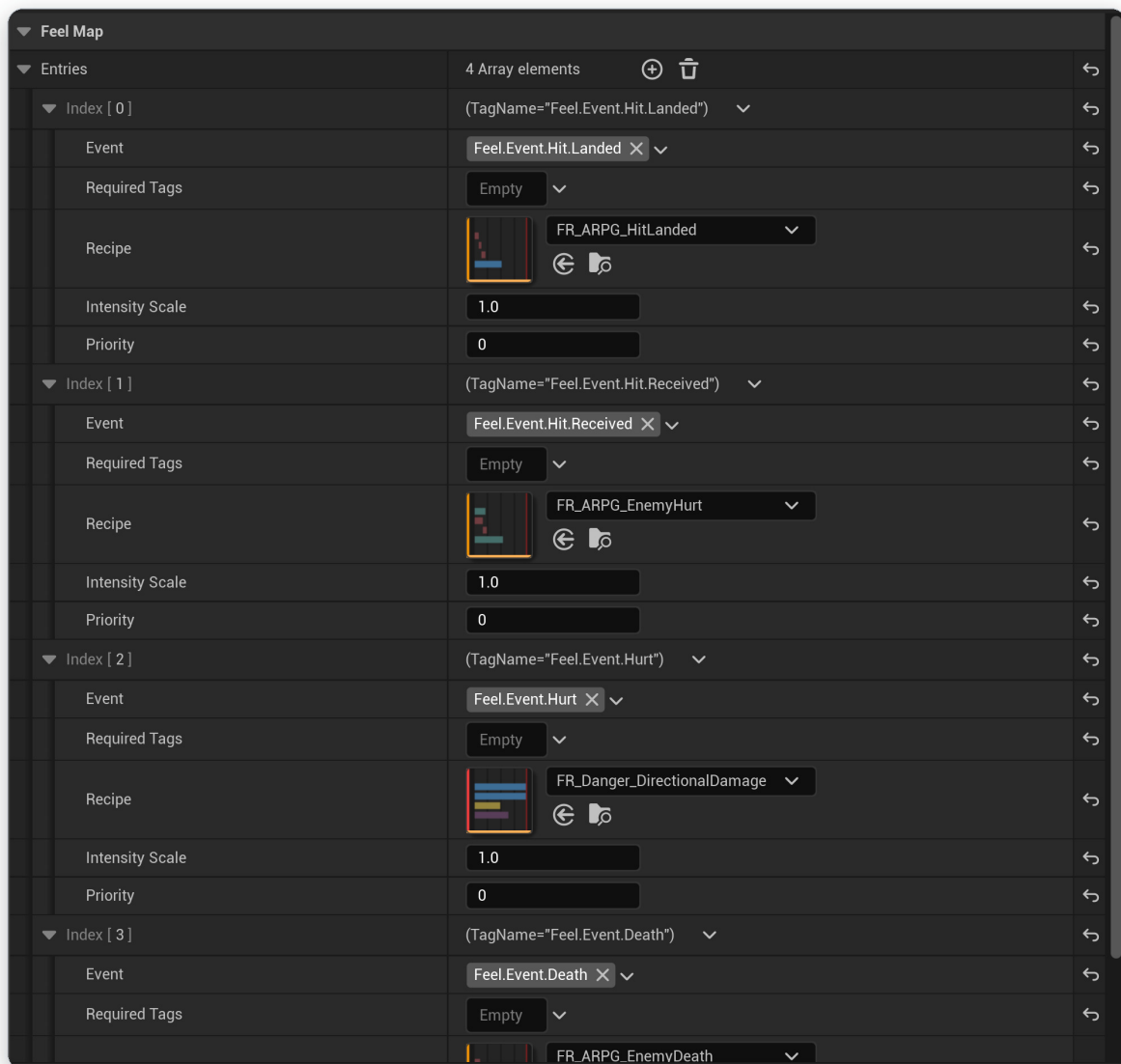


Figure 6.5 The Feel Map of the Action/RPG demo, with one row per event

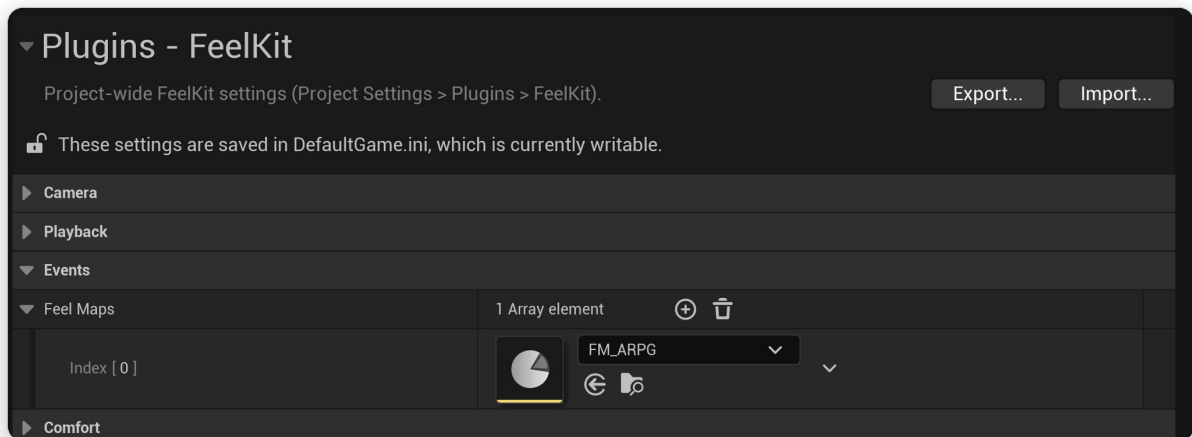


Figure 6.6 Project Settings > Plugins > FeelKit > Events, with the Feel Map in Feel Maps

Sending an event. Call **Send Feel Event** with the event tag, a target and, optionally, a context. FeelKit defines four starter events, which a project can use or extend with its own tags:

Event	Meaning
<code>Feel.Event.Hit.Landed</code>	An attack connected. Usually sent on the attacker, with the hit location.
<code>Feel.Event.Hit.Received</code>	Something was hit. Usually sent on what was hit, with the attacker as instigator and the hit location and direction.
<code>Feel.Event.Hurt</code>	The player was hurt. Usually sent on the player, with the direction the damage came from.
<code>Feel.Event.Death</code>	Something died or was destroyed. Usually sent on what died.

How a row is chosen. A row answers its event and the event's children: a row for `Feel.Event.Hit` also answers `Feel.Event.Hit.Landed` when no row is more specific. The context must contain all of the row's **Required Tags**. Among the rows that match, a more specific event wins, then the row with more required tags, then the higher **Priority**, then the earlier row. The row's **Intensity Scale** multiplies the intensity of the play.

For example, a map with a row for `Feel.Event.Hit.Landed` that plays a light hit, and a second row for the same event with the required tag `Hit.Heavy` that plays a heavy hit, plays the heavy hit only when the context carries `Hit.Heavy`. Tags such as `Hit.Heavy` are the project's own: add them under **Project Settings > Gameplay Tags**.

Maps on an actor. A **Feel Trigger** component can hold Feel Maps of its own (see 6.7 The Feel Trigger component). When an event targets its actor, those maps are checked first, so that one enemy type can sound and feel different from another.

When no row matches, nothing plays, and a warning in the Output Log names the event, once per event.

6.6 Animation notifies PRO

Animation notifies play feedback on an exact frame of an animation, such as the moment a sword connects or a foot lands. To add one, open the animation or montage, right-click the **Notifies** track, and choose **Add Notify** or **Add Notify State**, then the FeelKit notify:

Notify	What it does
Play Feel	Plays a recipe on that frame.
Send Feel Event	Sends a Feel Event on that frame; the Feel Maps choose the recipe.
Set Feel Value	Sets or adds to accumulators on that frame, for example how heavy the current swing is (7.3 Accumulators: effects that build up with repeated plays).
Play Feel (Window)	A notify state: plays a recipe for the length of the window. At its end a sustained recipe is released, and any other recipe is stopped unless Let Non Sustained Recipe Finish is on.

The notifies share **Target** (the skeletal mesh by default, or the owning actor), **Intensity**, **Parameters** and **Context Tags**.



Figure 6.7 The Notifies track of the Action/RPG combo montage: Set Feel Value markers ("Feel Value: Swing 0.35") and Play Feel markers ("Feel: [FR_ARPG_Swing](#)")

FeelKit's notifies also play in the animation editor's preview, so actor effects such as a scale punch can be tuned while scrubbing the animation. Camera, time and controller effects need a running game.

6.7 The Feel Trigger component PRO

The **Feel Trigger** component plays recipes when common things happen to its owner, with no Blueprint wiring. Add it to an actor, and add one entry to **Triggers** for each reaction.

Event	Fires when	Event value
Take Any Damage	The owner receives damage.	The damage amount.
Landed	The owner, a Character, lands after falling.	The downward speed at landing, in cm/s.
Component Hit	The owner's root component reports a blocking hit.	The strength of the hit's impulse.

Event	Fires when	Event value
Begin Overlap, End Overlap	Another actor starts or stops overlapping the owner.	0
Jumped	The owner, a Character, jumps from the ground.	1
Air Jumped	The owner jumps again in the air.	The jump number, 2 or more.
Launched	The owner is launched, for example by a jump pad or a wall jump.	The launch speed, in cm/s.

Each entry plays a **Recipe**, or sends a **Feel Event** when no recipe is set. **Value Parameter** passes the event value to a recipe parameter, so a harder landing plays a stronger recipe. The other fields choose where the recipe plays: **Target Skeletal Mesh** (on) plays on the owner's mesh rather than its capsule; **Play On** chooses the owner or the other actor of the event; **Other Actor as Instigator** (on) passes the other actor as the instigator; **Only for Players** (off) ignores events caused by anything but a player.

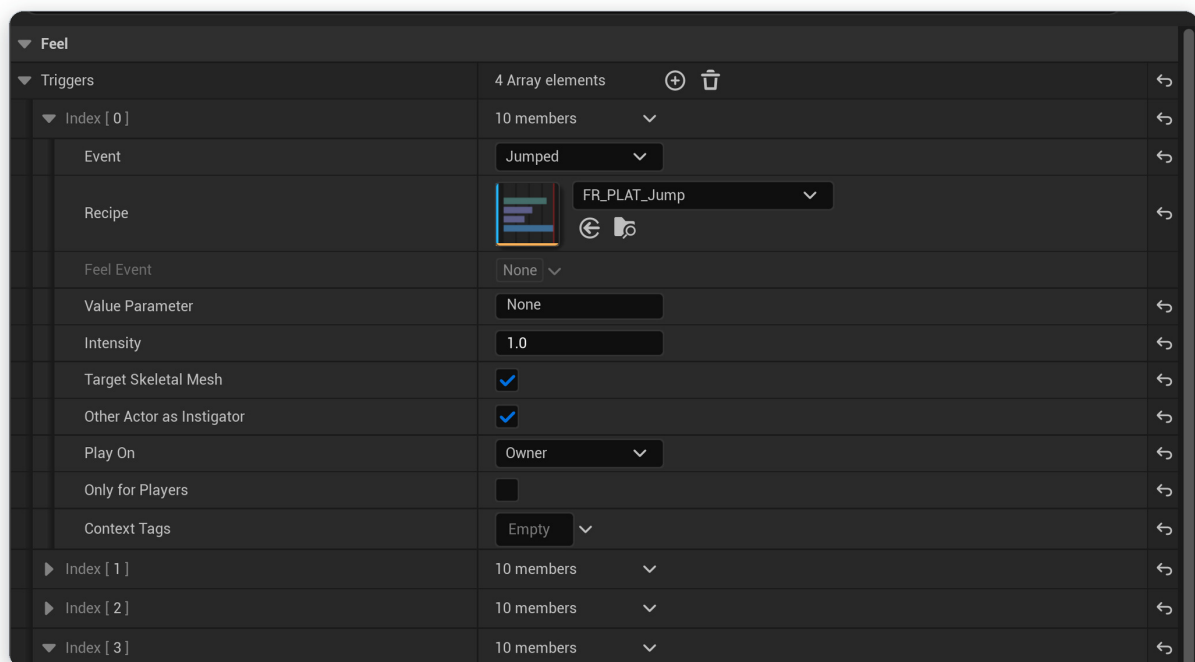


Figure 6.8 The Feel Trigger of the Platformer demo's character: jump, air jump, launch and landing

The Platformer demo uses four entries: **Jumped** plays `FR_PLAT_Jump`, **Air Jumped** plays `FR_PLAT_AirJump` with the jump number as `JumpNumber`, **Launched** plays `FR_PLAT_WallJump` with the launch speed as `LaunchSpeed`, and **Landed** plays `FR_PLAT_Land` with the landing speed as `LandSpeed`.

Fire Event fires the entries of an event from Blueprint, for events that the component cannot detect itself. The component ticks only when an entry uses **Jumped**, **Air Jumped** or **Launched**; the other events are bound once, when play begins.

6.8 Enhanced Input: the Feel Input component PRO

The **Feel Input** component plays recipes from Enhanced Input actions. Add it to a pawn or a player controller, and add one entry to **Bindings** for each action:

Field	Purpose
Action	The input action. It must be in an input mapping context that the player has added.
Play On	Started , Triggered , Completed or Canceled , as in Enhanced Input.
Recipe or Event	What to play.
Intensity, Scale Intensity by Value	The intensity, optionally multiplied by how far the input is pressed.
Value Parameter	A recipe parameter that receives the input's value.
End when Input Ends	Ends the play when the input is released: a sustained recipe plays its ending, others fade out. Suits a charge that lasts while a button is held.
Once Per Press	Plays at most once per press, even when the action fires every frame.

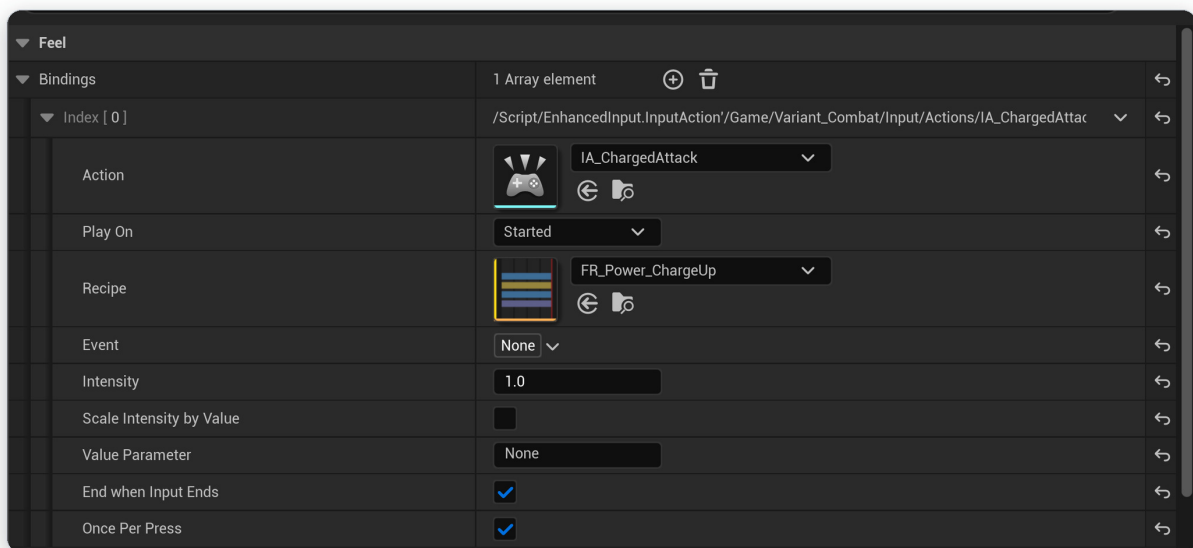


Figure 6.9 A Feel Input binding: the charged attack action plays `FR_Power_ChargeUp` while it is held, once per press

The recipes play on the component's owner. **Handle Input** simulates an input for testing.

6.9 Gameplay Ability System cues PRO

With the GAS add-on installed ([3.5 Adding the GAS add-on](#)), Gameplay Cues can play recipes. Create a Blueprint class with one of these parents and set its **Gameplay Cue Tag** as for any cue notify:

- **Feel Gameplay Cue Notify** plays once, when the cue is executed or added. It suits hits, pickups and other one-shot moments.
- **Feel Gameplay Cue Notify (Actor)** plays while the cue is active: it starts when the cue is added and ends when it is removed. With a sustained recipe, the feedback loops for as long as the gameplay effect lasts. **Stop on Remove** stops the play at once instead of letting it play its ending.

Both play a **Recipe**, or send an **Event**; with neither set, they send the cue's own tag as the event, so Feel Maps can answer cues directly. **Scale Intensity by Magnitude** scales the play by the cue's normalized magnitude, and the two magnitude parameters pass the raw and normalized magnitudes to recipe parameters. **Effect Causer as Instigator** uses the effect causer, such as a projectile, as the instigator.

When a cue is executed from Blueprint, connect a **Make GameplayCueParameters** node to its **Parameters** pin. The node cannot compile with that pin left empty.

6.10 Using FeelKit from C++

Add `FeelCore` to the dependencies of the game module in its `.Build.cs` file:

```
PublicDependencyModuleNames.AddRange(new string[] { "Core", "CoreUObject", "Engine",
"FeelCore" });
```

Plays go through the world's Feel Subsystem:

```
#include "FeelSubsystem.h"

if (UFeelSubsystem* Feel = UFeelSubsystem::Get(this))
{
    HitHandle = Feel->PlayFeel(HitRecipe, FFeelTarget::FromActor(this), 1.0f);
}
```

`FFeelTarget::FromActor`, `FromComponent`, `AtLocation`, `FromLocalPlayerCamera` and `FromWidget` match the Make Feel Target nodes. Every Blueprint node is also a static function of `UFeelBlueprintLibrary`. The subsystem broadcasts `OnFeelStarted` and `OnFeelFinished` (Pro) for code that reacts to plays. The Feel Input component needs the module `FeelEnhancedInput`, and the GAS cue notifies need `FeelGAS`. [Chapter 17, Blueprint nodes and components](#) lists the subsystem's functions.

6.11 The Feel Switch: feel off and on while playing

The **Feel Switch** is an actor that lets anyone playing the game turn FeelKit off and on, to see and feel what it adds. [4.4 Compare with the feel off](#) shows it in use. Place one in a level; it needs no input assets and no Blueprint changes.

Setting	Default	Purpose
Switch Keys	Tab, controller View / Share	Keys that turn FeelKit off and on.
Start Enabled	On	Whether FeelKit is on when the level starts. The choice is never saved.
Show Built in Display	On	The start card and the corner badge. Turn off to show your own display from the On Feel Switched event.
Show Start Card, Start Card Title, Start Card Text, Start Card Seconds	On, "FeelKit", a short explanation, 8 s	The card that explains the switch when the level starts.
Controls	empty	The level's controls, listed in a small panel under the badge.
Comfort Menu Keys	O, controller Menu / Options	Keys that open and close the players' comfort menu (Chapter 8, Comfort and accessibility). They are added to the controls panel.
Comfort Menu Class, Pause in Comfort Menu	Project setting, On	Which menu opens, and whether the game pauses while it is open.

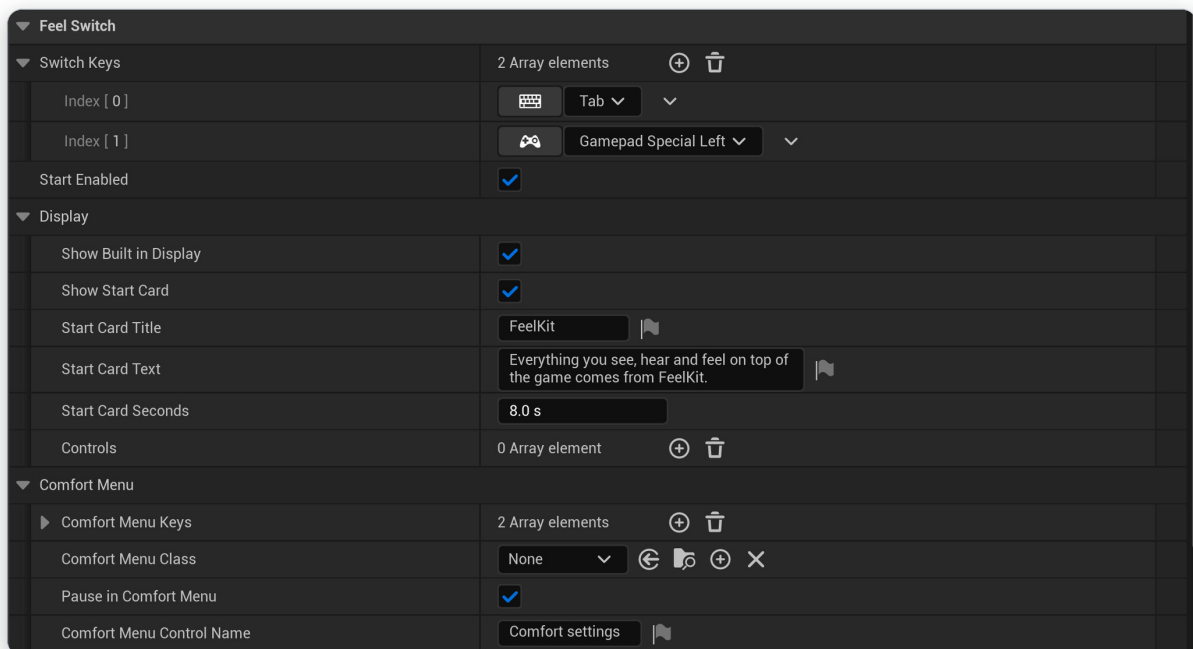


Figure 6.10 The Feel Switch's settings in Details, at their defaults

Turning FeelKit off stops every playing recipe at once, returns slowed time to normal, and blocks new plays until it is turned on again. The game's own camera shakes, sounds and effects are unaffected. The switch applies to the whole game on the machine, so every Play In Editor window switches together. A second Feel Switch in the same level logs a warning and does nothing.

The same switch is available as nodes, with or without a Feel Switch in the level: **Set Feel Enabled**, **Toggle Feel** and **Is Feel Enabled**. They change the same value as the console variable `feel.Enabled` ([Chapter 19, Console commands](#)).

7 Parameters, context and variation

IN THIS CHAPTER

- 7.1 Parameters and track mappings
- 7.2 The Distance parameter
- 7.3 Accumulators: effects that build up with repeated plays
- 7.4 Sustained recipes
- 7.5 The instigator, and the Applies To setting
- 7.6 Directions and locations
- 7.7 Randomness, conditions and Random Choice
- 7.8 Recipes inside recipes

A fixed recipe plays the same way every time. Parameters, accumulators and the play context let one recipe respond to what happened: how hard the hit was, how far away the explosion is, how long the combo has lasted, which way the blow came from. Random ranges, conditions and Random Choice add controlled variation, so that repeated plays do not look mechanical. All of these features are part of FeelKit Pro.

► **Video: Build a recipe on the timeline (YouTube, about 9 minutes)**

The link is added before release.

7.1 Parameters and track mappings

A **parameter** is a named number that the game passes each time a recipe plays, such as the damage of a hit. Tracks read it through **parameter mappings**, which turn the parameter into a multiplier for the track's intensity.

Declaring a parameter. In the recipe editor, with no track selected, add an entry to **Parameters** in **Details**:

Field	Default	Purpose
Name	None	The name the game uses, such as <code>Damage</code> .
Default Value	0	The value used when the game passes none.
Min Value, Max Value	0, 1	The meaningful range, in the game's own units. Values outside it are clamped.
Accumulator	None	An accumulator to read when the game passes no value (see 7.3 Accumulators: effects that build up with repeated plays).
Description	empty	What the value means, shown as the tooltip of the preview slider.

Mapping a track to it. Select a track and add an entry to **Parameter Mappings**: choose the **Parameter** and, optionally, a **Curve**. The curve's horizontal axis is the parameter from its minimum (0) to its maximum (1); its vertical axis is the multiplier. The default straight line from 0 to 1 makes the track grow in proportion to the parameter. A curve that starts at 0.3, for example, keeps a light hit at 30 percent strength instead of silencing it. Several mappings on one track multiply together, and they combine with the track's intensity curve.

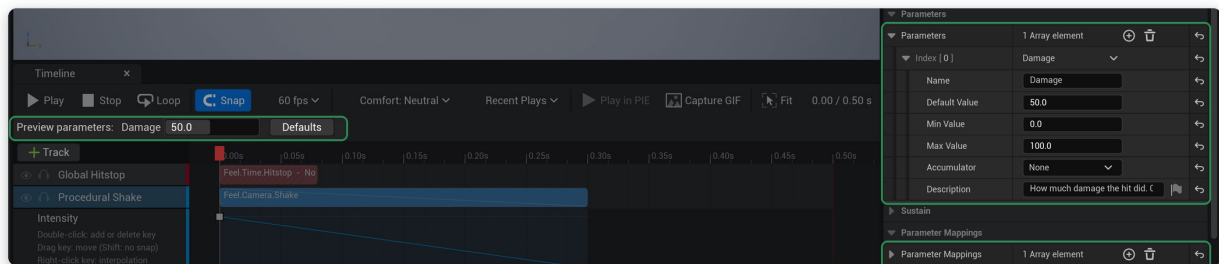


Figure 7.1 A copy of `FR_Impact_ScalableHit`: the Damage slider under the Timeline toolbar, the Damage parameter, and the selected track's Parameter Mappings

Passing the value. The game passes values in the **Parameters** of a **Feel Play Context** ([6.2 Passing context with Play Feel with Context](#)), from an animation notify's **Parameters**, or from a Feel Trigger entry's **Value Parameter** ([6.7 The Feel Trigger component](#)). **Set Feel Parameter** changes a value while the recipe plays ([6.3 Handles: stop, release, change a parameter](#)).

Previewing. The recipe editor shows a slider for each parameter under **Preview parameters**; so the whole range can be tuned without the game ([5.7 The preview: play, loop, scrub, preview mesh, parameter sliders](#)).

Example. `FR_Impact_ScalableHit` declares `Damage` from 0 to 100 (default 50), and maps its shake, punch, flash and rumble to it: one recipe covers a graze and a critical blow. `FR_Weight_Land` declares `FallSpeed` from 0 to 1600 cm/s; a Feel Trigger entry for **Landed** passes the landing speed to it.

7.2 The Distance parameter

A parameter named `Distance` is filled in automatically when the game does not pass it: FeelKit sets it to the distance, in centimeters, from the play's target to the nearest local player's camera. Map tracks to it with a curve that falls from 1 to 0 to make an effect weaken with distance.

`FR_Power_Explosion` uses it: its flash, camera punch, shake, chromatic aberration, muffled sound and rumble are strong close to the blast and fade with distance. For a hard cut-off instead, use the **Max Distance** condition of a track (see [7.7 Randomness, conditions and Random Choice](#)).

7.3 Accumulators: effects that build up with repeated plays

An **accumulator** is a named value that builds up as the game adds to it and decays over time, such as a combo counter or heat that rises with each shot. Recipes read it through a parameter, so feedback grows with a streak without any bookkeeping in the game.

Defining an accumulator. Add an entry to **Project Settings > Plugins > FeelKit > Playback > Accumulators**:

Field	Default	Purpose
Name	None	The name that nodes and parameters use.
Max Value	1	The value never rises above it.
Decay Per Second	1	How fast it falls back toward 0. 0 means it never decays.
Decay Delay	0 s	How long after the last change it starts to decay.



Figure 7.2 Accumulators in Project Settings: Combo, with a maximum of 10, a decay of 1 per second and a delay of 1.5 s

Changing it. The nodes **Add to Feel Accumulator**, **Set Feel Accumulator** and **Get Feel Accumulator** change and read a value. Their **Actor** pin makes the value belong to one actor, such as each enemy's own heat; left empty, the value is global. Their name pins are dropdowns listing the project's accumulators. The **Set Feel Value** animation notify sets or adds to accumulators from an animation ([6.6 Animation notifies](#)).

Reading it. Set a recipe parameter's **Accumulator** field. While the recipe plays, the parameter reads the target's own value, else the value of the play's instigator, else the global value, every frame. A value the game passes directly in the play context takes precedence.

Example. The Action/RPG demo defines `Combo` (maximum 10, decay 1 per second after 1.5 s), and the player's Blueprint adds 1 to the player's own value on every hit. `FR_ARPG_EnemyHurt` plays on the enemy that was hit; its parameter `Streak` reads `Combo` and, since the enemy has no value of its own, finds the player's through the instigator, so a long combo hits harder. The combo animations set the accumulator `Swing` to 0.35, 0.65 and 1.0 on their three attacks with **Set Feel Value** notifies, and the hit recipes read it, so the third attack of a combo lands hardest. The library recipes `FR_Reward_ComboStep` and `FR_Interface_ScoreTick` read `Combo` in the same way ([9.6 The 38 library recipes at a glance](#)).

A parameter that names an accumulator the project does not define plays with its default value, and saving the recipe reports it.

7.4 Sustained recipes

A sustained recipe keeps playing for as long as a condition lasts: a charge held down, an alarm, low health. In **Details**, turn on **Sustain** and set **Sustain Start** and **Sustain End**. When the play reaches **Sustain End**, time returns to **Sustain Start** and the region loops. When the play is released, it continues past the region and plays the rest of the recipe, its ending.

A sustained play is released by:

- **Release Feel** on its handle ([6.3 Handles: stop, release, change a parameter](#));
- the end of a **Play Feel (Window)** notify state ([6.6 Animation notifies](#));
- a Feel Input binding with **End when Input Ends**, when the input is released ([6.8 Enhanced Input: the Feel Input component](#));
- the removal of its Gameplay Cue, with a **Feel Gameplay Cue Notify (Actor)** ([6.9 Gameplay Ability System cues](#)).

Stop Feel ends it at once, without the ending. The recipe editor loops the region while previewing and has a **Release** button ([5.7 The preview: play, loop, scrub, preview mesh, parameter sliders](#)).

Example. `FR_Power_ChargeUp` loops from 0.35 s to 0.95 s. Combined with **Set Feel Parameter** on its `Charge` parameter, the hum, the tightening view and the rumble grow while the charge is held, and the release plays its ending. Seven library recipes are sustained ([9.6 The 38 library recipes at a glance](#)).

7.5 The instigator, and the Applies To setting

Many moments involve two actors: the attacker and the one hit, the player and the pickup. The play's target is one of them; the **Instigator** in the play context can be the other (6.2 [Passing context with Play Feel with Context](#)). A Feel Trigger passes the other actor of the event as the instigator by default.

Each track chooses which of the two its actor effects apply to, with **Applies To** in the **Track** category:

- **Play Target** (the default): actor effects such as a scale punch or a material flash apply to the target.
- **Instigator**: they apply to the instigator instead. Camera, screen and controller effects go to the local player of that actor. A play without an instigator skips the track, and the recipe editor labels it **Skipped: the play had no instigator**.

Example. `FR_ARPG_Parry` plays on the parrying player. Its Hit Flash and Actor Hitstop tracks are set to Instigator, so the attacker whose blow was parried flashes and freezes, from the same recipe.

7.6 Directions and locations

The play context can carry a **Direction**, a **Location** and a surface **Normal**. Some steps use them:

Step	Uses
Camera Punch	Direction Source: Step Settings (the punch as set on the step), Play Direction , Away From Play Location or Toward Play Location . The punch keeps its strength and points along that direction, as seen from the camera when the play started.
Spawn Decal	Appears at the play's Location , projects along its Normal when given.
Spawn Particle	Appears at the play's Location ; Orient to Normal aligns the system to the Normal .
Number Pop	Appears at the play's Location , or at the target.
Blueprint Event , steps written in Blueprint	Can read the whole context.

Each step falls back to its own settings or to the target when the play passes no direction or location. `FR_Danger_DirectionalDamage` uses **Away From Play Location**: passing the attacker's location pushes the camera away from where the damage came from.

7.7 Randomness, conditions and Random Choice

Random ranges. The **Randomness** category of a track holds **Random Intensity** and **Random Duration Scale**, each a range from **Min** to **Max**. Every play picks a value in each range: 0.8 to 1.2 makes each play between 80 and 120 percent as strong. 1 to 1, the default, means no variation.

Steps with noise, such as Procedural Shake, also change pattern with the track's **Seed**, and Camera Punch has a **Direction Jitter**.

Conditions. The **Conditions** category decides, once per play, whether the track plays at all:

Condition	Default	The track plays only when
Local Player Only	Off	The target belongs to a player on this machine.
Max Distance	0 (no limit)	The target is within this distance of the local camera.
Chance	1	A random roll succeeds: 0.5 plays about every second time.
Platforms	empty (all)	The game runs on one of the listed platforms, by Unreal's platform names, such as Windows or PS5.

The rolls are made when the play starts and kept for its whole length, so scrubbing in the editor shows the same result. A track skipped by a condition says why on its bar, for example **Skipped this play (chance 50%)**.

Random Choice. The **Random Choice** step plays one of several steps, picked for each play by weight: an option with weight 2 is picked twice as often as one with weight 1. Use it for variety within one track, such as three different impact sounds or two shake patterns. The pick is fixed for the play; the recipe editor labels the bar with the option picked for the preview.

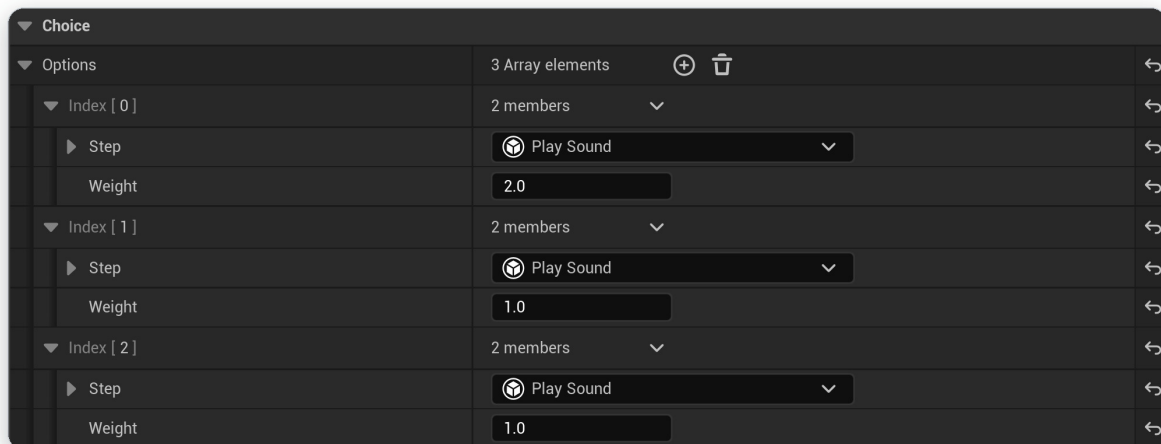


Figure 7.3 A Random Choice track with three Play Sound options and their weights

7.8 Recipes inside recipes

The **Play Recipe** step plays another recipe inside a track, with the same target, context and parameters. **Intensity Scale** multiplies the inner recipe on top of the track's intensity, and the inner recipe's tracks keep their own channels and comfort settings. Give the track at least the length of the inner recipe, which stops when the track ends.

Nesting lets common pieces be shared: one impact recipe used inside several weapon recipes, changed in one place. Recipes nest up to four deep, and a recipe cannot play itself; saving reports both cases. The recipe editor previews the inner recipe exactly.

8 Comfort and accessibility

IN THIS CHAPTER

- 8.1 Comfort groups, and how the scales apply
- 8.2 Presets
- 8.3 Reading and changing a player's settings
- 8.4 Essential tracks and substitutes
- 8.5 The flash limiter
- 8.6 Motion comfort: camera roll and field of view speed
- 8.7 Engine camera shakes and controller vibration
- 8.8 Saving settings, or using your own save system
- 8.9 The comfort menu for players
- 8.10 Restyling the comfort menu
- 8.11 The Comfort Audit

Strong feedback suits many players and troubles some. Camera shake and motion can cause motion sickness, flashes can be uncomfortable or, for people with photosensitive epilepsy, dangerous, and vibration is unwelcome to others. FeelKit lets every player turn each kind of effect down, applies the choice to every recipe the game plays, and saves it. The complete comfort layer, including the ready-made comfort menu, is part of both editions.

► **Video: Comfort settings for your players (YouTube, about 7 minutes)**

The link is added before release.

8.1 Comfort groups, and how the scales apply

Every effect belongs to a comfort group through its channel. Each player has a scale from 0 (off) to 1 (full strength) for each group, and a **Master** scale that multiplies all of them.

Group	Channels under the default mapping	Typical effects
Camera Shake	<code>Feel.Camera.Shake</code>	Procedural Shake
Camera Motion	<code>Feel.Camera.Motion</code>	Camera Punch, FOV Kick, Camera Roll, Camera Zoom
Flashes	<code>Feel.Screen.Flash</code> , <code>Feel.Actor.Light</code>	Screen Flash, Light Flash
Hitstop and Slow-mo	<code>Feel.Time.Hitstop</code> , <code>Feel.Time.SlowMo</code>	Global Hitstop, Actor Hitstop, Slow-mo Ramp
Screen Distortion	<code>Feel.Screen.Distortion</code> , <code>Feel.Screen.Color</code>	Vignette Pulse, Chromatic Aberration, Desaturate, Color Tint
Haptics	<code>Feel.Haptics</code>	Force Feedback Curve, Haptic Pattern

Channels that no group lists, such as sounds, actor motion and UI effects, are scaled by **Master** alone. The mapping is set in **Project Settings > Plugins > FeelKit > Channel Comfort Groups** (Chapter 18, [Project settings and editor preferences](#)); when a channel matches several rows, the most specific tag wins, so a project can place `Feel.Camera.Shake.Explosion` in a group of its own.

A track's final intensity is the intensity of the play, times its intensity curve, times the player's scale for the track's group, times **Master** (2.3 [Intensity: how strong each effect plays](#)). A track whose scale is 0 is skipped entirely, unless it is essential (see 8.4 [Essential tracks and substitutes](#)).

Which player. Each local player has settings of their own, and a play uses the settings of the local player its target belongs to. A play that belongs to no local player uses the project's default scales.

8.2 Presets

Presets set every scale and option at once. Players choose one as a starting point and can adjust it afterwards.

Preset	Values (defaults, editable in the project settings)
Default	The project's Default Comfort Scales : every scale 1, the flash limiter on at 3 flashes per second. New players start with it.
Reduced Motion	Camera Shake 0.25, Camera Motion 0.25, Screen Distortion 0.5, camera roll off, zoom speed limited to 40° per second.
Reduced Flashing	Flashes 0.2, flash limiter at 1 flash per second in Suppress mode.
No Haptics	Haptics 0.

The values of the built-in presets are set under **Project Settings > Plugins > FeelKit > Comfort > Presets**. A project can add presets of its own as **Feel Comfort Preset** assets (right-click in the Content Browser, **Miscellaneous > Data Asset**, class **Feel Comfort Preset**), each with a **Display Name** and a set of scales, and apply them with **Apply Custom Comfort Preset**.

8.3 Reading and changing a player's settings

The comfort settings of a player are held by the player's **Feel Comfort** subsystem. **Get Feel Comfort** returns it from a player controller; the other nodes are called on it:

Node	What it does
Get Comfort Scales, Set Comfort Scales	Reads or replaces all scales and options at once.
Set Master Comfort Scale	Sets Master .
Set Comfort Group Scale	Sets one group, for example Camera Shake to 0.5.
Apply Comfort Preset, Apply Custom Comfort Preset	Applies a built-in preset or a preset asset.
Save Comfort Settings, Load Comfort Settings	Saves or reloads the settings now (see 8.8 Saving settings, or using your own save system).
Get Effective Force Feedback Scale	How much of the game's own controller vibration reaches the player (see 8.7 Engine camera shakes and controller vibration).

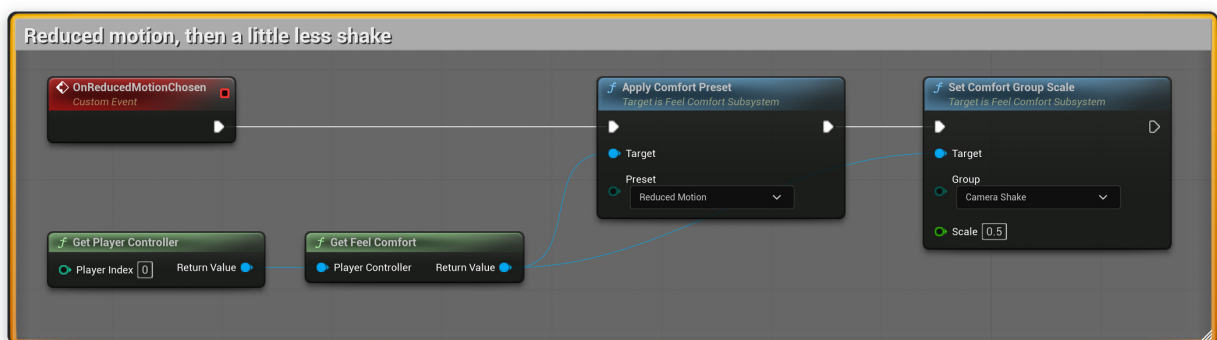


Figure 8.1 Get Feel Comfort from the player controller, then Apply Comfort Preset (Reduced Motion) and Set Comfort Group Scale (Camera Shake 0.5)

Changes apply at once to every recipe, including recipes already playing. With **Auto Save Comfort** on, the default, each change is also saved.

Besides the scales, each player's settings hold four options:

Option	Default	Effect
Limit Flashes, Max Flashes Per Second, Flash Limit Mode, Softened Flash Scale	On, 3, Soften, 0.3	The flash limiter (see 8.5 The flash limiter).
Allow Camera Roll	On	Off removes camera roll from every recipe.
Max Field Of View Change Per Second	0 (no limit)	The fastest zoom FeelKit may cause, in degrees per second (see 8.6 Motion comfort: camera roll and field of view speed).

8.4 Essential tracks and substitutes

Some feedback carries information the player needs, such as the flash that shows the player was hit. Turning that effect off must not remove the information. A track marked **Essential**, in the **Comfort** category of its settings, is treated differently when its group is turned down:

- With a **Substitute Step** and a scale of 0, the substitute plays instead. Choose a substitute from another group, for example a vignette or a sound instead of a flash, so the information still arrives in a form the player accepts. The substitute plays at the scale of its own group.
- Without a substitute, the scale never falls below the track's **Essential Floor**: at a floor of 0.3, a player who turns flashes off still sees the flash at 30 percent.
- On **Camera Shake** and **Camera Motion**, a player's choice always wins: at 0 these tracks stop completely, whatever the floor, because a little motion is still too much for a player who is made unwell by it. Give such tracks a substitute on another channel.

Tracks that are not essential stop playing when their group reaches 0.

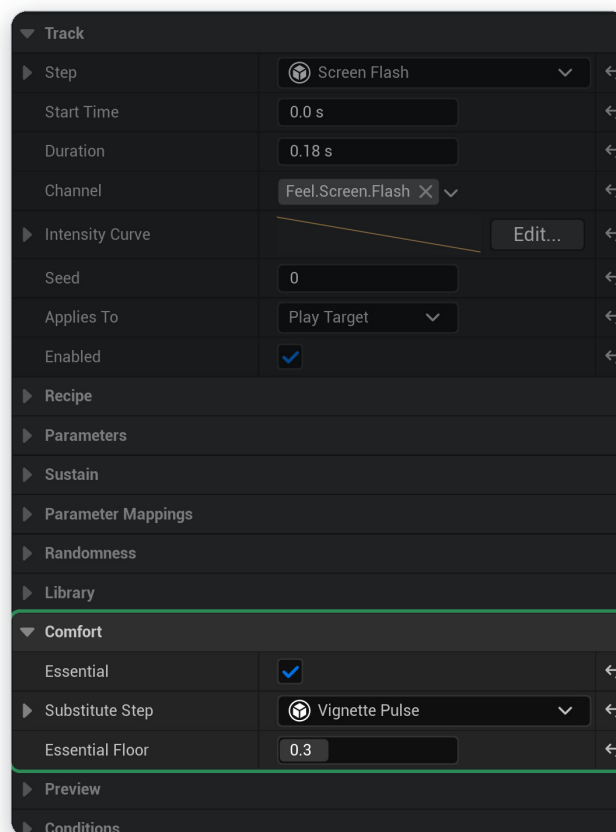


Figure 8.2 The Comfort category of the flash track of `FR_Dread_JumpScare`: Essential, a Vignette Pulse substitute and a floor

The library recipes `FR_Dread_JumpScare`, `FR_Danger_DamageTaken` and `FR_Danger_DirectionalDamage` each have an essential Screen Flash with a Vignette Pulse substitute. Saving a recipe reports an essential track that comfort settings could remove completely (5.12 [Validation messages on save](#)).

In the recipe editor, the **Comfort** menu of the preview plays a recipe with a preset, and tracks show what comfort does to them (5.9 [Checking comfort, and the intensity graph](#)). The substitute plays only when a group's scale is exactly 0. Of the built-in presets only **No Haptics** sets a scale of 0, so check a flash or motion substitute in the game, with that group set to 0.

8.5 The flash limiter

The flash limiter counts how many flashes start within a second: every track on a channel mapped to the **Flashes** group. When more start than **Max Flashes Per Second** allows (3 by default), the extra flashes are softened to **Softened Flash Scale** (0.3) or, in **Suppress** mode, not played. The limiter is on by default and is part of each player's settings, so a player can make it stricter.

In the recipe editor, flash tracks that the limiter changes say **Flash softened by the flash limiter** or **Flash suppressed by the flash limiter**.

The flash limiter reduces risk; it does not certify a game as safe. Test the finished game with a photosensitivity analysis tool.

8.6 Motion comfort: camera roll and field of view speed

Two options address motion sickness beyond the Camera Shake and Camera Motion scales:

- **Allow Camera Roll**, when off, removes every sideways tilt of the camera that FeelKit causes, wherever it comes from.
- **Max Field Of View Change Per Second** limits how fast FeelKit may zoom the view. A field of view kick that would change faster is slowed to the limit. 0 means no limit.

8.7 Engine camera shakes and controller vibration

A game usually has camera shakes and controller vibration of its own, played without FeelKit. FeelKit can apply the players' settings to them as well, so that one menu controls everything:

- **Apply Comfort to Engine Camera Shakes** (on) scales the game's own Unreal camera shakes by the player's Camera Shake and Master scales.
- **Apply Comfort to Engine Force Feedback** (on) scales the game's own controller vibration by the player's Haptics and Master scales. FeelKit multiplies the game's own force feedback scale rather than replacing it, and restores it when the setting changes.

When a player's settings silence the controller completely, the Output Log says so once, with the scales responsible. **Get Effective Force Feedback Scale** returns the combined scale, and the FeelKit Debugger shows it in the **Controller vibration** row ([Chapter 10, Debugging and tuning](#)). On

Windows, Unreal sends vibration only to Xbox-style controllers; [Chapter 14, Troubleshooting and questions](#) explains how PlayStation controllers are handled.

8.8 Saving settings, or using your own save system

With **Auto Save Comfort** on (the default), FeelKit saves each player's settings to a save game slot of its own, named after **Comfort Save Slot Prefix** and the player's index: `FeelComfort_0` for the first player. The settings are loaded when the local player is created, before any recipe plays, so they apply from the first frame.

A game with its own save system can store the settings there instead:

1. Create a class that implements the **Feel Comfort Storage** interface, with its two functions **Load Comfort Scales** and **Save Comfort Scales**. A Blueprint class works; so does a C++ class that implements `IFeelComfortStorage`.
2. Set **Comfort Storage Class** in **Project Settings > Plugins > FeelKit > Comfort > Storage** to that class, or call **Set Comfort Storage** on a player's Feel Comfort subsystem.

FeelKit then calls the class whenever settings are loaded or saved, and the save slot is not used.

8.9 The comfort menu for players

FeelKit includes a complete comfort menu, `WBP_FeelComfortMenu`, in **FeelKit Content > UI**. It contains:

Section	Rows
Master	Scales every effect at once.
EFFECTS	Camera shake, Camera motion, Flashes, Hitstop and slow motion, Screen distortion and Controller vibration : the six group scales, shown from 0 to 100 percent.
PRESETS	Default, Reduced motion, Reduced flashing and No vibration : the four built-in presets.
ADVANCED	Camera roll (Allow Camera Roll), Zoom speed (Max Field Of View Change Per Second; the far right means No limit) and Flash limiter (Limit Flashes).
Buttons	Try it plays a sample of the effects with the current settings; Reset returns to the project's defaults; Close closes the menu.

A line under the rows explains the selected setting in plain words, and moving a slider plays a short matching effect, so players feel the difference while they choose. Changes apply at once and are saved shortly after the player stops changing them. The menu works with the mouse, the keyboard and a controller; Esc, the controller's B / Circle button or its Menu button closes it.

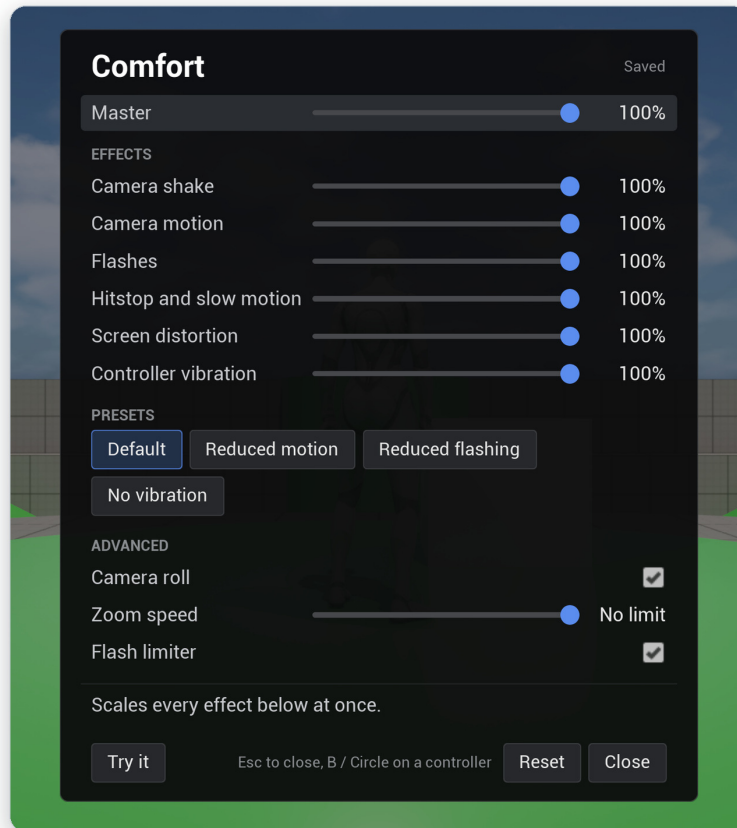


Figure 8.3 The comfort menu during play, opened with Show Feel Comfort Menu, at the project's default settings

Opening the menu. Call **Show Feel Comfort Menu** with the player controller. **Menu Class** chooses the menu, and when left empty uses **Comfort Menu Class** from the project settings, which is `WBP_FeelComfortMenu` unless changed. **Pause Game** pauses the game while the menu is open; the previews still play. The Feel Switch opens the same menu with O or the controller's Menu / Options button (6.11 The Feel Switch: feel off and on while playing).

8.10 Restyling the comfort menu

The comfort menu can be given the look of the game: its layout, fonts, colors and texts are all in the Widget Blueprint. Restyle a copy of it in the project, not FeelKit's own menu.

Why a copy. Fab installs FeelKit in the engine folder, not in the project. FeelKit's own `WBP_FeelComfortMenu` is therefore shared by every project that uses the same engine version, and the next FeelKit update replaces it with the original without warning. A change made to it would appear in every other project and then disappear at the next update.

Making the copy.

1. In the Content Browser, open **FeelKit Content > UI**, right-click `WBP_FeelComfortMenu` and choose **Copy to Project...**
2. Choose a folder and a name in the save dialog.

- When asked whether to use the copy as the project's comfort menu, click **Yes**. This sets **Comfort Menu Class** in the project settings, so **Show Feel Comfort Menu** and the Feel Switch open the copy. The choice can be changed later under **Project Settings > Plugins > FeelKit > Comfort**.

The copy opens in the UI designer and can be changed freely.

What must stay. The menu's logic finds its controls by name. A control that the copy removes is skipped, and the rest keeps working; a control that is renamed is treated as removed. The names are:

Kind	Names
Sliders	MasterSlider, CameraShakeSlider, CameraMotionSlider, FlashesSlider, HitstopSlider, ScreenDistortionSlider, HapticsSlider, ZoomSpeedSlider
Value texts	MasterValue, CameraShakeValue, CameraMotionValue, FlashesValue, HitstopValue, ScreenDistortionValue, HapticsValue, ZoomSpeedValue
Check boxes	CameraRollCheckBox, FlashLimiterCheckBox
Rows (for highlighting the selected row)	MasterRow, CameraShakeRow, CameraMotionRow, FlashesRow, HitstopRow, ScreenDistortionRow, HapticsRow, CameraRollRow, ZoomSpeedRow, FlashLimiterRow
Buttons	DefaultPresetButton, ReducedMotionPresetButton, ReducedFlashingPresetButton, NoHapticsPresetButton, TryButton, ResetButton, CloseButton
Texts	DescriptionText (the explanation line), SaveStatusText, CloseHintText

The menu's **Class Defaults** hold its remaining settings: the explanation of each row, the recipes played by **Try** and by each slider, the keys that close it, the row colors and the texts for "No limit", "Saved" and "Saving...".

Editing FeelKit's own menu also works, but only until the next FeelKit update.

8.11 The Comfort Audit PRO

Tools > FeelKit Comfort Audit checks every recipe and the comfort settings for likely comfort problems and lists them on the **FeelKit Comfort Audit** page of the Message Log. It warns about:

- more than three flashes starting within one second, in a recipe or in a sustain loop that repeats;
- saturated red flashes at an opacity above 0.5, the flashes most likely to cause photosensitive reactions;
- a track on a channel outside its step's comfort group, which players who turn that group down would still get;
- an essential flash with a floor and no substitute, which players who turn flashes off would still see;

- default settings without the flash limiter, and presets that do not reduce what their name promises.

It also notes when engine camera shakes or force feedback are left outside the comfort settings.

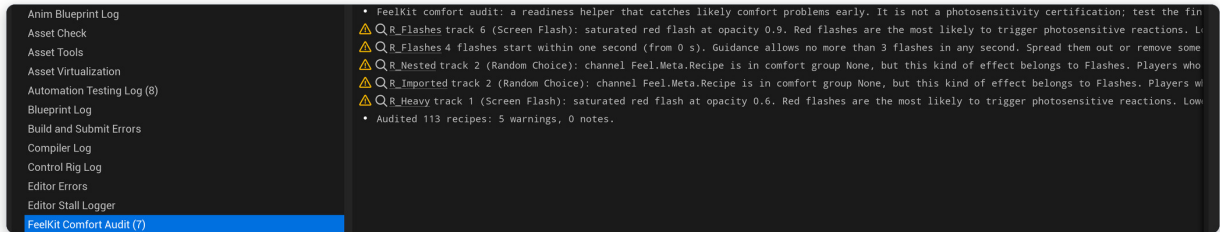


Figure 8.4 The FeelKit Comfort Audit page of the Message Log, here run on a project with test recipes

The audit is a readiness helper that catches common problems early. It is not a photosensitivity certification; test the finished game with an analysis tool.

9 The recipe library and browser

IN THIS CHAPTER

- 9.1 The Recipe Browser
- 9.2 Filters, search and live preview
- 9.3 Copying a library recipe into your project
- 9.4 Why library recipes are read-only
- 9.5 Content Browser tiles and tooltips
- 9.6 The 38 library recipes at a glance
- 9.7 Tagging your own recipes

FeelKit includes a library of 38 ready-made recipes, grouped by the feeling they produce. They are finished recipes that can be used as they are, and starting points for a project's own. FeelKit Lite includes a selection of them; FeelKit Pro includes all 38, together with the Recipe Browser, which shows and plays every recipe in one window.

9.1 The Recipe Browser PRO

Open the browser with **Tools > FeelKit Recipe Browser**, or right-click a folder in the Content Browser and choose **Browse Recipes....** It lists the library recipes and the project's own recipes side by side, and plays each one in a small preview.

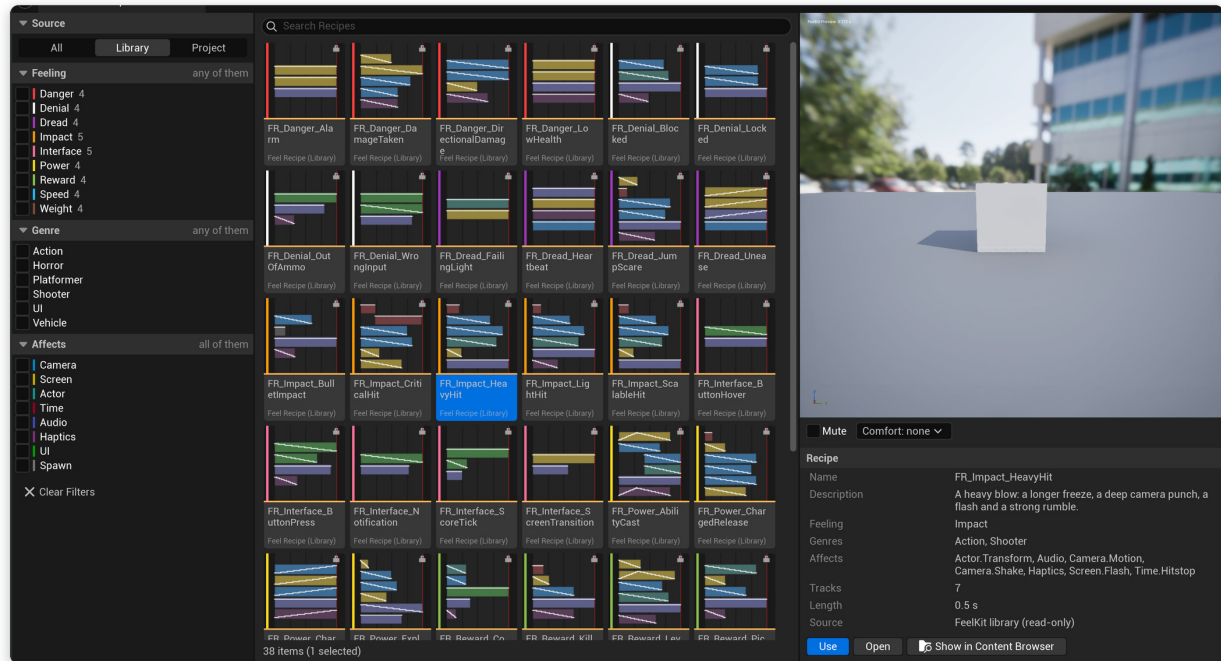


Figure 9.1 The Recipe Browser with `FR_Impact_HeavyHit` selected: filters on the left, the recipe list, and the preview with the recipe's details

The window has three areas:

Area	Contents
Filters (left)	Source , Feeling , Genre and Affects , described in see 9.2 Filters, search and live preview . Clear Filters shows every recipe again.
Recipe list (center)	One tile per recipe, with a search box above it and the number of recipes shown below it.
Preview (right)	The selected recipe playing in a preview, its details, a slider for each of its parameters, and the buttons Use , Open and Show in Content Browser .

The details list the recipe's **Name**, **Description**, **Feeling**, **Genres**, what it **Affects** (camera, screen, audio and so on), its number of **Tracks**, its **Length** and its **Source**: the FeelKit library (read-only) or this project.

9.2 Filters, search and live preview PRO

Filters. Each filter narrows the list:

Filter	Shows	With two or more picked
Source	All , Library (recipes that ship with FeelKit) or Project (the project's own recipes).	One choice at a time.
Feeling	Recipes of the picked feelings. The number next to each feeling counts its recipes.	Recipes with any of them.
Genre	Recipes suited to the picked genres.	Recipes with any of them.

Filter	Shows	With two or more picked
Affects	Recipes that affect the picked channels, such as camera shake or controller vibration.	Only recipes that affect all of them, for example shake and rumble.

Search. The search box matches recipe names, descriptions and parameter names, so a search for **Damage** finds the recipes that take a damage value.

Live preview. Resting the pointer on a tile for a quarter of a second plays that recipe once in the preview. Selecting a tile plays it in a loop. Parameter sliders under the preview play the recipe with other values, as in the recipe editor. Two controls of the browser change how the preview plays:

Mute silences the recipe's sounds, and the comfort menu plays it as a player with reduced motion, reduced flashing or no haptics would experience it ([Chapter 8, Comfort and accessibility](#)).

9.3 Copying a library recipe into your project

A project uses its own copy of a library recipe. Copies can be made in three ways:

- **In the Recipe Browser** (Pro): select the recipe and click **Use**, or double-click it.
- **In the recipe editor:** open the library recipe and click **Copy to Project** in the banner above the timeline ([4.1 Pick a recipe from the library](#)).
- **From a template** (Pro): right-click a folder in the Content Browser and choose **Recipe from Template...**. A window titled **Create a recipe from a template** shows the library in the same layout as the Recipe Browser; select a recipe and click **Create**.

A save dialog opens, with the name shortened to the part after the feeling: **FR_Impact_HeavyHit** becomes **HeavyHit**. The copy opens in the recipe editor and can be changed freely. Its **Based On** field records the library recipe it came from.

Some library recipes use sounds and materials from **FeelKit Content > Samples**. The copy refers to the same assets; duplicating them into the project is only necessary when they are to be changed.

9.4 Why library recipes are read-only

Fab installs FeelKit in the engine folder, which every project using that engine version shares, and a FeelKit update replaces the library. A change to a library recipe would therefore affect every project and would be lost at the next update. For this reason the library cannot be edited: its recipes open read-only, and the Content Browser does not save changes to them.

To author the library itself, for example in a studio that maintains its own version of FeelKit, turn on **Edit > Editor Preferences > Plugins > FeelKit > Allow Library Editing**. The setting is personal to each user and is off by default.

9.5 Content Browser tiles and tooltips

In the Content Browser, a recipe's tile shows a small picture of its timeline: one colored bar per track, in the channel colors of the recipe editor. Recipes can therefore be told apart at a glance, as [4.1 Pick a recipe from the library](#) shows for the Impact folder.

Resting the pointer on a tile shows a tooltip with the recipe's **Description, Feeling, Genres, Channels**, number of **Tracks, Length** and whether it is **Sustained**. These values are also asset registry tags, so the Content Browser's own filters and searches can use them.

9.6 The 38 library recipes at a glance

The descriptions are those shown in the Recipe Browser. Recipes marked sustained keep playing until they are released ([7.4 Sustained recipes](#)); parameters are listed where a recipe takes one.

Recipe	Description	Parameter
Impact		
FR_Impact_BulletImpact	A shot landing on a surface: a small punch, a mark on the surface and a short rumble.	
FR_Impact_CriticalHit	A critical hit: a longer freeze, a slow-motion beat, a bright flash and a color push.	
FR_Impact_HeavyHit	A heavy blow: a longer freeze, a deep camera punch, a flash and a strong rumble.	
FR_Impact_LightHit	A quick, light hit: a short shake, a small punch and a brief freeze.	
FR_Impact_ScalableHit	One hit that covers light to heavy: Damage drives how hard everything lands.	Damage
Weight		
FR_Weight_HeavyFootstep	Each step of something big: a small ground shake felt more than seen.	
FR_Weight_Land	Landing on the ground, scaled by how fast the fall was.	FallSpeed
FR_Weight_Slam	Something heavy hitting the ground nearby: a hard shake and a mark left behind.	
FR_Weight_Stomp	A deliberate, heavy stomp: a freeze, a drop of the camera and a long rumble.	
Power		
FR_Power_AbilityCast	Casting an ability: a short wind-up, a colored wash and a firm rumble.	
FR_Power_ChargeUp	Holding a charge: a rising hum, a tightening view and a growing rumble that keeps going until released. Sustained.	Charge

Recipe	Description	Parameter
FR_Power_ChargedRelease	The charge let go: a flash, a wide camera kick and a heavy rumble.	
FR_Power_Explosion	A blast going off nearby: the closer it is, the harder it hits.	Distance
Speed		
FR_Speed_Boost	Sustained speed: the view widens and the edges blur while the boost lasts. Sustained.	
FR_Speed_Dash	A burst of speed in the direction of travel: a camera push, a widening view and a whoosh.	
FR_Speed_SprintStart	Breaking into a sprint: a short lean and a rising sense of pace.	
FR_Speed_Whoosh	Something fast passing close by: a light camera turn and a sweep of sound.	
Reward		
FR_Reward_ComboStep	Each step of a streak feels bigger than the last, and the pitch climbs with it.	Combo
FR_Reward_KillConfirm	Confirmation that something went down: a crisp double tick and a short freeze.	
FR_Reward_LevelUp	A milestone reached: a warm flash, a lift of the camera and a triumphant sound.	
FR_Reward_Pickup	Collecting something: a bright, short pop with a rising sound.	
Danger		
FR_Danger_Alarm	An alarm going off: a repeating red wash and a warning tone. Sustained.	
FR_Danger_DamageTaken	Being hurt: a red wash at the edges, a jolt and a low rumble.	
FR_Danger_DirectionalDamage	Being hurt from a direction: the camera is pushed away from where it came from.	
FR_Danger_LowHealth	Running low: a slow pulse at the edges and a heartbeat that rises as health falls. Sustained.	Health
Dread		
FR_Dread_FailingLight	A light about to die: irregular flicker with a dip in color. Sustained.	
FR_Dread_Heartbeat	A heartbeat that follows fear: faster and heavier the closer the threat. Sustained.	Fear
FR_Dread_JumpScare	A sudden scare: one hard flash and a stab of sound, with a safe substitute when flashes are turned down.	
FR_Dread_Unease	Creeping unease: color drains, the edges close in and the world sounds muffled. Sustained.	

Recipe	Description	Parameter
Denial		
<code>FR_Denial_Blocked</code>	An action that will not happen: a short stop and a dull thud.	
<code>FR_Denial_Locked</code>	Something that will not open: a heavy rattle that goes nowhere.	
<code>FR_Denial_OutOfAmmo</code>	Nothing left to fire: a dry click and a small shake of refusal.	
<code>FR_Denial_WrongInput</code>	The wrong button: a red shake on the element that refused it.	
Interface		
<code>FR_Interface_ButtonHover</code>	The cursor arrives on a button: a small lift and a soft tick.	
<code>FR_Interface_ButtonPress</code>	A button taking the press: a squash inward, then a bounce back.	
<code>FR_Interface_Notification</code>	Something arrives on screen: it slides in, settles and chimes.	
<code>FR_Interface_ScoreTick</code>	A score counting up: numbers pop and the pitch climbs with the total.	<code>Combo</code>
<code>FR_Interface_ScreenTransition</code>	Moving between screens: a quick fade out and back in.	

Shared parameter names. Library recipes use the same names for the same kind of value, so one game value can drive several recipes: `Damage`, `FallSpeed`, `Health`, `Fear`, `Charge` and `Distance`. `Distance` is filled in automatically with the distance from the play to the nearest local camera (7.2 The Distance parameter).

The Combo accumulator. `FR_Reward_ComboStep` and `FR_Interface_ScoreTick` read the accumulator `Combo` when the game passes no value. A project that uses them adds `Combo` under **Project Settings** > **Plugins** > **FeelKit** > **Accumulators** and adds to it on each step of a streak (7.3 Accumulators: effects that build up with repeated plays). Until then, the recipes play with their default value and saving them reports that the accumulator is not defined.

Interface recipes. Most of them move, scale or tint a UMG widget, such as a button or a score counter, so they are played with a target from **Make Feel Target from Widget** (6.1 Play Feel, and choosing a target). `FR_Interface_ScreenTransition` fades the whole screen and needs no widget.

9.7 Tagging your own recipes PRO

A project's own recipes appear in the Recipe Browser beside the library, and its filters work for them when their library information is filled in. In the recipe editor, with no track selected, the **Library** category of **Details** holds:

- **Feeling:** one tag under `Feel.Feeling`, such as `Feel.Feeling.Impact`.
- **Genres:** any number of tags under `Feel.Genre`, such as `Feel.Genre.Shooter`.

- **Description:** one or two sentences, shown in the browser and in the Content Browser tooltip.

A project can add its own feelings and genres as gameplay tags under `Feel.Feeling` and `Feel.Genre` in **Project Settings** > **Gameplay Tags**; they then appear as filters in the browser. This information is used only by the editor and is not included in packaged games.

10 Debugging and tuning

IN THIS CHAPTER

- 10.1 The FeelKit Debugger
- 10.2 Replaying a recent play in the editor
- 10.3 Track labels that explain why a track did not play
- 10.4 `showdebug feel` and `stat Feel`
- 10.5 Log messages
- 10.6 Capture GIF

When a recipe does not play, plays too weakly or plays twice, the question is always the same: what did FeelKit decide, and why? Five tools answer it: the FeelKit Debugger during Play In Editor, replays of recorded plays in the recipe editor, the labels on track bars, the on-screen `showdebug feel` page, and the log. A sixth, Capture GIF, records a recipe for sharing.

The track labels (see [10.3 Track labels that explain why a track did not play](#)) and the log messages (see [10.5 Log messages](#)) are part of both editions. The Debugger, recent plays, `showdebug feel`, `stat Feel` and Capture GIF are part of FeelKit Pro.

10.1 The FeelKit Debugger PRO

The Debugger lists what FeelKit is doing in every running game world, and keeps a list of plays that have ended. Open it from **Tools > FeelKit Debugger**. It can stay open while the game runs; it updates four times a second.

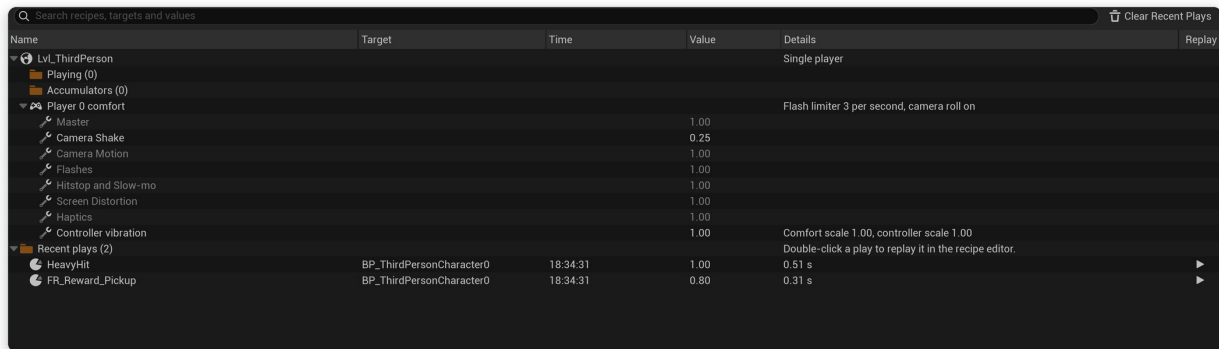


Figure 10.1 The FeelKit Debugger during Play In Editor: three recipes playing, the Combo accumulator, the player's comfort with Camera Shake at 0.25, and two recent plays

Each running world is one row, named after its level, with its role in the **Details** column: **Single player**, **Listen server**, **Client** or **Dedicated server**. A network test in Play In Editor shows one row per world, so the plays of the server and of each client can be compared side by side. Under each world:

Folder	Row	Columns
Playing	One row per play.	Target: the actor it plays on. Time: how far the play is, out of its length, such as 0.21 / 0.50 s . Value: the play's intensity. Details: Playing , Sustaining while a sustained recipe waits in its loop, or Stopping during a blend out, followed by the play's parameters, such as Damage 80.00 .
Accumulators	One row per accumulator value.	Target: the actor the value belongs to, or Global . Value: its current value.
Player N comfort	One row per group, from Master to Haptics .	Value: the player's scale. Groups at 1 are dimmed, so changed ones stand out. The player row's Details show the flash limiter and whether camera roll is allowed.
Controller vibration	Under each player.	Value: the scale that reaches the controller. Details: the comfort scale and the controller scale. When nothing reaches the controller, the row shows a warning icon and says so.

Below the worlds, **Recent plays** lists the plays that have ended, newest first (see [10.2 Replaying a recent play in the editor](#)). Each row names the recipe, the target, the time the play ended, its intensity, and in **Details** how long it lasted, whether it was released or stopped early, and its parameters.

The search box at the top filters every folder by recipe, target and value. **Clear Recent Plays** empties the list of recent plays. With no game running, the Debugger says **Nothing running** and still shows the recent plays.

Reading it. A few common findings:

- A recipe that should play is missing from **Playing**: the play never started. Check the log for a warning (see [10.5 Log messages](#)), and that the call reaches **Play Feel** at all.

- A play is listed but nothing is seen: check its **Value** (an intensity of 0 plays silently) and the player's comfort scales; then open the play from **Recent plays** and read the track labels (see [10.3 Track labels that explain why a track did not play](#)).
- A sustained recipe stays at **Sustaining**: nothing releases it ([7.4 Sustained recipes](#)).
- An accumulator never rises: its name is not defined in the project settings, and the log says so.
- The controller does not vibrate: read the **Controller vibration** row ([Chapter 14, Troubleshooting and questions](#)).

10.2 Replaying a recent play in the editor PRO

Every play that ends during Play In Editor is recorded, with everything needed to show it again: the random seed, the intensity, the parameters, the player's comfort scales, the flash limiter's decisions, the distance from the camera, whether the target was a local player's, whether the play had an instigator, and the view direction. The recipe editor can then replay that exact play in its preview and scrub through it, after play has stopped.

To open a recorded play:

- In the Debugger, double-click its row under **Recent plays**, or click the play button in its **Replay** column. The recipe opens and the replay starts.
- In an open recipe editor, open the **Recent Plays** menu in the Timeline toolbar and choose a play. Each entry shows how long ago the play ended, its intensity and its target.

While a replay is active, the menu reads **Replaying a Recorded Play** and the preview uses the recorded inputs in place of its own settings: the **Comfort** menu and the parameter sliders make no difference. Edits to the recipe apply to the replay at once, which makes it the quickest way to fix a play that felt wrong in the game: open it, change a track, and watch the same moment again. **Back to Normal Preview** in the same menu ends the replay.

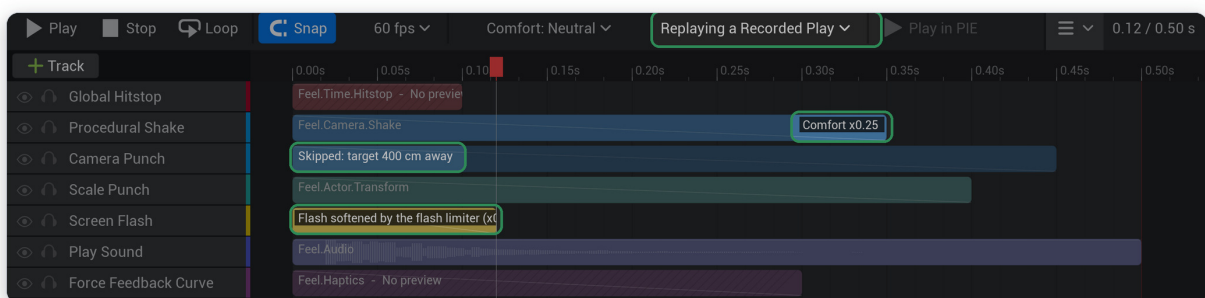


Figure 10.2 A replayed play of HeavyHit: the shake plays at a quarter because of the player's comfort, the camera punch was skipped by its distance condition, and the flash was softened by the flash limiter

FeelKit keeps the last 64 plays. The list lives until the editor closes, so plays from earlier Play In Editor sessions stay available. A replay needs its recipe: a play whose recipe asset has since been deleted is shown dimmed and cannot be opened.

10.3 Track labels that explain why a track did not play

A track bar in the recipe editor can carry a short label after its channel name. The label explains what the preview, or the replayed play, decided for that track. Only the first reason that applies is shown, in this order:

Label	Meaning
Muted	The track's mute button is on (5.4 Adding, moving and resizing tracks).
Silent: another track is soloed	Another track is soloed and this one is not.
Skipped: not on this platform	The track's Platforms condition does not list the current platform.
Skipped: target is not a local player's	Local Player Only is on and the play's target did not belong to a player on this machine.
Skipped: target N cm away	The target was farther from the local camera than the track's Max Distance .
Skipped this play (chance N%)	The track's Chance roll failed for this play.
Skipped: the play had no instigator	Applies To is Instigator and the play had none (7.5 The instigator, and the Applies To setting).
Removed by comfort settings	The player's scale for the track's group is 0 and the track is not essential (8.4 Essential tracks and substitutes).
Substitute plays (comfort)	The track is essential, its group's scale is 0, and its substitute step plays instead.
Flash suppressed by the flash limiter	The flash came faster than the player's limit allows, and the limiter is set to suppress (8.5 The flash limiter).
Flash softened by the flash limiter (xN)	The same, with the limiter set to soften: the flash plays at N of its strength.
Picked: ..., Picked nothing	A Random Choice track, and the option it picked for this play (7.7 Randomness, conditions and Random Choice).
Comfort xN	The track plays, scaled by the player's comfort to N of its strength.

Separately, **No preview** after the channel name marks a step that acts only in a running game, such as a hitstop or controller vibration: the preview skips it, and the game plays it.

The normal preview treats its target as the local player's, at an unknown distance and with an instigator, so only the **Chance** and **Platforms** conditions can skip a track there. Choosing a preset in the **Comfort** menu shows the comfort labels (5.7 The preview: play, loop, scrub, preview mesh, parameter sliders). A replayed play shows every label, from the values the game recorded.

10.4 showdebug feel and stat Feel PRO

Two console commands show FeelKit's state over the running game, which helps in a packaged development build where the Debugger is not available:

- `showdebug feel` draws the plays, accumulators and comfort of the local player over the game view, where Unreal draws its other `showdebug` pages. Enter it again to hide it.
- `stat Feel` shows the time FeelKit's update takes and the number of plays it runs. FeelKit updates only while something plays, so the time reads zero while it is idle.

Chapter 19, [Console commands](#) lists every line of both. Neither command is available in a Shipping build, like the rest of Unreal's debug display.

10.5 Log messages

FeelKit writes to the Output Log under **LogFeel**, and the Blueprint Event step under **LogFeelSteps**. Every message is a warning that names what to change. Filter the Output Log by `LogFeel` to see only these.

Message begins with	Cause	What to do
PlayFeel: ... was given an actor or component target that is not valid	The target actor or component was destroyed, or never set.	Check the value passed to the target node (6.1 Play Feel, and choosing a target).
Send Feel Event ...: no Feel Map entry matches, so nothing plays	No row of any Feel Map matches the event and its context tags. The message counts the maps that were searched. Shown once per event.	Add a row for the event, or check its required tags (6.5 Events and Feel Maps).
Send Feel Event ...: the matching Feel Map entry has no recipe	A row matched, but its Recipe is empty.	Set the recipe on that row.
Accumulator ... is not defined	A node or notify names an accumulator the project settings do not define.	Add it under Accumulators (7.3 Accumulators: effects that build up with repeated plays).
Controller vibration is off for this player	The player's Haptics or Master comfort is 0, so no vibration reaches the controller. Shown once, with the scales responsible.	Nothing, if the player chose it; otherwise check the comfort settings (8.7 Engine camera shakes and controller vibration).
Show Feel Comfort Menu needs a local player controller	The node was given a controller that is not a local player's.	Pass the local player's controller.
Show Feel Comfort Menu: no comfort menu to open	Comfort Menu Class is empty and the node passed no menu.	Set the class in the project settings (8.9 The comfort menu for players).

Message begins with	Cause	What to do
...: this level already has a Feel Switch	A second Feel Switch in the same level; it does nothing.	Keep one Feel Switch per level (6.11 The Feel Switch: feel off and on while playing).
Blueprint Event: ... has no event or function named ...	The Blueprint Event step names an event the target does not have.	Check the event name and the target (Chapter 16, Step reference).
Blueprint Event: ... must take no inputs or a single float	The event exists, but its inputs do not fit.	Give the event no inputs, or one float input for the intensity.

Saving a recipe also checks it and reports problems in the Message Log, on its **Asset Check** page ([5.12 Validation messages on save](#)).

10.6 Capture GIF PRO

Capture GIF in the Timeline toolbar records the preview twice, without and with the recipe, and saves both side by side as one animated GIF: the left half without the recipe, the right half with it. It shows the difference a recipe makes, for a design review, a bug report or a store page.

- Each half is 480 pixels wide, at 20 frames per second.
- The capture lasts the recipe's length plus a quarter of a second, between half a second and six seconds.
- The file goes to the project's `Saved/FeelKit/Captures` folder. A notification names it and offers **Show in Explorer**.
- Keep the preview visible while it records; a hidden preview cannot be captured, and the notification says so.

The GIF shows what the preview shows, so steps marked **No preview**, such as hitstops and controller vibration, are not in it, and sounds are not recorded. The comfort preset chosen in the preview applies to the capture.

11 Multiplayer

IN THIS CHAPTER

- 11.1 What travels over the network
- 11.2 The Feel Replication component and its modes
- 11.3 Relevancy distance
- 11.4 Hitstop and slow motion in networked games
- 11.5 Dedicated servers

Feedback is personal: each player sees their own screen, feels their own controller and has their own comfort settings. FeelKit therefore never sends effects over the network. It sends only the request to play, and every machine that receives it plays the recipe itself, with its own players' comfort. The sections below cover how requests travel with the Feel Replication component of FeelKit Pro, how far they reach, and how hitstops behave when several machines share one world.

11.1 What travels over the network

Play Feel plays on the machine that calls it, and nowhere else. In a networked game, that is often all that is needed: code that already runs on every machine, such as a replicated hit event or an animation notify, can call **Play Feel** on each of them, and each plays its own copy.

When a play has to start on one machine and reach others, the Feel Replication component of FeelKit Pro sends it. What travels is the request: the recipe or event, the target, the intensity, and the play context (parameters, instigator, direction, location, normal and context tags). Each receiving machine then plays the recipe with its own comfort settings, so a player who turned camera shake off never gets shake from another player's action.

Camera, screen and controller effects go to the local player of the play's target on each machine (2.4 [Targets: what a recipe plays on](#)). A target that belongs to another machine's player sends none of them on this machine; that player's own machine shows them.

11.2 The Feel Replication component and its modes PRO

Add a **Feel Replication** component to a replicated actor, such as a character or a weapon. It has two nodes:

- **Play Feel Networked** plays a recipe.
- **Send Feel Event Networked** sends a Feel Event; each machine picks the recipe from its own Feel Maps (6.5 Events and Feel Maps).

Both take the same inputs as their local counterparts, plus **Mode** and, under the node's advanced pins, **Relevancy Distance** (see 11.3 Relevancy distance). Both return the handle of the play on the calling machine, which is invalid when that machine does not play it.



Figure 11.1 A character with a Feel Replication component plays HeavyHit through Play Feel Networked, with Mode Everyone

Mode chooses which machines play:

Mode	Plays on
Everyone	Every machine.
Owner Only	Only the machine that owns the component's actor, for feedback meant for one player. The actor must belong to a player: their pawn, their controller, or something they own.
Skip Owner	Every machine except the owner's, for feedback the owner has already played locally. In a single-player game nothing plays.

Where to call it. The component routes the request by where it is called:

- **On the server**, the request goes to the machines the mode selects.
- **On the client that owns the actor**, that client plays at once, without waiting for the server, and the server forwards the play to the others. Feedback for the player's own action therefore has no network delay.

- **On any other client**, the play stays on that client.

Targets. The target must exist on every machine that plays: a replicated actor, or one placed in the level. A widget is local interface and is sent as no target.

Reliability. Requests are sent unreliably, like other cosmetic events. Under heavy packet loss, a remote machine can miss a play; the game's state never depends on one.

11.3 Relevancy distance PRO

Relevancy Distance keeps distant plays off machines that would not see them. A machine skips the play when all of its local players are farther than this from the play's target, measured from each player's character or camera, whichever is closer. A player's own character is always in range, and a play whose distance cannot be measured is played. 0, the default, means no limit.

For distance-dependent strength rather than a cut-off, use the **Distance** parameter, which each machine fills in with its own camera distance ([7.2 The Distance parameter](#)).

11.4 Hitstop and slow motion in networked games

In a networked game, world time is shared by every player on a machine, and on a listen server by every connected player. A global hitstop that slowed the whole world would freeze other players' games in the middle of their own actions, and a client's world time can be overridden by the server's.

So in networked games, **Global Hitstop** and **Slow-mo Ramp** slow only the play's target, on that machine, instead of the whole world. A single-player game always changes world time.

Allow Global Time Dilation in Multiplayer, under **Project Settings > Plugins > FeelKit > Playback**, changes world time anyway: on a listen server that slows every connected player, and on a client the server's time can override it. Use it for games where a shared freeze is intended, such as a cooperative finishing move. The setting is part of FeelKit Pro; in Lite, networked games always slow only the target.

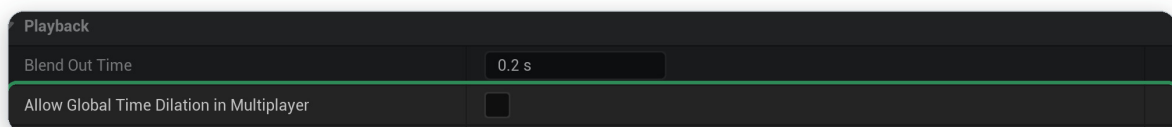


Figure 11.2 The Playback settings, with Allow Global Time Dilation in Multiplayer

11.5 Dedicated servers

A dedicated server has no players to show feedback to, so FeelKit plays nothing there: **Play Feel** returns no play, and the Feel Replication component forwards requests without playing them. Code shared between server and clients can call FeelKit without checking where it runs.

Testing. Play In Editor with **Net Mode** set to **Play As Listen Server** or **Play As Client** and two or more players runs a networked game on one computer. The FeelKit Debugger shows each world as its own row, labeled **Listen server**, **Client** or **Dedicated server**, with the plays of each ([10.1 The FeelKit Debugger](#)).

Split-screen, with several local players on one machine, has not been tested.

12 Performance

IN THIS CHAPTER

- 12.1 When FeelKit runs
- 12.2 What a play costs
- 12.3 Measuring it
- 12.4 Keeping many plays in check
- 12.5 Assets and memory

FeelKit runs a few short timelines at a time, so its cost is small next to the rendering and gameplay it decorates. The sections below show where the cost is, how to see it, and how to keep large numbers of plays in check. It gives no timing figures: FeelKit's cost depends on the recipes and the machine, and `stat Feel` measures it for any game.

12.1 When FeelKit runs

FeelKit updates once per frame while there is something to do, and not at all otherwise. Its update runs while any of these is true:

- a recipe plays, or waits to start;
- a hitstop or slow motion still owns the time it changed;
- an actor, widget, material or sound still has to be put back after an effect;
- a flash, fade, tint or post-process material is still fading out;
- a controller still vibrates;
- an accumulator is still decaying.

Once everything has ended and been restored, the update stops, and an idle FeelKit costs nothing per frame. An automated test checks this.

12.2 What a play costs

Each frame, FeelKit evaluates every active track of every playing recipe. Evaluation is arithmetic on the track's curve and step settings; it reads no assets and creates no objects. The results of all plays are then combined and delivered once per frame: one camera modifier per local player for camera effects, one screen overlay, one post-process blend, and the actor, widget, sound and controller changes.

A few steps create objects when their track starts, because the effect needs one:

Step	Creates
Play Sound	An audio component, or a one-shot sound, as Unreal's own sound nodes do.
Spawn Decal, Spawn Particle	A decal or a Niagara system, removed after its lifetime.
Number Pop	Nothing per number: all numbers are drawn by one layer over the game view, added the first time a number appears.
Material Parameter Pulse	A dynamic material instance for each material it changes that has none, unless the step writes custom primitive data instead. The original materials are put back afterwards.
Hit Flash	A dynamic instance of its flash material, set as the mesh's overlay material while the flash plays. The mesh's own overlay material is put back afterwards.
Post Process Material Pulse	A dynamic instance of its material, kept for later plays.

Every other step creates nothing.

12.3 Measuring it

In Play In Editor or a development build of a FeelKit Pro project, enter `stat Feel1`. It shows **Feel Tick**, the time of FeelKit's update in the current frame, and **Feel Instances**, the number of plays running ([Chapter 19, Console commands](#)). In both editions, Unreal Insights shows FeelKit's update in its timing view, next to everything else the frame does.

Measure in a development or test build, not in the editor: the editor adds its own cost to every frame, and it records each finished play for the Debugger's recent plays ([10.2 Replaying a recent play in the editor](#)), which development builds without the editor do as well and Shipping builds do not.

12.4 Keeping many plays in check

A recipe on every footstep, bullet impact or coin is fine; a hundred of them in one frame is wasted work that no player can tell apart. Each recipe has its own limits for this:

- **Max Concurrent** refuses a play once that many copies of the recipe play on the same target. 1 or 2 suits footsteps and hit reactions.
- **Cooldown** refuses a play on the same target for that many seconds after the last one.
- **Max Distance** on a track skips it for targets far from the camera, and the **Distance** parameter can fade it out on the way ([7.7 Randomness, conditions and Random Choice](#)).
- In a networked game, **Relevancy Distance** keeps far-away plays off other machines entirely ([11.3 Relevancy distance](#)).

Camera and screen effects of simultaneous plays compete rather than add up, by default ([2.2.2 Between separate plays](#)), so a burst of plays does not produce a burst of camera motion.

12.5 Assets and memory

A recipe is a small data asset. It loads with whatever references it, like any other asset, and its sounds, materials and particle systems load with it. Library recipes load only when a project uses them.

Editor-only data, such as a recipe's preview mesh, stays out of packaged games, and so does the whole FeelEditor module ([3.6 Blueprint-only projects and packaging](#)). The recipe tiles in the Content Browser are drawn from the recipe's data, without rendering a scene.

13 Demo levels

IN THIS CHAPTER

- 13.1 Getting the demos
- 13.2 Action/RPG: Weight Class
- 13.3 Platformer: Bounce Feel
- 13.4 Shooter: Every Bullet Has an Opinion
- 13.5 Horror: Heartbeat

Four demo levels show FeelKit in games of different kinds. Each is built on one of Unreal's own templates and adds FeelKit to it, so the feedback can be compared with the template as Epic ships it: every level has a Feel Switch that turns FeelKit off and on while playing ([6.11 The Feel Switch: feel off and on while playing](#)). The recipes are part of FeelKit Pro. Below, each level is described with what it shows, how it is hooked up, and what its recipes do, so the levels can serve as working examples.

13.1 Getting the demos

- **Playable builds** of all four levels are free downloads, linked from the FeelKit listings on Fab. They run without Unreal.
- **Source projects.** Owners of FeelKit can ask for the Unreal Engine 5.6 projects behind the levels, on the FeelKit Discord or by email ([14.8 Getting help](#)). They need FeelKit Pro installed in the engine.
- **Engine version.** The levels use Epic's 5.6 template content, whose levels crash in Play In Editor on 5.7 and 5.8 even without FeelKit. The source projects are therefore for 5.6 only. On 5.7 and 5.8, the recipes themselves work; they can be hooked into that engine's own templates in the same way.

In every level, **Tab** or the controller's **View** button turns FeelKit off and on, and **O** or the controller's **Menu** button opens the comfort menu (8.9 The comfort menu for players). Each level lists its controls under the FEEL ON / OFF badge.

13.2 Action/RPG: Weight Class

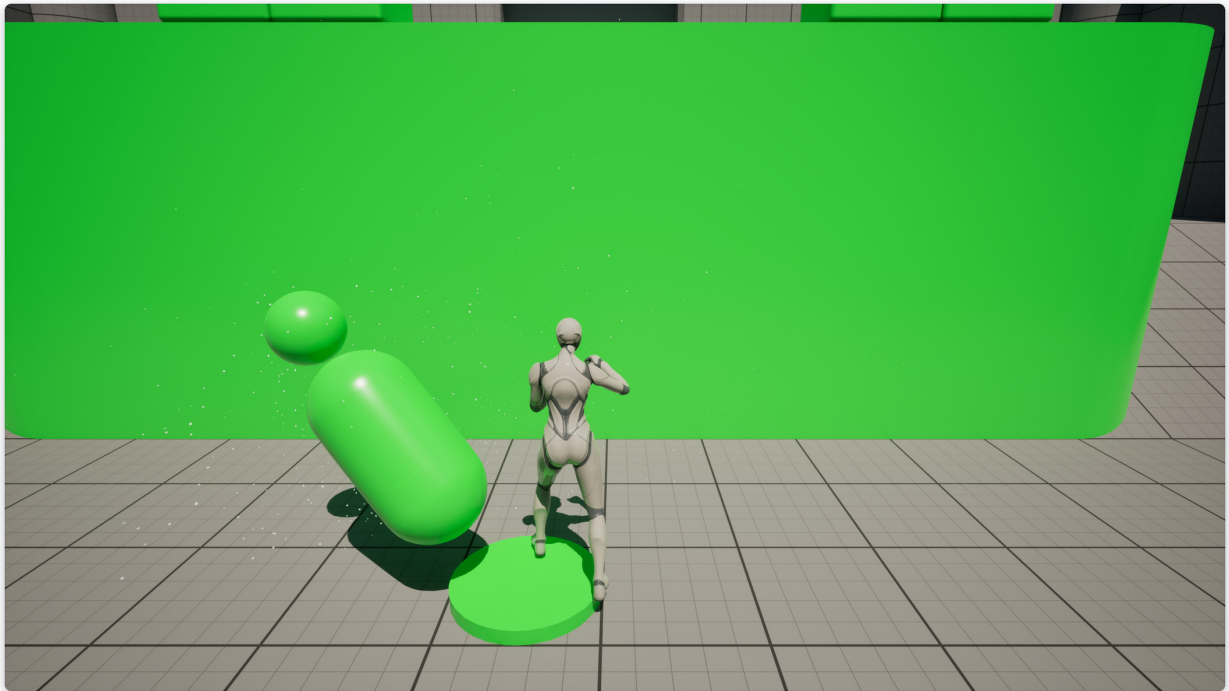


Figure 13.1 Action/RPG: a combo hit lands on the training dummy

Template: Third Person, Combat variant. **Controls:** move WASD or left stick; look mouse or right stick; attack left mouse or RB; charged attack hold right mouse or RT; block and parry hold Left Shift or LT.

The player fights training dummies and enemies with a three-hit combo and a charged attack, and can block and parry. Hits get heavier through the combo, a charged strike breaks an enemy's guard, and a well-timed block parries.

How it is hooked up.

- **Animation notifies** on the combo and charged-attack montages: **Set Feel Value** sets the accumulator `Swing` to 0.35, 0.65 and 1.0 on the three attacks, and `Charged` and `ChargeLevel1` during the charge; **Play Feel** plays the swing recipe (6.6 Animation notifies).
- **Feel Map** `FM_ARPG`: the template's damage events send `Feel.Event.Hit.Landed`, `Feel.Event.Hit.Received`, `Feel.Event.Hurt` and `Feel.Event.Death` (6.5 Events and Feel Maps).
- **Guards:** a guarding enemy and the player's block play the guard recipes on the defender.
- The player's Blueprint adds 1 to `Combo` on every hit (7.3 Accumulators: effects that build up with repeated plays).

Recipe	What it does
<code>FR_ARPG_Swing</code>	Every swing, hit or miss: a whoosh and a slight lean of the camera into the swing.
<code>FR_ARPG_HitLanded</code>	The player's hit connects. Light, heavy and finisher from one recipe, driven by <code>Swing</code> ; the charged strike adds a critical hit on top.
<code>FR_ARPG_EnemyHurt</code>	What was hit reacts: a hard white flash, a freeze, a squash and wobble, sparks, and a streak counter from the second hit on.
<code>FR_ARPG_EnemyDeath</code>	An enemy goes down: a freeze, a slow-motion beat with the color drained, a zoom, a heavy thud and a long rumble.
<code>FR_ARPG_ChargePulse</code>	One loop of holding a charged attack: a hum, a shake and a rumble that grow the longer it is held.
<code>FR_ARPG_GuardBlock</code>	A guard holds: the shield flares, sparks fly, a clang that deepens with heavier swings, a short freeze, and the attacker's swing stops dead.
<code>FR_ARPG_GuardBreak</code>	A charged strike breaks a guard: the shield shatters over a heavy clang, a freeze into slow motion, a warm flash.
<code>FR_ARPG_Parry</code>	A guard raised just before the hit turns it aside: a clear ring, a longer freeze, a white flash and a small zoom on the one who parried.

Project setup it relies on: the accumulators `Combo` (maximum 10, decay 1 per second after 1.5 s), `Swing`, `Charged` and `ChargeLevel` (maximum 1, no decay), `FM_ARPG` in the project's Feel Maps, and the Combat template's `NS_Damage` particle system for the sparks. In the source project these recipes live in the project, under `/Game/FeelKitDemos/ActionRPG`, because they refer to the template's content.

13.3 Platformer: Bounce Feel

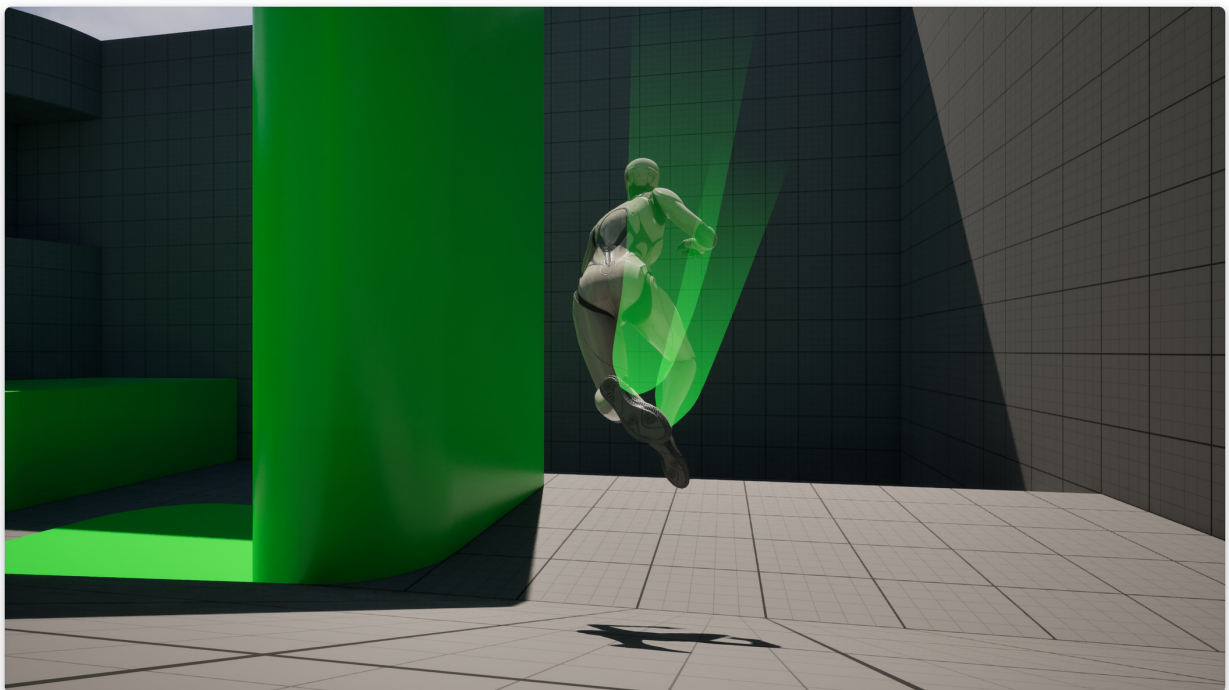


Figure 13.2 Platformer: a dash

Template: Third Person, Platforming variant. **Controls:** move WASD or left stick; look mouse or right stick; jump Space or A; double jump by jumping again in the air; wall jump by jumping against a wall; dash Left Shift or B.

Every movement has its own response, and landings scale with the fall: a hop lands lightly, a long drop hits the ground hard. Movement cameras push one way and settle, without shaking back and forth, which keeps a fast platformer comfortable.

How it is hooked up. A **Feel Trigger** component on the character plays the recipes on **Jumped**, **Air Jumped**, **Launched** (the template's wall jump) and **Landed**, with the landing speed passed as a parameter ([6.7 The Feel Trigger component](#)). The dash plays its recipe from a marker in the dash.

Recipe	What it does
<code>FR_PLAT_Jump</code>	A jump from the ground: the body stretches up, a quick whoosh with a scuff of the feet, the camera lifts a little.
<code>FR_PLAT_AirJump</code>	A jump in the air: a bigger stretch, a rising lift sound on top of the whoosh, the view widens and the camera pops up.
<code>FR_PLAT_WallJump</code>	A wall jump, or any launch: a split-second grip on the wall, the camera thrown along the jump, a thump off the wall and a whoosh.
<code>FR_PLAT_Dash</code>	A dash: the view stretches wide, color fringes at the edges, the camera lags behind, a deep whoosh and a rumble.
<code>FR_PLAT_Land</code>	Landing, scaled by the fall: a light step after a hop; a squash, a dip of the view and a thud after a real jump; a deep drop of the view after a long fall.

13.4 Shooter: Every Bullet Has an Opinion



Figure 13.3 Shooter: firing the rifle

Template: First Person, Shooter variant. **Controls:** move WASD or left stick; look mouse or right stick; fire left mouse or RT; switch weapon Left Shift or Y; jump Space or A.

Each weapon feels different, hits and kills confirm themselves, grenades shake the world by distance, and the interface reacts too: the bullet counter kicks with each shot and burns red when empty, the life bar flashes when hurt, the score jumps.

How it is hooked up. The template's C++ weapon, projectile, enemy and player classes got recipe slots, marked `// FeelKit` in the source project, which play the recipes at the right moments. The template's interface classes play the interface recipes on their widgets. Jump pads were added to the level.

Recipe	What it does
<code>FR_SHOOT_Pistol</code>	A pistol shot: a sharp crack over a low thump, the view kicks up and zooms in a touch, a short rumble.
<code>FR_SHOOT_Rifle</code>	A rifle shot, fired in bursts: a small kick per shot and a shake that builds up the longer the trigger is held.
<code>FR_SHOOT_Launcher</code>	A grenade launch: a deep boom, a heavy kick up and back, the view widens, a strong rumble.
<code>FR_SHOOT_DryFire</code>	The trigger pulled with nothing in hand: a dull click, nothing more.
<code>FR_SHOOT_Switch</code>	Switching weapons: a metal latch and a slight dip of the view.
<code>FR_SHOOT_Pickup</code>	Walking over a weapon pickup: a short chime and a light tap on the controller.

Recipe	What it does
<code>FR_SHOOT_Impact</code>	A bullet hits the world: a metallic ping over a dull thud, and a scorch mark that fades.
<code>FR_SHOOT_Hit</code>	The player hits an enemy: a crisp tick, the enemy flashes white and freezes for a moment, the damage pops up over it.
<code>FR_SHOOT_Kill</code>	The player takes an enemy down: a chime, a red flash on the enemy, a brief freeze into slow motion, a small zoom and a rumble.
<code>FR_SHOOT_Explosion</code>	A grenade explodes, scaled by the player's distance: a crunch over a low boom and a hard shake; up close, a freeze, a warm flash and color fringes.
<code>FR_SHOOT_EnemyFire</code>	Anyone else firing: the shot sounds from the shooter and fades with distance. No camera effects.
<code>FR_SHOOT_Hurt</code>	The player is hit: the view is knocked away from the shot, a red pulse at the edges, a body hit and a rumble.
<code>FR_SHOOT_Death</code>	The player dies: time slows, the color drains, the view closes in, a heavy fall and a long rumble.
<code>FR_SHOOT_JumpPad</code>	Stepping on a jump pad: a rising whoosh, the view widens on the way up, the camera pressed down for a moment, a smooth rumble.
<code>FR_SHOOT_Land</code>	Landing from a real drop: a camera dip and a thud that grow with the fall. Small steps stay silent.
<code>FR_SHOOT_HudShot</code>	One shot fired: the bullet counter kicks down; when the magazine runs low it flashes red with every shot.
<code>FR_SHOOT_HudEmpty</code>	The magazine ran dry: the bullet counter shakes and burns red.
<code>FR_SHOOT_HudReload</code>	The magazine is full again: the counter swells once and brightens, with a soft click.
<code>FR_SHOOT_HudHurt</code>	Life lost: the life bar shakes and flashes red.
<code>FR_SHOOT_HudScore</code>	A score changes: the number jumps and lands; gold for the player's point, red when the player's team lost someone.

Project setup it relies on: the accumulator `ShotHeat` (maximum 8, decay 10 per second after 0.12 s), which the rifle's shake reads, and the C++ changes to the template.

13.5 Horror: Heartbeat



Figure 13.4 Horror: sprinting through the corridor

Template: First Person, Horror variant. **Controls:** move WASD or left stick; look mouse or right stick; sprint hold Left Shift or LB; jump Space or A.

Quiet feedback that builds dread: the sprint wears the player out, failing lights buzz and drop out, doorways creak, and scare spots hit hard. The sprint meter reacts as well.

How it is hooked up. Sprinting plays a sustained recipe that lasts as long as the sprint, and running out of stamina plays a sustained out-of-breath recipe until it refills (7.4 Sustained recipes). Lamps, doorways and scare spots carry **Feel Trigger** components that play their recipes when the player comes near or steps through. The template's sprint meter plays the interface recipes.

Recipe	What it does
FR_HOR_Sprint	While sprinting: the view widens a touch, sways with the run, and heavy footfalls carry through the floor.
FR_HOR_OutOfBreath	Out of breath: a heavy heartbeat, the edges close in, color drains a little, the world sounds muffled, and the view heaves with each breath.
FR_HOR_LightUnease	Walking under a failing light: the lamp buzzes, surges and drops out for a moment, and the color goes cold.
FR_HOR_Doorway	Stepping through a doorway: the frame creaks somewhere above.
FR_HOR_Scare	A scare spot: the flashlight dies, the screen nearly goes black for a moment, a low boom, the heart jumps and the camera flinches back.
FR_HOR_Land	Landing after a real drop: a soft thud and a small dip that grow with the fall.

Recipe	What it does
<code>FR_HOR_HudBreathless</code>	The sprint meter ran empty: it beats red with the heartbeat until it has refilled.
<code>FR_HOR_HudRecovered</code>	The sprint meter is full again after running empty: it brightens once.

The Platformer, Shooter and Horror recipes ship with FeelKit Pro in **FeelKit Content > Demos**, so they can be opened and studied, or copied into a project, without the source projects.

14 Troubleshooting and questions

IN THIS CHAPTER

- 14.1 Nothing plays
- 14.2 Controllers and vibration
- 14.3 Camera and screen effects
- 14.4 Materials and decals
- 14.5 Blueprints, animations and GAS
- 14.6 Recipes and the library
- 14.7 Installing, packaging and engine versions
- 14.8 Getting help

The questions below are the ones that come up most often, grouped by what the problem looks like. Most answers start in the same two places: the Output Log, filtered by [LogFeel1](#) (10.5 [Log messages](#)), and, in FeelKit Pro, the FeelKit Debugger (10.1 [The FeelKit Debugger](#)).

14.1 Nothing plays

A recipe plays in the recipe editor but not in the game. Work through these in order:

1. FeelKit is switched on. The console variable `feel.Enabled` at 0 stops every play and refuses new ones ([Chapter 19, Console commands](#)); a Feel Switch showing **FEEL: OFF** does the same while the game runs ([6.11 The Feel Switch: feel off and on while playing](#)).
2. The call is reached. Put a **Print String** next to **Play Feel**, or look for the recipe under **Playing** in the Debugger.
3. The target is valid. A destroyed or empty target makes **Play Feel** refuse the play, and the log says so.

4. The recipe is not held back by its own limits: **Cooldown** refuses a second play on the same target within that many seconds, and **Max Concurrent** refuses a play once that many copies play on the target (0 means no limit).
5. The player's comfort does not remove the tracks. A group at 0 skips its tracks; the Debugger's **Player N comfort** rows show the scales.

Some tracks play in the editor but not in the game. The recipe editor's preview passes the **Local Player Only** and **Max Distance** conditions, since it has no real distance or player. In the game they apply. Open the play from **Recent plays** in the Debugger: the replay labels each skipped track with the reason ([10.3 Track labels that explain why a track did not play](#)).

Some tracks play in the game but not in the editor. Tracks marked **No preview**, such as hitstops, slow motion, controller vibration and Blueprint events, act only in a running game ([5.4 Adding, moving and resizing tracks](#)).

Send Feel Event plays nothing. No Feel Map row matches the event and its context tags, or the matching row has no recipe. The log names the event and the number of maps searched, once per event ([6.5 Events and Feel Maps](#)).

A parameter or accumulator stays at its default. The accumulator's name is not defined under **Project Settings > Plugins > FeelKit > Accumulators**, or the parameter's name in the recipe differs from the name the game passes. Names are compared exactly ([7.3 Accumulators: effects that build up with repeated plays](#)).

14.2 Controllers and vibration

A PlayStation, Switch or other controller does not vibrate on Windows. On Windows, Unreal sends vibration only to Xbox-style controllers (XInput). Other controllers vibrate when the game runs through Steam with Steam Input on, which translates them into Xbox-style controllers; this also applies to a packaged game started from Steam. FeelKit uses Unreal's standard force feedback, so it vibrates every controller that Unreal itself drives.

An Xbox controller does not vibrate either. Check, in this order:

- The player's **Haptics** or **Master** comfort is 0. The log says so once, with the scales, and the Debugger's **Controller vibration** row shows a warning ([8.7 Engine camera shakes and controller vibration](#)).
- The game has set the player controller's own **Force Feedback Scale** to 0. The same Debugger row shows it as the controller scale.
- The console command `showdebug forcefeedback` lists every running force feedback effect with its values, before the controller's scale is applied. FeelKit's effects appear there while they play.

14.3 Camera and screen effects

Camera effects of a recipe played on an enemy show up on my screen. Is that intended? Yes.

Camera, screen and controller effects go to the local player of the play's target. A target that belongs to no player, such as an enemy or a crate, sends them to the first local player, so the player sees the hit they caused. To keep them off the player's screen, give those tracks a **Local Player Only** condition, or split the recipe into one for the enemy and one for the player.

In multiplayer, another player's hit does not shake my camera. A target that belongs to a remote player sends no camera, screen or controller effects to this machine: that player's own machine shows them ([Chapter 11, Multiplayer](#)).

Camera Shake is at 0 in the comfort settings, but a recipe still moves the camera. A player's 0 on **Camera Shake** or **Camera Motion** always silences those groups, even for essential tracks. A track that still moves the camera is on a channel that is not mapped to either group: check its **Channel**, and the channel mapping in the project settings ([8.1 Comfort groups, and how the scales apply](#)).

Two recipes that move the camera make the view jump. With the default **Strongest Wins** arbitration, the stronger of two camera motions wins each frame. When two plays push the camera in different directions and their strengths cross, the view switches from one to the other within one frame. Push in one direction in both recipes, or switch **Camera Arbitration** to **Additive Capped** ([2.2.2 Between separate plays](#)).

During a hitstop, the shake keeps moving. That is intended: FeelKit times its effects in real time, so a freeze of the game does not freeze the feedback that sells it ([2.1 Recipes, tracks and steps](#)).

A hitstop in a network game slows only one character. In a networked game, a global hitstop or slow motion slows only the play's target on that machine, because slowing the whole world would put one machine out of step with the others. **Allow Global Time Dilation In Multiplayer** in the project settings changes that ([11.4 Hitstop and slow motion in networked games](#)).

14.4 Materials and decals PRO

A Post Process Material Pulse shows the material at full strength for the whole track. Unreal shows an active post-process material at full strength. The step fades it by writing its weight into a scalar parameter of the material, named **Weight** by default: add that parameter and use it as the Alpha of a **Lerp** from the scene (**Scene Texture: PostProcessInput0**) to the effect. The scene texture's color has four channels, so compare it with a **Constant4Vector**, not a **Constant3Vector**, or the material does not compile.

A Spawn Decal step shows nothing. Check three things:

- The material's domain is **Deferred Decal**.

- Its inputs are connected nodes. Values typed on the result node's pins without a connected node are ignored by decal materials, and the decal draws nothing. Connect a **Constant3Vector** to **Base Color**, for example.
- A surface is within reach: with **Find Surface** on, the step looks up to **Surface Search Distance** (500 cm) beyond the target for a surface to project on.

14.5 Blueprints, animations and GAS

The Blueprint Event step does nothing. The event name must match a custom event or function on the target exactly, and the event must take no inputs or a single float, which receives the intensity. The log names the target and the event when either is wrong ([10.5 Log messages](#)).

A widget reaction keeps restarting while the pointer rests near a button. A Widget Punch or Widget Shake on a button moves the button itself, and with it the area that takes the pointer: the pointer leaves the moved button, then enters it again, and the hover reaction restarts. Put a panel inside the button, let it draw the button's look, and play the reactions on the panel, not on the button. (Widget Punch and Widget Shake are part of FeelKit Pro.)

Does a scale punch or squash on a character change its collision? No. Actor effects on a character move and scale its mesh, never its collision capsule, so movement and hits are unaffected.

Gameplay Cue nodes fail to compile after adding the GAS add-on. **Execute Gameplay Cue On Actor**, **Add Gameplay Cue On Actor** and **Remove Gameplay Cue On Actor** have a **Parameters** pin that must be connected. Connect a **Make GameplayCueParameters** node to it ([6.9 Gameplay Ability System cues](#)).

14.6 Recipes and the library

A library recipe cannot be edited. Library recipes are read-only, so that an update of FeelKit never changes a project's game. Copy one into the project and edit the copy ([9.3 Copying a library recipe into your project](#)). **Allow Library Editing** in the editor preferences is for authoring the library itself.

Changes to a recipe during Play In Editor do not show at once. They apply from the next play of the recipe, without restarting Play In Editor.

Does the preview look exactly like the game? The preview and the game run the same code, so a track that plays in both looks the same. The differences come from the inputs: steps marked **No preview** act only in the game, the preview has its own comfort preset and parameter values, and its camera is not the game's camera. A replay of a recorded play removes the last two differences ([10.2 Replaying a recent play in the editor](#)).

14.7 Installing, packaging and engine versions

Packaging a Blueprint-only project fails or asks for Visual Studio. Any code plugin that the engine does not enable by default makes Unreal build a game executable for the project, which needs Visual Studio 2022 with the C++ game development workload ([3.6 Blueprint-only projects and packaging](#)).

The editor reports two plugins named FeelKit. FeelKit Lite and FeelKit Pro are the same plugin to Unreal, so only one of them can be installed per engine version. Remove Lite before installing Pro ([3.7 Moving a project from Lite to Pro](#)).

The GAS add-on asks to be rebuilt when the project opens. That is expected: the add-on compiles with the project. Click **Yes** ([3.5 Adding the GAS add-on](#)).

A demo project from FeelKit crashes in Unreal Engine 5.7 or 5.8. The demo projects are built on Epic's 5.6 template content, whose levels crash in Play In Editor on 5.7 and 5.8 even without FeelKit. Open them with 5.6. FeelKit itself runs on all three versions.

14.8 Getting help

When the answer is not here, ask on the FeelKit Discord, <https://discord.gg/AtJ6RdwaxA>, or write to billoue4@gmail.com. A report is easiest to act on with the engine version, the FeelKit version (**Edit > Plugins**), what was expected, what happened, and any `LogFeel` lines from the Output Log.

15 Lite and Pro

IN THIS CHAPTER

- 15.1 What each edition contains
- 15.2 What Lite leaves out
- 15.3 The Lite library
- 15.4 Recipes and data in Lite
- 15.5 Moving from Lite to Pro
- 15.6 Licenses

FeelKit comes in two editions on Fab. **FeelKit Lite** is free and complete in itself: the recipe editor, twelve effects, the whole comfort layer and a small library. **FeelKit Pro** adds the rest: every effect, the full library and its browser, the ways of hooking recipes to gameplay without Blueprint wiring, multiplayer, GAS, and the debugging tools. Both editions are the same plugin with the same modules and class names, so a project can start on Lite and move to Pro without changing anything.

This manual describes both. A section or table row that applies only to Pro carries a small **PRO** label; a section without one applies to both editions.

15.1 What each edition contains

Area	FeelKit Lite	FeelKit Pro
Recipe editor	Timeline, preview, intensity curves, scrubbing, mute and solo, copy and paste, comfort preview, Play in PIE, validation on save	The same, plus preview parameter sliders, sustain preview, Recent Plays, Capture GIF, sound waveforms and tracks from sound
Effects	12	All 37
Playing recipes	Play Feel on an actor, component, widget, location or camera; handles, Stop Feel , Stop All Feel , Is Feel Playing ; Cooldown and Max Concurrent	The same, plus Play Feel with Context , Play Feel and Wait , parameters, accumulators and sustained recipes

Area	FeelKit Lite	FeelKit Pro
Hooking up gameplay	Blueprint and C++ calls	The same, plus Feel Maps and events, animation notifies, the Feel Trigger and Feel Input components, and a Gameplay Ability System add-on
Multiplayer	Effects play on the machine that calls Play Feel	The Feel Replication component sends plays to other machines
Comfort	Every comfort setting, the presets, the flash limiter, engine camera shakes and force feedback, saving, custom storage, essential tracks, the comfort menu	The same, plus the Comfort Audit
Feel Switch	Yes	Yes
Library	11 recipes	38 recipes, the Recipe Browser and Recipe from Template
Tools	Content Browser recipe tiles and tooltips	The same, plus the FeelKit Debugger, <code>showdebug feel</code> , <code>stat Feel</code> , <code>feel.GlobalScale</code> , and JSON import and export
Demo recipes	None	The recipes of the Shooter, Horror and Platformer demo levels
Engine plugins it enables	None	Niagara and Enhanced Input

15.2 What Lite leaves out

Everything below is part of Pro only. Lite does not contain it at all, so it never appears in a Lite project's menus, pickers or node search.

Effects. Lite has Procedural Shake, Camera Punch, FOV Kick, Global Hitstop, Slow-mo Ramp, Screen Flash, Vignette Pulse, Scale Punch, Squash and Stretch, Play Sound, Force Feedback Curve and Blueprint Event. Pro adds:

Group	Effects
Camera	Camera Roll, Camera Zoom, Look-at Nudge
Time	Actor Hitstop
Screen	Chromatic Aberration, Desaturate, Color Tint, Screen Fade, Post Process Material Pulse
Actor	Material Parameter Pulse, Hit Flash, Mesh Wobble, Light Flash
Audio	Sound Class Duck, Pitch Bend, Low-pass Sweep
Controller	Haptic Pattern
UI	Widget Punch, Widget Shake, Widget Flash, Number Pop
Spawn	Spawn Decal, Spawn Particle

Group	Effects
Recipes	Play Recipe, Random Choice

Nodes and components. Play Feel with Context, Play Feel and Wait, Release Feel, Set Feel Parameter, Add to Feel Accumulator, Set Feel Accumulator, Get Feel Accumulator, Send Feel Event; the Feel Map asset; the Play Feel, Send Feel Event, Set Feel Value and Play Feel (Window) animation notifies; the Feel Trigger, Feel Input and Feel Replication components; the Gameplay Cue notifies of the GAS add-on; the editor scripting functions for JSON and project settings ([Chapter 17, Blueprint nodes and components](#)).

Recipe and track fields. Parameters, Sustain, the library fields (Feeling, Genres, Description, Based On), Applies To, Parameter Mappings, Randomness and Conditions.

Project settings. Accumulators, Allow Global Time Dilation in Multiplayer, and the Feel Maps list ([Chapter 18, Project settings and editor preferences](#)).

Editor tools. The Recipe Browser, Recipe from Template, the FeelKit Debugger, Recent Plays, Capture GIF, sound waveforms and tracks from sound, JSON import and export, and the Comfort Audit.

Console. `feel.GlobalScale`, `stat Feel` and `showdebug feel`. `feel.Enabled` is part of both ([Chapter 19, Console commands](#)).

15.3 The Lite library

FeelKit Lite ships 11 of the 38 library recipes: the ones built only from Lite's effects. They are listed with their descriptions in [9.6 The 38 library recipes at a glance](#).

Feeling	Recipes
Impact	<code>FR_Impact_LightHit</code> , <code>FR_Impact_HeavyHit</code>
Weight	<code>FR_Weight_Stomp</code> , <code>FR_Weight_HeavyFootstep</code>
Speed	<code>FR_Speed_SprintStart</code>
Reward	<code>FR_Reward_Pickup</code> , <code>FR_Reward_KillConfirm</code>
Danger	<code>FR_Danger_DamageTaken</code>
Dread	<code>FR_Dread_JumpScare</code>
Denial	<code>FR_Denial_Blocked</code> , <code>FR_Denial_Locked</code>

Lite also ships the seven sample sounds these recipes use, the comfort menu and its preview recipes. Lite has no Recipe Browser: the recipes are in the Content Browser under **Plugins > FeelKit Lite Content > Library**, where their tiles show their tracks. Open one and click **Copy to Project** in

the banner above the timeline to get an editable copy ([9.3 Copying a library recipe into your project](#)).

15.4 Recipes and data in Lite

Lite and Pro save recipes in the same format. A recipe keeps all of its fields in both editions: Lite hides the Pro fields from the editor but does not remove their values, so parameters, random ranges and conditions set in Pro survive a trip through Lite.

Recipes made in Lite open in Pro unchanged. The other way round has one limit: a recipe that uses an effect Lite does not have loses that track's step when it is opened in Lite, and saving it in Lite makes the loss permanent. Open such recipes in Pro only.

The **Play Recipe** and **Random Choice** steps are part of the evaluation core, so a Lite project still plays Pro recipes that contain them, as long as the steps they hold are Lite steps. Lite does not offer them when adding a track.

15.5 Moving from Lite to Pro

Remove FeelKit Lite from the engine and install FeelKit Pro to the same engine version. Every recipe, Blueprint node and setting carries over; the Pro features appear in the editor, and the Pro fields of existing recipes start at their defaults. Both editions are the same plugin to Unreal, so only one can be installed per engine version. The steps are in [3.7 Moving a project from Lite to Pro](#).

15.6 Licenses

Both editions are sold under Fab's Standard License. FeelKit Lite is free, has no time limit and no watermark, and games made with it can be sold. FeelKit Pro is paid, with Personal and Professional licenses as Fab offers them. The terms themselves are Fab's; the listing on Fab links to them.

PART TWO

Reference

- 16 Step reference
- 17 Blueprint nodes and components
- 18 Project settings and editor preferences
- 19 Console commands
- 20 Writing your own steps in Blueprint and C++
- 21 Recipe JSON files and editor scripting
- 22 Credits and licences
- 23 Version history

16 Step reference

IN THIS CHAPTER

- 16.1 Recipe fields
- 16.2 Track fields
- 16.3 Reading the step entries
- 16.4 Shared motion settings
- 16.5 Camera steps
- 16.6 Time steps
- 16.7 Screen steps
- 16.8 Actor steps
- 16.9 Audio steps
- 16.10 Haptics steps
- 16.11 UI steps
- 16.12 Spawn steps
- 16.13 Meta steps

16.1 Recipe fields

The recipe editor's **Details** tab shows these fields. **Tracks** is hidden there, because tracks are edited on the timeline: when a track is selected, its fields appear in the same tab and the **Recipe** category folds away.

Property	Default	Description
Recipe		
Tracks	empty (edited on the timeline)	Tracks of this recipe. Edited in the recipe timeline.
Cooldown	0 s	Seconds before this recipe can play again on the same target. At least 0.

Property	Default	Description
Max Concurrent	0	Maximum simultaneous instances per target. 0 means unlimited. At least 0.
Default Intensity	1	Multiplier applied to every play of this recipe. At least 0.
Schema Version	0	Version of the recipe data format, written on save. 0 for recipes saved before versioning. Read only. Under the advanced arrow of its category.
Parameters		
Parameters	empty	PRO Named numbers the game can pass each time the recipe plays. Tracks can scale their intensity by them through parameter mappings, and the recipe editor shows a slider for each to preview the range. A parameter named Distance is filled in automatically when the game does not pass it.
Sustain		
Sustain	Off	PRO Keeps the recipe playing while a condition lasts: time loops between Sustain Start and Sustain End until the play is released (Release Feel, the end of an anim notify state, or Stop Feel), then plays the rest of the recipe.
Sustain Start	0 s	PRO Start of the looping region, in seconds from the recipe start. At least 0. Editable only when Sustain is on.
Sustain End	1 s	PRO End of the looping region, in seconds from the recipe start. At least 0. Editable only when Sustain is on.
Library		
Feeling	None	PRO What this recipe makes the player feel. It groups the recipe browser and the recipe library, for example <code>Feel.Feeling.Impact</code> . Projects can add their own tags under <code>Feel.Feeling</code> . Editor only: not included in packaged games.
Genres	empty	PRO Kinds of game this recipe suits, for example <code>Feel.Genre.Shooter</code> . Used by the recipe browser filters. Editor only: not included in packaged games.
Description	empty	PRO One or two sentences describing the recipe, shown in the recipe browser and the Content Browser tooltip. Editor only: not included in packaged games.
Based On	None	PRO The library recipe this one was copied from, when it was created with Recipe from Template. Read only. Editor only: not included in packaged games.
Preview		
Preview Mesh	None	Mesh shown in the recipe editor preview. Static or skeletal; empty shows a cube. Editor only: not included in packaged games.

Each entry of **Parameters**:

Field	Default	Description
Name	None	Name the game uses to pass a value, and tracks use to read it. A parameter named Distance is special: when the game does not pass it, it is filled in with the distance in centimeters from the play's target to the nearest local player camera.
Default Value	0	Value used when the game does not pass one.
Min Value	0	Lowest meaningful value. Lower values are clamped to it. Track mappings read this as 0.
Max Value	1	Highest meaningful value. Higher values are clamped to it. Track mappings read this as 1.
Accumulator	None	Optional accumulator (Project Settings > Plugins > FeelKit > Accumulators) this parameter reads when the game does not pass a value. It is read every frame while the recipe plays: the value stored for the play's target actor, else the value of the play's instigator, else the global value. Leave empty to use the default value.
Description	empty	Explains what the value represents, for whoever sets it. Shown as the tooltip of the preview slider.

16.2 Track fields

Select a track on the timeline to see these fields in **Details**. The fields of the **Track** category appear under the track's number (**Track 1**, **Track 2** and so on), followed by **Parameter Mappings**, **Randomness**, **Comfort** and **Conditions**. The settings of the track's step are inside **Step**.

Property	Default	Description
Track		
Step	None (chosen with + Track)	The effect this track plays.
Start Time	0 s	Seconds from recipe start. At least 0.
Duration	0.5 s	Length in seconds. 0 means an instant step. At least 0.
Channel	the step's default channel	Channel of this track, for example <code>Feel.Camera.Shake</code> .
Intensity Curve	fades from 1 to 0	Intensity over the track's normalized time, from 0 at its start to 1 at its end. An empty curve means a constant 1. New tracks of steps that shape their own motion start with a flat curve at 1.
Seed	0	Seed for noise and randomness in the step.

Property	Default	Description
Applies To	Play Target	PRO Which actor this track's actor effects (scale, materials, actor time, attached sounds, Blueprint events) apply to: the play's target, or the Instigator actor passed in the play context. Camera, screen and haptic effects go to the local player of that actor.
Enabled	On	Editor mute. Disabled tracks produce no output, in the preview and in the game.
Parameter Mappings		
Parameter Mappings	empty	PRO Recipe parameters that scale this track's intensity. Each mapping multiplies the intensity; they combine with the intensity curve.
Randomness		
Random Intensity	Min 1, Max 1	PRO Intensity multiplier picked at random between Min and Max for each play. 1 to 1 means no variation.
Random Duration Scale	Min 1, Max 1	PRO Length multiplier picked at random between Min and Max for each play. 1 to 1 means no variation.
Comfort		
Essential	Off	Marks the track as carrying gameplay-critical information.
Substitute Step	None	Played instead when comfort settings disable this channel and the track is essential.
Essential Floor	0	Minimum intensity scale when the track is essential and has no substitute. Range 0 to 1.
Conditions		
Local Player Only	Off	PRO Only play when the target belongs to a local player.
Max Distance	0 cm	PRO Skip when the target is farther than this from the local camera. 0 means unlimited. At least 0.
Chance	1	PRO Probability that the track plays. Range 0 to 1.
Platforms	empty	PRO Platforms the track plays on. Empty means all platforms.

Each entry of **Parameter Mappings**:

Field	Default	Description
Parameter	None	Recipe parameter this mapping reads.
Curve	straight line from 0 to 1	How the parameter turns into an intensity multiplier. Horizontal axis: the parameter from its min (0) to its max (1). Vertical axis: the multiplier applied to the track. The default straight line from 0 to 1 means intensity grows in proportion to the parameter. An empty curve behaves like that line.

16.3 Reading the step entries

Every step below has the same parts:

- **Class** and module: the C++ class, for searching the source and for C++ code. The **+ Track** menu of the recipe editor lists steps by their display name, which is the heading of the entry.
- **Default channel**: the channel a new track of this step gets. You can change it on the track; comfort and arbitration follow the track's channel, not the step.
- **Comfort group**: the group that scales the default channel under the default **Channel Comfort Groups** in the project settings (see [18.5 Comfort](#)).
- **Editor preview**: whether the recipe editor's preview shows the step. Steps it cannot show still play in the game, and their tracks carry a **No preview** label.
- **Track length**: steps that act once when the track starts can sit on an instant track of length 0.
- **New tracks start with**: the intensity curve a new track of the step gets.
- The table: every setting of the step as the **Details** tab shows it, with its default and units, followed by what it does, its allowed range and when the editor shows it.
- **Checks when the recipe is saved**: problems with the step's settings that data validation reports.

Rotation values are listed as pitch, yaw and roll. The **Details** tab shows a rotator as X (roll), Y (pitch) and Z (yaw). Camera offsets are in camera space: X points forward, Y right and Z up.

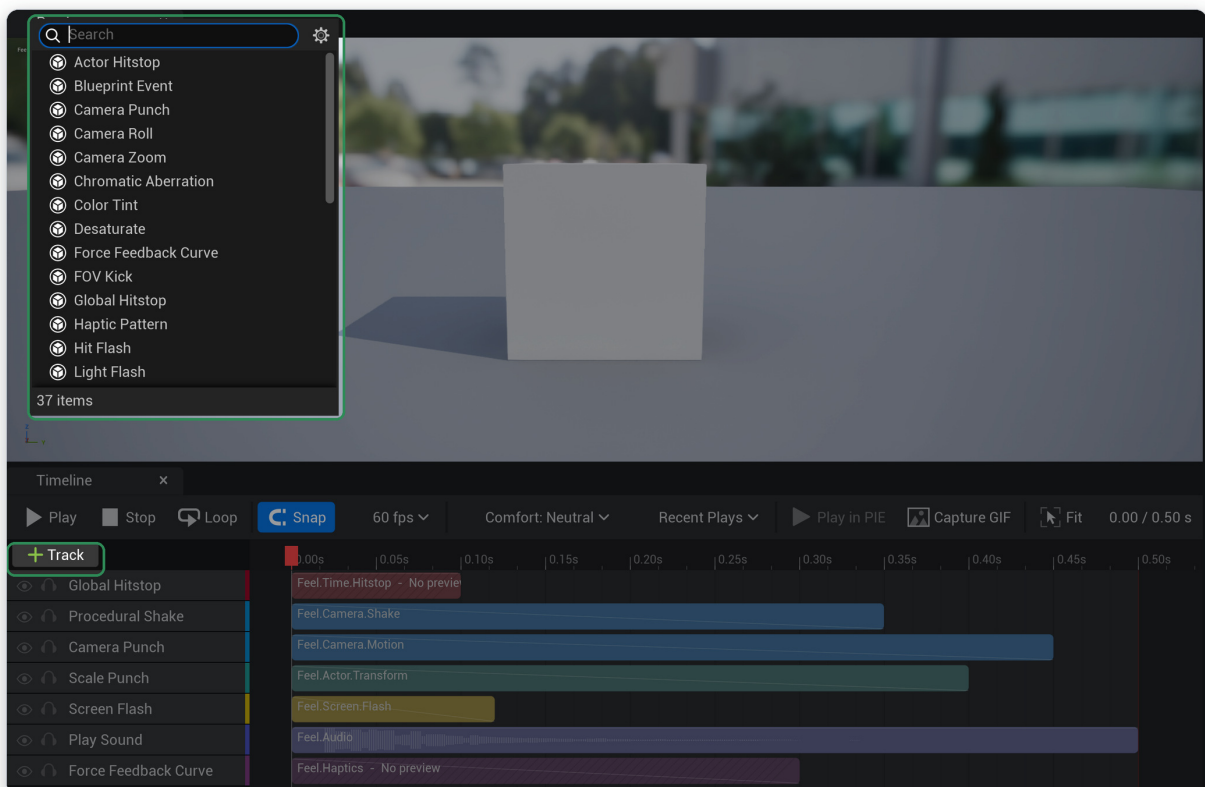


Figure 16.1 The + Track menu of the recipe editor, listing every step

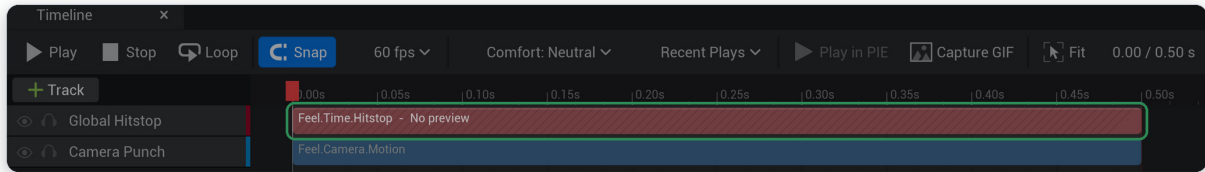


Figure 16.2 A Global Hitstop track, labeled No preview, above a Camera Punch track that previews

16.4 Shared motion settings

Camera Punch, FOV Kick and the other steps in the table below move along a motion shape. The shape gives the form of the motion over the track, from 0 to a peak of 1 and back; the step's own settings give the size at the peak, and the track's intensity multiplies the result. New tracks of these steps start with a flat intensity curve, so the shape alone sets the motion.

With **Spring**, the motion follows time in seconds, not the length of the track: a track shorter than the spring's settling time cuts the motion off, and a longer track stays at rest after it settles. **Kick** and **Smooth** stretch to fill the track. **Repeats** splits the track into equal parts and plays the shape once in each.

Property	Default	Description
Shape	Spring	How the motion moves over the track.
Frequency	5 Hz	Spring oscillations per second. At least 0.1. Shown only when Shape is Spring .
Damping	7	How quickly the spring settles. Higher values bounce less. At least 0. Shown only when Shape is Spring .
Attack Fraction	0.15	Portion of the track spent rising to the peak. Range 0.01 to 0.99. Shown only when Shape is Kick .
Repeats	1	Plays the shape this many times across the track, for heartbeats, double hits and flurries. Range 1 to 32.

Shapes:

- **Spring**: Springy overshoot that settles, like a physical hit.
- **Kick**: Fast rise to the peak, then an eased return.
- **Smooth**: Smooth rise and fall.

Default shape of each step that uses these settings:

Step	Default shape
Camera Punch	Spring
FOV Kick	Kick
Camera Roll	Spring

Step	Default shape
Look-at Nudge	Kick
Vignette Pulse	Smooth
Chromatic Aberration	Kick
Desaturate	Smooth
Color Tint	Smooth
Post Process Material Pulse	Smooth
Squash and Stretch	Spring
Material Parameter Pulse	Smooth
Hit Flash	Smooth
Light Flash	Kick
Force Feedback Curve	Kick
Widget Punch	Spring
Widget Flash	Kick

16.5 Camera steps

Camera steps offset the view of the target's local player, through a camera modifier that FeelKit adds to each local player's camera manager. Tracks of one play add up; separate plays are combined by the **Camera Arbitration** setting (see [18.2 Camera](#)).

16.5.1 Procedural Shake

Moves and turns the camera with noise or a sine wave computed from the track time and the seed, so the same seed gives the same shake every time and the preview shows exact frames when scrubbing. **Perlin** gives smooth random motion on each axis. **Sine** gives a regular oscillation with a random phase per axis. **Directional** moves back and forth along **Direction** only, with the rotation swinging in step. The intensity curve sets the strength over time; the default curve gives a shake that dies away over the track.

Edition	Lite and Pro	Default channel	Feel1.Camera.Shake
Comfort group	Camera Shake.	Editor preview	Yes.
Track length	Needs a length.		
Class	UFeelStep_ProceduralShake , module FeelCore.		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Mode	Perlin	Shape of the shake.
Frequency	12 Hz	Oscillations per second. At least 0.01.
Location Amplitude	(0, 3, 3) cm	Maximum location offset per camera axis (X forward, Y right, Z up). Shown only when Mode is not Directional .
Direction	(0, 0, 1)	Camera-space direction of a directional shake. Shown only when Mode is Directional .
Directional Amplitude	8 cm	Maximum offset along Direction. Shown only when Mode is Directional .
Rotation Amplitude	Pitch 1, Yaw 1, Roll 0.5 (degrees)	Maximum rotation offset in degrees per axis.
Field Of View Amplitude	0°	Maximum field of view change.

16.5.2 Camera Punch

Pushes and turns the camera to a peak and back along the motion shape; with the default Spring it overshoots and settles like a physical hit. **Location Punch** and **Rotation Punch** are the offsets at the peak. **Direction Source** can point the punch along the play's Direction, or away from or toward the play's Location, as seen from the camera when the play started. The strength stays that of the step settings, and the step settings are used when the play passed no direction or location. **Direction Jitter** turns the direction by a random angle for each play, which stays fixed for the whole play.

Edition	Lite and Pro	Default channel	Feel.Camera.Motion
Comfort group	Camera Motion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_CameraPunch , module FeelCore.		
Motion shape	Spring by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Location Punch	(-10, 0, -4) cm	Camera-space location offset at the peak (X forward, Y right, Z up).
Rotation Punch	Pitch -2.5, Yaw 0, Roll 0 (degrees)	Rotation offset at the peak, in degrees.
Direction Source	Step Settings	Where the punch direction comes from. With a play direction or location, the punch keeps the strength of Location Punch and Rotation Punch but points along that direction, as seen from the camera when the play started.
Direction Jitter	0°	Varies the punch direction per play by up to this angle, so repeated plays never look identical. Range 0 to 90.

16.5.3 FOV Kick

Changes the field of view by **Field Of View Kick** at the peak and back. Positive values widen the view, which reads as speed; negative values narrow it, which reads as an impact zoom. The default Kick shape rises fast and eases back.

Edition	Lite and Pro	Default channel	Feel.Camera.Motion
Comfort group	Camera Motion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_FOVKick , module FeelCore.		
Motion shape	Kick by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Field Of View Kick	8°	Field of view change at the peak. Positive widens (speed), negative narrows (impact zoom). Range -60 to 60.

16.5.4 Camera Roll PRO

Rolls the camera around its view axis to **Roll Degrees** at the peak and back. With **Random Direction** on, each play picks clockwise or counterclockwise from its seed. Players who turn **Allow Camera Roll** off in their comfort settings see no camera roll from any recipe.

Edition	Pro only	Default channel	Feel.Camera.Motion
Editor preview	Yes.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_CameraRoll , module FeelCore.		
Comfort group	Camera Motion; the roll is also removed completely when a player turns Allow Camera Roll off.		
Motion shape	Spring by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Roll Degrees	4°	Roll angle at the peak. Positive rolls clockwise. Range -45 to 45.
Random Direction	On	Pick clockwise or counterclockwise at random for each play.

16.5.5 Camera Zoom PRO

Eases the field of view by **Field Of View Change** over the first part of the track (**Ease in Fraction**), holds it, and eases back over the last part (**Ease Out Fraction**). Negative values zoom in. The track length sets how long the zoom holds.

Edition	Pro only	Default channel	Feel.Camera.Motion
Comfort group	Camera Motion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_CameraZoom , module FeelCore .		

Property	Default	Description
Field Of View Change	-10°	Field of view change while held. Negative zooms in, positive zooms out. Range -60 to 60.
Ease in Fraction	0.25	Portion of the track spent easing into the zoom. Range 0 to 1.
Ease Out Fraction	0.35	Portion of the track spent easing back out. Range 0 to 1.

16.5.6 Look-at Nudge PRO

Turns the camera part of the way toward the play's Location, or along the play's Direction when it passed no location, and back. **Turn Fraction** is the share of the angle to that point reached at the peak, and **Max Turn Degrees** limits the turn in yaw and in pitch. A play with neither a location nor a direction produces no motion.

Edition	Pro only	Default channel	Feel.Camera.Motion
Comfort group	Camera Motion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_LookAtNudge , module FeelCore .		
Motion shape	Kick by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Turn Fraction	0.2	Portion of the angle to the location that the camera turns at the peak. Range 0 to 1.
Max Turn Degrees	6°	Largest turn at the peak, per axis. Range 0 to 45.

16.6 Time steps

Time steps change time dilation. Recipes are timed in real time, so a track keeps its length and the rest of the recipe keeps playing while the world is slowed. When time requests overlap on the same clock, the highest **Priority** wins, and among equal priorities the slowest dilation wins. When no request remains, the clock gets back the dilation it had before FeelKit changed it. In networked games a global request slows only the play's target unless **Allow Global Time Dilation in Multiplayer** is on (see [18.3 Playback](#)).

16.6.1 Global Hitstop

Slows the whole world to **Time Dilation** while the track plays. The track's intensity, clamped to 0 to 1, blends between normal speed at 0 and **Time Dilation** at 1, so a flat curve gives a hard stop and a fading curve lets time recover over the track. The editor preview cannot slow time; use **Play in PIE** to judge a hitstop.

Edition	Lite and Pro	Default channel	Feel.Time.Hitstop
Comfort group	Hitstop and Slow-mo.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_GlobalHitstop , module FeelCore.		
Editor preview	No, the track shows No preview .		

Property	Default	Description
Time Dilation	0.05	Time dilation at full intensity. Small values freeze the action. Range 0.0001 to 1.
Priority	0	When hitstops on the same clock overlap, the highest priority wins.

16.6.2 Actor Hitstop PRO

Works like Global Hitstop, but slows only one actor through its custom time dilation, so the rest of the world keeps moving: the play's target, or the instigator when the track's **Applies To** is Instigator.

Edition	Pro only	Default channel	Feel.Time.Hitstop
Comfort group	Hitstop and Slow-mo.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_ActorHitstop , module FeelCore.		
Editor preview	No, the track shows No preview .		

Property	Default	Description
Time Dilation	0.05	Time dilation at full intensity. Small values freeze the action. Range 0.0001 to 1.
Priority	0	When hitstops on the same clock overlap, the highest priority wins.

16.6.3 Slow-mo Ramp

Eases the whole world into slow motion over **Ramp in Time**, holds at **Time Dilation**, and eases back to normal speed over **Ramp Out Time** before the track ends. Both ramps are real seconds with a smooth start and end. The intensity, clamped to 0 to 1, sets how far time slows: at 0.5 it slows half as far.

Edition	Lite and Pro	Default channel	Feel.Time.SlowMo
Comfort group	Hitstop and Slow-mo.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_SlowMoRamp , module FeelCore.		
Editor preview	No, the track shows No preview .		

Property	Default	Description
Time Dilation	0.3	Time dilation while fully slowed. Range 0.0001 to 1.
Ramp in Time	0.15 s	Real seconds to ease into slow motion at the start of the track. At least 0.
Ramp Out Time	0.3 s	Real seconds to ease back to normal speed before the track ends. At least 0.
Priority	0	When slow motions and hitstops overlap, the highest priority wins; equal priorities keep the slowest.

16.7 Screen steps

Screen steps change the whole view of the target's local player. Flashes and fades are drawn over the view; tints, vignette, chromatic aberration, saturation and post-process materials are blended into the view's post-process settings. Across separate plays the strongest contribution wins for each of these.

16.7.1 Screen Flash

Covers the view with **Color** at up to **Max Opacity**, drawn on top of the game's own camera fades. The intensity curve shapes the flash: the default curve starts at full opacity and clears over the track. Screen Flash tracks count toward the flash limiter of each player's comfort settings.

Edition	Lite and Pro	Default channel	Feel.Screen.Flash
Comfort group	Flashes.	Editor preview	Yes.
Track length	Needs a length.		
Class	UFeelStep_ScreenFlash , module FeelCore.		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Color	White (R 1, G 1, B 1)	Flash color.
Max Opacity	0.6	Opacity at full intensity. Range 0 to 1.

16.7.2 Vignette Pulse

Darkens the edges of the view by blending the scene's vignette toward **Vignette Intensity** and back. The engine's own vignette is about 0.4, so values near 1 give a strong darkening. With **Repeats** above 1 it pulses several times across the track, for example as a heartbeat.

Edition	Lite and Pro	Default channel	Feel.Screen.Distortion
Comfort group	Screen Distortion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_VignettePulse , module FeelCore.		
Motion shape	Smooth by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Vignette Intensity	1	Vignette intensity at the peak, blended over the scene's own vignette (the engine default is about 0.4). Range 0 to 1.

16.7.3 Chromatic Aberration PRO

Splits the colors toward the edges of the view by blending the scene's chromatic aberration toward **Fringe Intensity** and back. The default Kick shape rises fast and eases back.

Edition	Pro only	Default channel	Feel.Screen.Distortion
Comfort group	Screen Distortion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_ChromaticAberration , module FeelCore.		
Motion shape	Kick by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Fringe Intensity	3	Chromatic aberration intensity at the peak, blended over the scene's own setting. Range 0 to 5.

16.7.4 Desaturate PRO

Drains color from the scene and brings it back. **Amount** 1 makes the picture grayscale at the peak; the default 0.8 leaves a trace of color.

Edition	Pro only	Default channel	Feel.Screen.Color
Comfort group	Screen Distortion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_Desaturate , module FeelCore.		
Motion shape	Smooth by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Amount	0.8	How much color is removed at the peak: 1 is grayscale. Range 0 to 1.

16.7.5 Color Tint PRO

Multiplies the scene toward **Tint Color** and back. **Strength** sets how far the scene moves toward the tint at the peak. A white tint leaves the scene unchanged.

Edition	Pro only	Default channel	Feel.Screen.Color
Comfort group	Screen Distortion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_ColorTint , module FeelCore.		
Motion shape	Smooth by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Tint Color	R 1, G 0.4, B 0.35	Color the scene is multiplied toward. White leaves it unchanged.
Strength	1	How far the scene moves toward the tint at the peak. Range 0 to 1.

16.7.6 Screen Fade PRO

Fades the view to **Fade Color**, holds, and fades back: the first **Fade In Fraction** of the track fades in, the last **Fade Out Fraction** fades out, and the view holds at **Max Opacity** in between. With **Fade Out Fraction** at 0 the view stays faded until the track ends. The fade is drawn under flashes, so a flash during a fade stays visible.

Edition	Pro only	Default channel	Feel.Screen.Fade
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_ScreenFade , module FeelCore.		

Property	Default	Description
Fade Color	Black (R 0, G 0, B 0)	Color faded to.
Max Opacity	1	Opacity while held. Range 0 to 1.
Fade in Fraction	0.3	Portion of the track spent fading in. Range 0 to 1.
Fade Out Fraction	0.3	Portion of the track spent fading out. 0 stays faded until the track ends. Range 0 to 1.

16.7.7 Post Process Material Pulse PRO

Blends a post-process material over the scene and back, for effects the engine has no setting for, such as radial blur, scanlines or heat haze. The material must use the Post Process material domain. Unreal draws an active post-process material at full strength, so FeelKit passes the current weight, from 0 to 1, to the scalar parameter named in **Weight Parameter**. The material should use that value to fade its own effect, for example as the Alpha of a Lerp from Scene Texture PostProcessInput0 to the effect. The sample material `M_FK_PP_Pulse` in `/FeelKit/Samples/Materials` is set up this way.

Edition	Pro only	Default channel	<code>Feel.Screen.Distortion</code>
Comfort group	Screen Distortion.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	<code>UFeelStep_PostProcessMaterialPulse</code> , module <code>FeelCore</code> .		
Motion shape	Smooth by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Material	None	Post-process material to blend in.
Max Weight	1	Blend weight at the peak. Range 0 to 1.
Weight Parameter	<code>Weight</code>	Scalar parameter of the material that receives the current weight (0 to 1), so the material can fade its own effect, for example as the Alpha of a Lerp from the scene (Scene Texture: PostProcessInput0) to the effect. Unreal shows a post-process material at full strength whenever it is active, so a material without this parameter appears at full strength for the whole track. Set to None to blend the material directly.

Checks when the recipe is saved:

- Error: "Post Process Material Pulse has no material."

16.8 Actor steps

Actor steps change the actor the track applies to: the play's target, or the instigator when the track's **Applies To** is Instigator. On an actor whose root takes part in collision, such as a character's capsule, motion and scale go to its visible child components, so collision and movement stay unchanged. FeelKit restores every original value once no track changes it any more.

16.8.1 Scale Punch

Scales the target up by **Amount** and back with a springy swing: the scale starts and ends at the target's own scale, and each of the **Bounces** adds one more swing below and above it in between. **Amount** is relative, so 0.3 makes the target 30% larger at the peak. The intensity curve sets how quickly the swings die down.

Edition	Lite and Pro	Default channel	<code>Feel.Actor.Transform</code>
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.		
Class	<code>UFeelStep_ScalePunch</code> , module <code>FeelCore</code> .		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Amount	(0.3, 0.3, 0.3)	Relative scale change at the peak, per axis. 0.3 means 30% larger.
Bounces	1	Extra bounces after the first swing. Range 0 to 10.

16.8.2 Squash and Stretch

Stretches the target along its local **Axis** by **Amount** at the peak; negative amounts squash it. With **Preserve Volume** on, the other two axes thin out while it stretches and bulge while it squashes, so the target keeps its volume. With the default Spring shape a stretch overshoots into a squash before it settles.

Edition	Lite and Pro	Default channel	<code>Feel.Actor.Transform</code>
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	<code>UFeelStep_SquashStretch</code> , module <code>FeelCore</code> .		
Motion shape	Spring by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Axis	Z	Local axis the target stretches along.
Amount	0.3	Stretch at the peak. 0.3 makes the target 30% longer along the axis; negative values squash. A spring overshoots into the opposite. Range -0.9 to 3.
Preserve Volume	On	Keeps volume constant: the other axes thin out while stretching and bulge while squashing.

16.8.3 Material Parameter Pulse PRO

Pulses material parameters on the target's meshes and puts their original values back afterwards. For each entry of **Parameters**, a color parameter blends from its original color toward **Color**, or a scalar parameter gets **Scalar Amount** added at the peak. **Route** chooses how the values reach the materials: **Material Instance** creates dynamic material instances and works with any material parameter; **Custom Primitive Data** needs no material instances, but the material has to read the parameter as custom primitive data. When several tracks pulse the same parameter, scalar changes add up and the strongest color wins.

Edition	Pro only	Default channel	Feel.Actor.Material
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_MaterialPulse , module FeelCore.		
Motion shape	Smooth by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Parameters	one entry with the defaults below	Parameters pulsed together.
Route	Material Instance	How the parameters reach the materials.

Each entry of **Parameters**:

Field	Default	Description
Parameter Name	Color	Name of the material parameter.
Is Color	On	Blend a color parameter toward Color instead of offsetting a scalar parameter.
Scalar Amount	1	Added to the scalar parameter's original value at the peak. Shown only when Is Color is off.
Color	R 1, G 0.15, B 0.1	Color reached at the peak, blended from the parameter's original color. Shown only when Is Color is on.

Checks when the recipe is saved:

- Warning: "Material Parameter Pulse has no parameters, so it does nothing."
- Error: "Material Parameter Pulse has a parameter without a name."

16.8.4 Hit Flash PRO

Flashes the target's meshes with **Color** by drawing an instance of **Flash Material** in each mesh's overlay material slot, so it works on any mesh without changes to its own materials. The flash material must be a translucent overlay material with a vector parameter FlashColor and a scalar parameter FlashAmount from 0 to 1. The sample [M_FK_HitFlash](#) in /FeelKit/Samples/Materials is one. The mesh's own overlay material is put back afterwards.

Edition	Pro only	Default channel	Feel.Actor.Material
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_HitFlash , module FeelCore.		
Motion shape	Smooth by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Flash Material	None	Overlay material drawn over the meshes. It reads FlashColor (vector) and FlashAmount (scalar, 0 to 1).
Color	White (R 1, G 1, B 1)	Flash color at the peak.

Checks when the recipe is saved:

- Warning: "Hit Flash has no flash material, so it does nothing. Pick a translucent overlay material with FlashColor and FlashAmount parameters."

16.8.5 Mesh Wobble PRO

Tilts and moves the target back and forth around wherever it is, fading out over the track, so it also works on moving targets. **Tilt Amplitude** and **Move Amplitude** are the largest swings at the start, **Frequency** the swings per second, and **Decay** how quickly the swing dies: 0 keeps full strength, 1 fades linearly and higher values fade faster. Each axis swings with its own phase. **Noise** replaces the regular swing with irregular motion. A component whose physics simulates, such as a ragdoll, is left to physics.

Edition	Pro only	Default channel	<code>Feel.Actor.Transform</code>
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	<code>UFeelStep_MeshWobble</code> , module FeelCore.		

Property	Default	Description
Tilt Amplitude	Pitch 0, Yaw 0, Roll 8 (degrees)	Largest tilt around each local axis at the start, in degrees.
Move Amplitude	(0, 0, 0) cm	Largest movement along each local axis at the start, in centimeters.
Frequency	10 Hz	Wobbles per second. Range 0.1 to 60.
Decay	1.5	How quickly the wobble dies down over the track. 0 keeps full strength to the end; 1 fades linearly; higher fades faster. Range 0 to 8.
Noise	Off	Irregular noise motion instead of a regular back-and-forth swing.

16.8.6 Light Flash PRO

Brightens the target's lights and can shift their color, then returns them. **Intensity Change** is a multiple of each light's own intensity: 2 makes it three times as bright and -1 turns it off. It applies to the target when the target is a light component, and otherwise to every light component of the target actor. **Color Strength** moves the light color toward **Color**. With **Flicker** on, the light switches

between flashed and normal **Flicker Rate** times per second, in an on and off pattern taken from the seed. Light Flash tracks count toward the flash limiter under the default channel mapping.

Edition	Pro only	Default channel	<code>Feel.Actor.Light</code>
Comfort group	Flashes.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	<code>UFeelStep_LightFlash</code> , module <code>FeelCore</code> .		
Editor preview	No, the track shows No preview .		
Motion shape	Kick by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Light		
Intensity Change	2	Extra brightness at the peak, as a multiple of the light's own intensity: 2 makes it three times as bright, -1 turns it off. Applies to the target when it is a light component, otherwise to every light component of the target actor. Range -1 to 50.
Color	White (R 1, G 1, B 1)	Color the light moves toward.
Color Strength	0	How far the light color moves toward Color at the peak. 0 keeps the light's color. Range 0 to 1.
Flicker		
Flicker	Off	Rapidly switch between bright and normal while the track plays, like an unstable light.
Flicker Rate	18 Hz	Flicker changes per second. Range 1 to 60. Editable only when Flicker is on.

16.9 Audio steps

Audio steps play sounds or change the mix. They have no comfort group under the default mapping, so only Master scales them.

16.9.1 Play Sound

Plays **Sound** when the track starts, in 2D, attached to the target (at **Attach Socket Name** when set) or at the target's location. **Attached to Target** falls back to the target's location when the target has no component. Each play varies the volume and pitch at random by up to **Volume Variation** and **Pitch Variation**. With **Scale Volume with Intensity** on, the volume follows the track's intensity at the moment the track starts, comfort included. On a track with a length, **Stop at Track End** and **Stop when Recipe Stops** fade the sound out over **Fade Out Time**; on an instant track the sound simply plays to its end. The sound is audible in the editor preview.

Edition	Lite and Pro	Default channel	Feel.Audio
Comfort group	None, only Master applies.	Editor preview	Yes, the sound is audible.
Class	UFeelStep_PlaySound , module FeelCore .		
Track length	Can be an instant track (length 0).		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Sound		
Sound	None	Sound to play.
Placement	2D	Where the sound plays.
Attach Socket Name	None	Socket or bone to attach to. Leave empty for the component origin. Shown only when Placement is Attached to Target .
Volume Multiplier	1	Volume at full intensity. Range 0 to 4.
Scale Volume with Intensity	On	Scale the volume with the track's intensity, including comfort settings.
Pitch Multiplier	1	Pitch multiplier. Range 0.1 to 4.
Sound Start Time	0 s	Seconds into the sound to start playing. At least 0.
Attenuation Settings	None	Attenuation for positioned sounds. Leave empty to use the sound's own. Shown only when Placement is not 2D .
Concurrency Settings	None	Concurrency settings. Leave empty to use the sound's own.
Variation		
Volume Variation	0.05	Random volume change per play, as a fraction: 0.1 varies the volume by up to 10%. Range 0 to 1.
Pitch Variation	0.05	Random pitch change per play, as a fraction. A little variation keeps repeated sounds from feeling mechanical. Range 0 to 0.5.
Stopping		
Stop at Track End	Off	Fade the sound out when the track ends. Only for tracks with a length.
Stop when Recipe Stops	On	Fade the sound out when the recipe is stopped early. Only for tracks with a length.
Fade Out Time	0.15 s	Fade-out length when the sound is stopped. At least 0.

Checks when the recipe is saved:

- Error: "Play Sound has no sound."

16.9.2 Sound Class Duck PRO

Lowers the volume of **Sound Class** and its child classes, then brings it back. The change builds up over the first **Attack Fraction** of the track, holds, and releases over the last **Release Fraction**.

Volume Reduction 1 silences the class at full intensity. With **Sound Class** empty, the project's default sound class is used (Project Settings > Audio). FeelKit applies the change through a sound mix it creates while the track plays. When several tracks change the same class, the lowest volume wins.

Edition	Pro only	Default channel	Feel.Audio.Mix
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_SoundClassDuck , module FeelCore.		

Property	Default	Description
Sound Class	None	Sound class to adjust, including its child classes. Leave empty for the project's default sound class (Project Settings > Audio).
Attack Fraction	0.1	Portion of the track spent moving to the full adjustment. Range 0 to 1.
Release Fraction	0.4	Portion of the track spent returning to normal. Range 0 to 1.
Volume Reduction	0.6	Portion of the volume removed at full strength: 1 silences the class. Range 0 to 1.

16.9.3 Pitch Bend PRO

Bends the pitch of **Sound Class** and its child classes by **Pitch Change** and back, with the same attack, hold and release settings as Sound Class Duck. -0.3 plays 30% lower and 0.5 plays 50% higher. When several tracks change the same class, the pitch farthest from normal wins.

Edition	Pro only	Default channel	Feel.Audio.Mix
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_PitchBend , module FeelCore.		

Property	Default	Description
Sound Class	None	Sound class to adjust, including its child classes. Leave empty for the project's default sound class (Project Settings > Audio).
Attack Fraction	0.1	Portion of the track spent moving to the full adjustment. Range 0 to 1.
Release Fraction	0.4	Portion of the track spent returning to normal. Range 0 to 1.
Pitch Change	-0.3	Pitch change at full strength: -0.3 plays 30% lower, 0.5 plays 50% higher. Range -0.9 to 2.

16.9.4 Low-pass Sweep PRO

Filters the high frequencies of **Sound Class** and its child classes down to **Cutoff Frequency** at full strength, then opens the filter again, with the same attack, hold and release settings as Sound Class Duck. The cutoff moves between fixed levels rather than continuously: no filter (20000 Hz), 8000, 4000, 2000, 1000, 500 and 250 Hz. The level closest to the current cutoff is used. When several tracks change the same class, the lowest cutoff wins.

Edition	Pro only	Default channel	Feel.Audio.Mix
Comfort group	None, only Master applies.	Editor preview	Yes.
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	UFeelStep_LowPassSweep , module FeelCore.		

Property	Default	Description
Sound Class	None	Sound class to adjust, including its child classes. Leave empty for the project's default sound class (Project Settings > Audio).
Attack Fraction	0.1	Portion of the track spent moving to the full adjustment. Range 0 to 1.
Release Fraction	0.4	Portion of the track spent returning to normal. Range 0 to 1.
Cutoff Frequency	800 Hz	Cutoff frequency at full strength. Lower is more muffled. Range 100 to 20000.

16.10 Haptics steps

Haptics steps vibrate the controller of the target's local player. On Windows, Xbox controllers vibrate directly; PlayStation and other controllers vibrate when the game runs through Steam with Steam Input.

16.10.1 Force Feedback Curve

Vibrates the controller with four motor strengths, which are the values at full intensity. The motion shape (Kick by default) and the intensity curve shape the vibration over the track. **Ripple Frequency** adds regular dips in strength, for textures such as an engine or a heartbeat, and **Ripple Depth** sets how deep the dips go. When several plays vibrate the same controller, the strongest value wins for each motor.

Edition	Lite and Pro	Default channel	Feel.Haptics
Comfort group	Haptics.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_ForceFeedbackCurve , module FeelCore.		
Editor preview	No, the track shows No preview .		
Motion shape	Kick by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Left Large	1	Left heavy motor strength at full intensity. Range 0 to 1.
Left Small	0.4	Left light motor strength at full intensity. Range 0 to 1.
Right Large	1	Right heavy motor strength at full intensity. Range 0 to 1.
Right Small	0.4	Right light motor strength at full intensity. Range 0 to 1.
Ripple Frequency	0 Hz	Adds texture: the strength ripples this many times per second, like an engine or a heartbeat. 0 is a steady rumble. Range 0 to 60.
Ripple Depth	0.5	How deep the ripple dips, from 0 (no dip) to 1 (fully off at each dip). Range 0 to 1. Editable only when Ripple Frequency is above 0.

Checks when the recipe is saved:

- Warning: "Force Feedback Curve has every motor at 0, so it never vibrates."

16.10.2 Haptic Pattern PRO

Plays a Force Feedback Effect asset on the controller when the track starts, with the asset's own curves. With **Looping** on and a track length, the effect repeats until the track ends. **Stop with Track** stops it when the track ends or the recipe stops. The effect ignores time dilation, so hitstops do not stretch it. The track's intensity does not change the effect's strength. A player's Haptics comfort still applies: at 0 the track does not play, and with **Apply Comfort to Engine Force Feedback** on, the player controller's force feedback scale reduces this effect like any other vibration.

Edition	Pro only	Default channel	Feel.Haptics
Class	UFeelStep_HapticPattern , module FeelCore.		
Comfort group	Haptics; comfort can switch the effect off (Haptics at 0) but cannot make it weaker.		
Editor preview	No, the track shows No preview .		
Track length	Can be an instant track (length 0).		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Effect	None	Force feedback effect to play.

Property	Default	Description
Looping	Off	Repeat the effect until the track ends. Only for tracks with a length.
Stop with Track	On	Stop the effect when the track ends or the recipe stops, even if it is not looping.

Checks when the recipe is saved:

- Error: "Haptic Pattern has no force feedback effect."

16.11 UI steps

Widget Punch, Widget Shake and Widget Flash need a widget target (see [17.2.5 Make Feel Target from Widget](#)); on other targets they do nothing. They change the widget's render transform and color, which leaves the layout alone, and FeelKit restores both afterwards. Number Pop draws text over the game view of the target's player and works with any target.

16.11.1 Widget Punch PRO

Scales, moves and turns the widget target by **Scale Change**, **Translation** and **Angle Degrees** at the peak and back; the default Spring shape overshoots before it settles. Offsets from several tracks add up.

Edition	Pro only	Default channel	Feel.UI
Comfort group	None, only Master applies.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_WidgetPunch , module FeelCore.		
Editor preview	No, the track shows No preview .		
Motion shape	Spring by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Scale Change	(0.2, 0.2)	Scale change at the peak: 0.2 makes the widget 20% larger.
Translation	(0, 0) slate units	Movement at the peak, in slate units.
Angle Degrees	0°	Rotation at the peak.

16.11.2 Widget Shake PRO

Shakes the widget target with smooth noise on each axis. **Amplitude** is the largest movement at the start, **Frequency** the speed, and **Decay** how quickly the shake dies over the track: 0 keeps full strength and 1 fades linearly. **Angle Amplitude** adds rotation.

Edition	Pro only	Default channel	Feel.UI
Comfort group	None, only Master applies.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_WidgetShake , module FeelCore.		
Editor preview	No, the track shows No preview .		

Property	Default	Description
Amplitude	(8, 3) slate units	Largest movement at the start, in slate units.
Angle Amplitude	0°	Largest rotation at the start.
Frequency	30 Hz	Shake speed. Range 1 to 80.
Decay	1	How quickly the shake dies down: 0 keeps full strength, 1 fades linearly, higher fades faster. Range 0 to 8.

16.11.3 Widget Flash PRO

Tints the widget target toward **Color** and back. It applies to user widgets, images and text blocks (their color and opacity) and to borders (their brush color). **Strength** sets how far the color moves at the peak. When several tracks tint the same widget, the strongest wins.

Edition	Pro only	Default channel	Feel.UI
Comfort group	None, only Master applies.	Track length	Needs a length.
New tracks start with	A flat intensity curve at 1.		
Class	UFeelStep_WidgetFlash , module FeelCore.		
Editor preview	No, the track shows No preview .		
Motion shape	Kick by default; the shape settings are described in 16.4 Shared motion settings .		

Property	Default	Description
Color	R 1, G 0.2, B 0.2	Color at the peak.
Strength	1	How far the widget color moves toward Color at the peak. Range 0 to 1.

16.11.4 Number Pop PRO

Shows a number or short text that pops up at the target, or at the play's Location, and floats up over the game view of the target's player. With **Value Parameter** set, it shows that recipe parameter's value with **Decimals** digits after the point; otherwise it shows **Text**. **Prefix** and **Suffix** are added around it. The text appears at **Pop Scale** times its size and settles, rises by **Rise Distance** and gets a random sideways offset of up to **Spread**, so repeated pops do not cover each other. It stays for the length of the track, or for **Default Lifetime** on an instant track. With **Scale with Intensity** on, the font size follows the intensity, between 0.25 and 4 times **Font Size**.

Edition	Pro only	Default channel	Feel.UI
Comfort group	None, only Master applies.		
Class	UFeelStep_NumberPop , module FeelCore.		
Editor preview	No, the track shows No preview .		
Track length	Can be an instant track (length 0).		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Text		
Value Parameter	None	Parameter whose value is shown. Leave empty to show Text instead.
Decimals	0	Digits after the decimal point when showing a parameter value. Range 0 to 4.
Text	empty	Text shown when no value parameter is set.
Prefix	empty	Added before the text.
Suffix	empty	Added after the text.
Style		
Color	White (R 1, G 1, B 1)	Text color.
Font Size	28	Font size at intensity 1. Range 6 to 200.
Scale with Intensity	On	Grow the text with the track's intensity.
Placement		
Location	Play Location or Target	Where the text appears.
World Offset	(0, 0, 60) cm	World offset from that location, in centimeters.
Motion		
Rise Distance	60 slate units	How far the text floats up on screen over its lifetime, in slate units. At least 0.
Spread	24 slate units	Random sideways spread, in slate units, so repeated pops do not stack. At least 0.
Pop Scale	1.6	Size at the moment the text appears, relative to its settled size. Range 0.1 to 4.
Default Lifetime	1 s	Seconds the text stays when the track has no length. Tracks with a length use it instead. At least 0.1.

16.12 Spawn steps

Spawn steps create something in the world when the track starts and leave it to the engine afterwards.

16.12.1 Spawn Decal PRO

Spawns **Decal Material** at the play's Location, or at the target when the play passed none; with **Location** set to Target it always uses the target. With **Find Surface** on, FeelKit looks for a surface from that point along the projection direction, which is down unless the play passed a Normal, up to **Surface Search Distance** beyond the edge of the target, and places the decal on it, facing the surface. **Random Rotation** turns each decal around the normal. The decal stays for **Lifetime** and then fades over **Fade Out Time**. The decal material must use the Deferred Decal domain; `M_FK_Decal_Scorch` in `/FeelKit/Samples/Materials` is a sample.

Edition	Pro only	Default channel	<code>Feel1.Spawn</code>
Comfort group	None, only Master applies.	Editor preview	Yes.
Class	<code>UFeelStep_SpawnDecal</code> , module <code>FeelCore</code> .		
Track length	Can be an instant track (length 0).		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Decal Material	None	Decal material (Deferred Decal material domain).
Decal Size	(16, 48, 48) cm	Decal size: X is projection depth, Y and Z the width and height.
Location	Play Location or Target	Where the decal appears.
Lifetime	5 s	Seconds before the decal starts fading. At least 0.
Fade Out Time	1 s	Fade-out length. At least 0.
Random Rotation	On	Rotate the decal randomly around the normal for variety.
Find Surface	On	Looks for a surface from the spawn point along the direction the decal projects (down, unless the play gives a Normal), and places the decal on it with that surface's orientation. Useful because actor locations are usually above the ground: a character's location is the middle of its capsule. Off places the decal exactly at the spawn point.
Surface Search Distance	500 cm	How far to look for a surface, beyond the edge of the target (for a character: below its feet). At least 0. Editable only when Find Surface is on.

Checks when the recipe is saved:

- Error: "Spawn Decal has no decal material."

16.12.2 Spawn Particle PRO

Spawns the Niagara system **System** at the play's Location or at the target, or attached to the target at **Attach Socket Name** when **Attach To Target** is on. **Orient to Normal** turns the system's up axis to the play's Normal. **Parameters** pass values into float user parameters of the system when it

spawns: the value of a recipe parameter, or the track's intensity when **Recipe Parameter** is empty. With **Deactivate with Track** on and a track length, the system stops spawning new particles when the track ends or the recipe stops. The step is in the FeelNiagara module, which is enabled with FeelKit.

Edition	Pro only	Default channel	<code>Feel.Spawn</code>
Comfort group	None, only Master applies.	Editor preview	Yes.
Class	<code>UFeelStep_SpawnParticle</code> , module FeelNiagara.		
Track length	Can be an instant track (length 0).		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
System	None	Particle system to spawn.
Location	Play Location or Target	Where the system appears when not attached.
Attach to Target	Off	Follow the target component instead of staying where it spawned.
Attach Socket Name	None	Socket or bone to attach to. Editable only when Attach to Target is on.
Orient to Normal	On	Point the system's up axis along the play context's Normal when the play passed one.
Scale	(1, 1, 1)	Scale of the spawned system.
Parameters	empty	Values passed into user parameters of the system.
Deactivate with Track	On	Stop the system spawning new particles when the track ends or the recipe stops. Only for tracks with a length.

Each entry of **Parameters**:

Field	Default	Description
User Parameter	None	Float user parameter of the system, without the "User." prefix.
Recipe Parameter	None	Recipe parameter whose value is passed. Leave empty to pass the track's intensity.

Checks when the recipe is saved:

- Error: "Spawn Particle has no particle system."

16.13 Meta steps

Meta steps play other steps and recipes, or call into game code.

16.13.1 Play Recipe PRO

Plays another recipe inside this track. The inner recipe starts with the track and plays for as long as the track lasts, so give the track at least the inner recipe's length. It uses the same target, context and parameters, and **Intensity Scale** multiplies its intensity on top of this track's intensity. Its tracks keep their own channels and comfort settings. Recipes nest up to a depth of 4; deeper tracks are skipped, which also stops a recipe from playing itself without end.

Edition	Pro only	Default channel	<code>Feel.Meta.Recipe</code>
Track length	Needs a length.	New tracks start with	A flat intensity curve at 1.
Class	<code>UFeelStep_Recipe</code> , module <code>FeelCore</code> .		
Comfort group	None for the track itself; the inner recipe's tracks keep their own channels and comfort groups.		
Editor preview	Yes, exactly like the inner recipe.		

Property	Default	Description
Recipe	None	Recipe to play inside this track.
Intensity Scale	1	Multiplies the inner recipe's intensity, on top of this track's intensity curve. At least 0.

Checks when the recipe is saved:

- Error: "Play Recipe has no recipe."
- Error: "Play Recipe plays the recipe it is in."

16.13.2 Random Choice PRO

Plays one of its **Options**, picked at random for each play by **Weight**: an option with weight 2 is picked twice as often as one with weight 1. The pick is fixed for the whole play, so scrubbing shows the same option, and every new play picks again. Options without a step or with weight 0 are never picked. Checks on save include the settings of every option's step.

Edition	Pro only	Comfort group	The group of the track's channel.
Track length	Needs a length.		
Class	<code>UFeelStep_RandomChoice</code> , module <code>FeelCore</code> .		
Default channel	The default channel of the first option's step, or <code>Feel.Meta.Recipe</code> when there is none.		
Editor preview	As the step of the option picked for the preview play.		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Options	empty	Steps to choose from.

Each entry of **Options**:

Field	Default	Description
Step	None	Step played when this option is chosen.
Weight	1	Relative chance of this option. An option with weight 2 is picked twice as often as one with weight 1. At least 0.

Checks when the recipe is saved:

- Warning: "Random Choice has no option with a step and a weight above 0, so it plays nothing."

16.13.3 Blueprint Event

Calls a custom event or function by name when the track starts (**Start Event Name**) and when it ends (**Stop Event Name**). **Receiver** chooses the object that is called: the target actor, the pawn or the player controller of the target's local player, or the Level Blueprint. The event must take no inputs, or a single float that receives the track's intensity; with any other inputs, or when the name is not found, nothing is called and a warning appears in the log under LogFeelSteps. With **Call Stop Event when Interrupted** on, the stop event is also called when the recipe is stopped early.

Blueprint calls need a game world, so the editor preview does not make them.

Edition	Lite and Pro	Default channel	Feel.Meta.Event
Comfort group	None, only Master applies.		
Class	UFeelStep_BlueprintEvent , module FeelCore.		
Editor preview	No, the track shows No preview .		
Track length	Can be an instant track (length 0).		
New tracks start with	An intensity curve that fades from 1 to 0.		

Property	Default	Description
Receiver	Target Actor	Object that receives the calls.
Start Event Name	None	Custom event or function called when the track starts. Leave empty to skip.
Stop Event Name	None	Custom event or function called when the track ends. Leave empty to skip.
Call Stop Event when Interrupted	On	Also call the stop event when the recipe is stopped early.

Checks when the recipe is saved:

- Warning: "Blueprint Event has no start or stop event name, so it does nothing."

17 Blueprint nodes and components

IN THIS CHAPTER

- 17.1 Playing and stopping recipes
- 17.2 Targets
- 17.3 Play context
- 17.4 Parameters and accumulators
- 17.5 Events and Feel Maps
- 17.6 Comfort
- 17.7 Feel Switch
- 17.8 Animation notifies
- 17.9 Feel Trigger component
- 17.10 Feel Input component
- 17.11 Feel Replication component
- 17.12 Gameplay Cue notifies (GAS add-on)
- 17.13 Editor scripting
- 17.14 C++ access

17.1 Playing and stopping recipes

These nodes are in the **Feel** category of the Blueprint context menu.

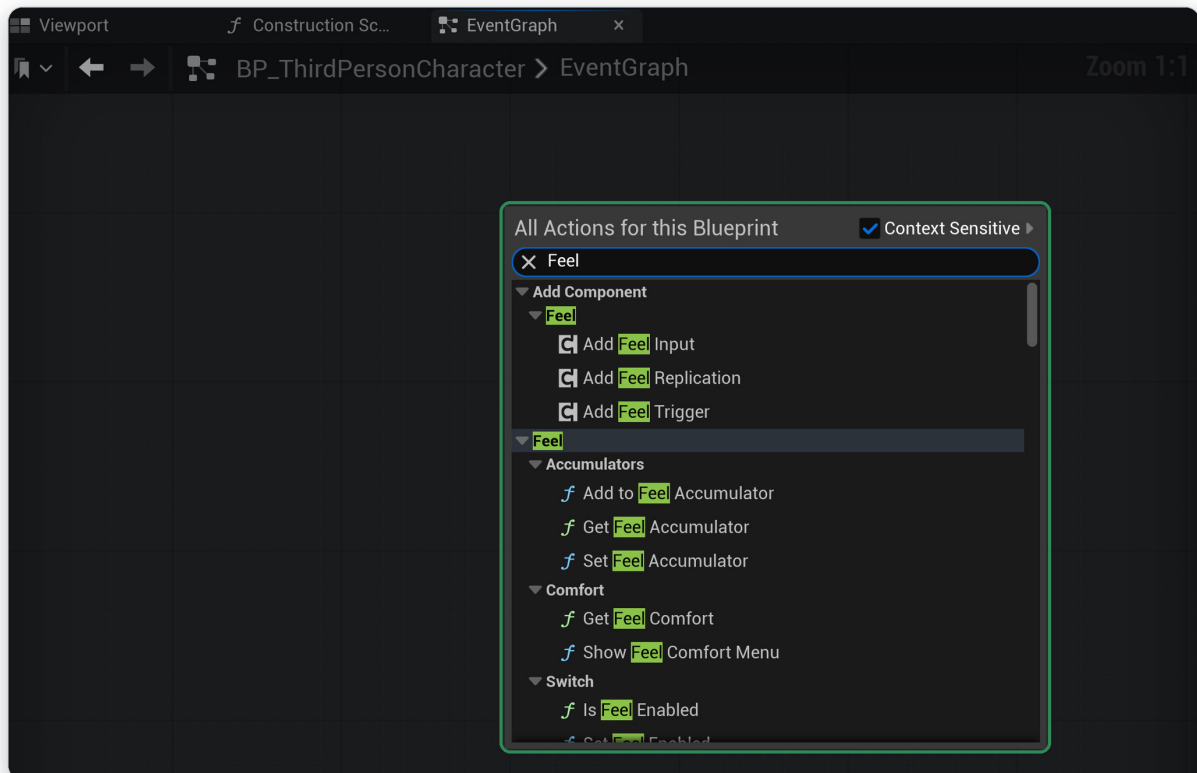


Figure 17.1 The Blueprint context menu searched for Feel

17.1.1 Play Feel

Plays a recipe.

The returned handle is invalid when the recipe could not start: no recipe was given, a **Cooldown** or **Max Concurrent** limit of the recipe is reached on this target, `feel.Enabled` is 0, the game runs as a dedicated server, or the target is an actor or component that no longer exists (this last case also logs a warning).

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel.	C++	<code>UFeelBlueprintLibrary::PlayFeel</code>

Pin	In or out	Type	Default	Description
Recipe	In	Feel Recipe		The recipe to play.
Target	In	Feel Target		What the recipe plays on. Camera and screen effects go to the target's local player.
Intensity	In	Float	1	Multiplier for every track of this play.
Return Value	Out	Feel Handle		Handle to stop or query this play. Invalid if the recipe could not start (cooldown, max concurrent, or <code>feel.Enabled</code> 0).

17.1.2 Play Feel with Context PRO

Plays a recipe like Play Feel, with extra information for this play. Use it when the recipe declares parameters, has tracks that apply to a second actor, or uses context tags.

- Parameters: values for the recipe's parameters; tracks with parameter mappings scale their intensity by them.
- Instigator: a second actor; tracks set to Applies To = Instigator play their actor effects on it.
- Direction, Location, Normal, Context Tags: passed to steps and Blueprint events, and used by Feel Maps.

With an empty context this behaves exactly like Play Feel.

Edition	Pro only	Node	Callable node with execution pins.
Category	Feel.		
C++	UFeelBlueprintLibrary::PlayFeelWithContext		

Pin	In or out	Type	Default	Description
Recipe	In	Feel Recipe		The recipe to play.
Target	In	Feel Target		What the recipe plays on.
Context	In	Feel Play Context		Parameter values, second actor and details for this play. Every field is optional. Can be left unconnected.
Intensity	In	Float	1	Multiplier for every track of this play.
Return Value	Out	Feel Handle		Handle to stop or query this play. Invalid if the recipe could not start.

17.1.3 Play Feel and Wait PRO

Plays a recipe and waits for it to end. On Finished fires when the recipe plays to its end. On Cancelled fires when it is stopped early, its target is destroyed, or it could not start (cooldown, max concurrent, feel.Enabled 0).

Edition	Pro only	Category	Feel.
Node	Latent node with an output execution pin for each outcome.		
C++	UFeelPlayAndWaitAction::PlayFeelAndWait		

Pin	In or out	Type	Default	Description
Recipe	In	Feel Recipe		The recipe to play.
Target	In	Feel Target		What the recipe plays on.

Pin	In or out	Type	Default	Description
Context	In	Feel Play Context		Optional parameter values, second actor and details for this play. Can be left unconnected.
Intensity	In	Float	1	Multiplier for every track of this play.
Return Value	Out	Feel Play and Wait Action		

17.1.4 Stop Feel

Stops one playing recipe.

With **Blend Out** on, the recipe fades out over **Blend Out Time** from the project settings (see 18.3 [Playback](#)) and still counts as playing until the fade ends.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel.	C++	<code>UFeelBlueprintLibrary::StopFeel</code>

Pin	In or out	Type	Default	Description
Handle	In	Feel Handle		Handle returned by Play Feel.
Blend Out	In	Boolean	On	Fade out over the project's blend-out time instead of stopping at once.

17.1.5 Stop All Feel

Stops playing recipes at once.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::StopAllFeel</code>		

Pin	In or out	Type	Default	Description
Target	In	Actor	None	Only stop recipes playing on this actor. Leave empty to stop all.

17.1.6 Release Feel PRO

Ends the sustain loop of a playing recipe so it plays its remaining tracks and finishes. Recipes without a sustain region are not affected.

Edition	Pro only	Node	Callable node with execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::ReleaseFeel</code>		

Pin	In or out	Type	Description
Handle	In	Feel Handle	Handle returned when the recipe started.

17.1.7 Is Feel Playing

Whether a recipe is still playing, including while it blends out.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::IsFeelPlaying</code>		

Pin	In or out	Type	Description
Handle	In	Feel Handle	Handle returned by Play Feel.
Return Value	Out	Boolean	True while playing.

17.1.8 Is Valid (Feel Handle)

Whether a handle came from a successful Play Feel. The recipe may have finished since.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::IsValidFeelHandle</code>		

Pin	In or out	Type	Description
Handle	In	Feel Handle	Handle to check.
Return Value	Out	Boolean	True for handles of recipes that started.

17.1.9 Get Duration

Nominal length of the recipe: the latest track end time, without random duration variation.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Target	A Feel Recipe.	Category	Feel.
C++	<code>UFeelRecipe::GetDuration</code>		

Pin	In or out	Type	Description
Return Value	Out	Float	Length in seconds.

17.2 Targets

A Feel Target left at its default (type None) sends camera and screen effects to the first local player. A target that belongs to a player on another machine gets no camera, screen or controller effects on this machine; that player's own machine shows them.

17.2.1 Make Feel Target from Actor

Target an actor. Actor effects such as Scale Punch use its root component; when the root takes part in collision (a character's capsule), they use its visible child components instead, so collision and movement stay unchanged.

Use this node for characters: scale, wobble and other actor motion then go to the character's mesh and never move its capsule.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::MakeFeelTargetFromActor</code>		

Pin	In or out	Type	Description
Actor	In	Actor	The actor to play on.
Return Value	Out	Feel Target	The target.

17.2.2 Make Feel Target from Component

Target a scene component, such as a character's mesh.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::MakeFeelTargetFromComponent</code>		

Pin	In or out	Type	Description
Component	In	Scene Component	The component to play on.
Return Value	Out	Feel Target	The target.

17.2.3 Make Feel Target at Location

Target a world location. Camera and screen effects go to the first local player.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel.		
C++	UFeelBlueprintLibrary::MakeFeelTargetAtLocation		

Pin	In or out	Type	Description
Location	In	Vector	The location to play at.
Return Value	Out	Feel Target	The target.

17.2.4 Make Feel Target from Local Player Camera

Target a local player's camera.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel.		
C++	UFeelBlueprintLibrary::MakeFeelTargetFromLocalPlayerCamera		

Pin	In or out	Type	Default	Description
Player Index	In	Integer	0	Index of the local player.
Return Value	Out	Feel Target		The target.

17.2.5 Make Feel Target from Widget

Target a UMG widget. Widget steps (punch, shake, flash) apply to it; camera, screen and haptic effects go to its owning player.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel.		
C++	UFeelBlueprintLibrary::MakeFeelTargetFromWidget		

Pin	In or out	Type	Description
Widget	In	Widget	The widget to play on.
Return Value	Out	Feel Target	The target.

17.2.6 Feel Target structure

What a recipe plays on.

Property	Default	Description
Type	None	Kind of target.
Actor	None	Target actor when Type is Actor. Shown only when Type is Actor .
Component	None	Target component when Type is Scene Component. Shown only when Type is Scene Component .
Location	(0, 0, 0)	World location when Type is World Location. Shown only when Type is World Location .
Local Player Index	0	Index of the local player when Type is Local Player Camera. At least 0. Shown only when Type is Local Player Camera .
Widget	None	Target widget when Type is Widget. Shown only when Type is Widget .

Values of **Type**:

- **None**: No target: camera and screen effects go to the first local player.
- **Actor**: An actor; actor effects use its root component, or its visible child components when the root takes part in collision (such as a character's capsule).
- **Scene Component**: A specific scene component.
- **World Location**: A point in the world.
- **Local Player Camera**: The camera of a local player.
- **Widget**: A UMG widget. Widget steps apply to it; camera and screen effects go to its owning player.

17.3 Play context

In Blueprint, build the structure with **Make FeelPlayContext** and connect it to the **Context** pin of Play Feel with Context, Send Feel Event, Play Feel and Wait or the networked nodes. Every field is optional.

17.3.1 Feel Play Context structure PRO

Extra information passed with a play. Everything is optional; an empty context plays the recipe exactly like Play Feel. It carries values for the recipe's parameters, an optional second actor that tracks can apply to, and details about what triggered the play that steps and Blueprint events can read.

Property	Default	Description
Parameters	empty	Values for the recipe's parameters, by parameter name. Parameters not listed use their default value.

Property	Default	Description
Instigator	None	A second actor involved in this play, besides the target: whatever caused or started it. Tracks whose Applies To is Instigator play their actor effects on this actor instead of the target. Leave empty when only the target matters.
Direction	(0, 0, 0)	A world-space direction associated with the play, if any. Zero means none.
Location	(0, 0, 0)	A world-space location associated with the play, when it differs from the target's location. Zero means none.
Normal	(0, 0, 0)	A world-space surface normal associated with the play, if any. Zero means none.
Context Tags	empty	Gameplay tags describing the circumstances of the play. Feel Maps use them to choose between recipe variants.

17.4 Parameters and accumulators

Set Feel Parameter changes a value of one play. The accumulator nodes change named values that any play can read through a recipe parameter's **Accumulator** field; accumulators are defined in the project settings (see [18.3 Playback](#)). The name pins of the accumulator nodes are dropdowns that list the accumulators of the project.

17.4.1 Set Feel Parameter PRO

Changes a parameter of a recipe that is playing. Tracks mapped to the parameter follow the new value from the next frame. Useful for sustained recipes whose strength follows a changing value.

Edition	Pro only	Node	Callable node with execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::SetFeelParameter</code>		

Pin	In or out	Type	Description
Handle	In	Feel Handle	Handle returned when the recipe started.
Parameter Name	In	Name	Name of a parameter the recipe declares. The dropdown lists the parameters of the recipe played by the node the Handle comes from, or every recipe parameter in the project.
Value	In	Float	New value. Clamped to the parameter's range.
Return Value	Out	Boolean	False when the recipe is no longer playing.

17.4.2 Add to Feel Accumulator PRO

Adds to an accumulator (Project Settings > Plugins > FeelKit > Accumulators). The value is clamped to the accumulator's max and decays over time. Recipe parameters that read the accumulator follow it.

Edition	Pro only	Node	Callable node with execution pins.
Category	Feel > Accumulators.		
C++	<code>UFeelBlueprintLibrary::AddToFeelAccumulator</code>		

Pin	In or out	Type	Default	Description
Accumulator Name	In	Name		Name of the accumulator.
Amount	In	Float		Amount to add. Negative amounts subtract.
Actor	In	Actor	None	Actor the value belongs to. Leave empty for the global value.

17.4.3 Set Feel Accumulator PRO

Sets an accumulator to a value, clamped to its max.

Edition	Pro only	Node	Callable node with execution pins.
Category	Feel > Accumulators.		
C++	<code>UFeelBlueprintLibrary::SetFeelAccumulator</code>		

Pin	In or out	Type	Default	Description
Accumulator Name	In	Name		Name of the accumulator.
Value	In	Float		New value.
Actor	In	Actor	None	Actor the value belongs to. Leave empty for the global value.

17.4.4 Get Feel Accumulator PRO

Current value of an accumulator.

Edition	Pro only	Node	Pure node, no execution pins.
Category	Feel > Accumulators.		
C++	<code>UFeelBlueprintLibrary::GetFeelAccumulator</code>		

Pin	In or out	Type	Default	Description
Accumulator Name	In	Name		Name of the accumulator.
Actor	In	Actor	None	Actor the value belongs to. Leave empty for the global value.
Return Value	Out	Float		The value, or 0 when it was never set or has decayed.

17.5 Events and Feel Maps

17.5.1 Send Feel Event PRO

Sends a named event and plays the recipe a Feel Map assigns to it. The map row is chosen by the event tag and the context tags: the most specific match wins. Maps on a Feel Trigger component of the target actor are checked before the project's maps (Project Settings > Plugins > FeelKit > Feel Maps). Use it to keep gameplay code free of recipe choices: gameplay sends what happened, the maps decide how it feels.

Edition	Pro only	Node	Callable node with execution pins.
Category	Feel.		
C++	<code>UFeelBlueprintLibrary::SendFeelEvent</code>		

Pin	In or out	Type	Default	Description
Event	In	Gameplay Tag		Event tag to look up.
Target	In	Feel Target		What the recipe plays on.
Context	In	Feel Play Context		Optional parameter values, second actor, details and context tags. Can be left unconnected.
Intensity	In	Float	1	Multiplier for every track of this play, multiplied by the row's intensity scale.
Return Value	Out	Feel Handle		Handle of the play. Invalid when no row matches or the recipe could not start.

17.5.2 Feel Map asset PRO

Decides which recipe plays for a named event. Gameplay sends an event tag with a play context; the map picks the row that matches most specifically, so variants for different circumstances need no branching in Blueprint. Matching: the row's event must match the sent event (same tag or a parent), and the context must contain all of the row's required tags. Among matches, a more specific event wins, then more required tags, then higher priority, then the earlier row.

Property	Default	Description
Entries	empty	Rows of this map.

Each entry of **Entries**:

Field	Default	Description
Event	None	Event this row answers. Also answers child events: a row for A.B answers A.B.C when no row is more specific.
Required Tags	empty	Tags the play context must all contain for this row to apply. Empty applies in every circumstance.
Recipe	None	Recipe to play.
Intensity Scale	1	Multiplies the intensity of the event when this row plays. At least 0.
Priority	0	Breaks ties between rows that match equally specifically. Higher wins.

17.6 Comfort

Comfort settings belong to a local player. **Get Feel Comfort** returns the player's Feel Comfort Subsystem, and the other comfort nodes are called on it. The settings are loaded when the local player is created, before any recipe plays, and saved automatically after each change while **Auto Save Comfort** is on (see [18.7 Comfort storage](#)).

17.6.1 Get Feel Comfort

Comfort settings of a local player: read and change scales, apply presets, save.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel > Comfort.		
C++	UFeelBlueprintLibrary::GetFeelComfort		

Pin	In or out	Type	Description
Player Controller	In	Player Controller	A local player controller.
Return Value	Out	Feel Comfort Subsystem	The player's comfort subsystem, or none for controllers without a local player.

17.6.2 Get Comfort Scales

Current comfort scales of this player.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::GetComfortScales</code>		

Pin	In or out	Type	Description
Return Value	Out	Feel Comfort Scales	All scales, each from 0 to 1.

17.6.3 Set Comfort Scales

Replaces all comfort scales. Values are clamped to 0 to 1 and saved when auto save is on.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::SetComfortScales</code>		

Pin	In or out	Type	Description
New Scales	In	Feel Comfort Scales	The new scales.

17.6.4 Set Master Comfort Scale

Sets the master scale that applies to every effect.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::SetMasterComfortScale</code>		

Pin	In or out	Type	Description
Scale	In	Float	0 turns off all non-essential effects, 1 is full strength.

17.6.5 Set Comfort Group Scale

Sets the scale of one comfort group.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::SetComfortGroupScale</code>		

Pin	In or out	Type	Description
Group	In	Feel Comfort Group	The group to change.
Scale	In	Float	0 turns off the group's non-essential effects, 1 is full strength.

Values of **Group**:

- **None**: Not part of a group; only Master applies.
- **Camera Shake**: Camera shakes.
- **Camera Motion**: Camera punches, FOV kicks, roll and zoom.
- **Flashes**: Full-screen flashes.
- **Hitstop and Slow-mo**: Hitstops and slow motion.
- **Screen Distortion**: Vignette, chromatic aberration and other distortion.
- **Haptics**: Controller vibration.

17.6.6 Apply Comfort Preset

Replaces the scales with a built-in preset whose values come from the project settings.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::ApplyComfortPreset</code>		

Pin	In or out	Type	Description
Preset	In	Feel Built in Comfort Preset	The preset to apply.

Values of **Preset**:

- **Default**: The project's default scales.
- **Reduced Motion**: Less camera shake, camera motion and distortion.
- **Reduced Flashing**: Weaker flashes.
- **No Haptics**: No controller vibration.

17.6.7 Apply Custom Comfort Preset

Replaces the scales with a custom preset asset.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::ApplyCustomComfortPreset</code>		

Pin	In or out	Type	Description
Preset	In	Feel Comfort Preset	The preset asset to apply.

17.6.8 Save Comfort Settings

Saves the scales through the comfort storage.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::SaveComfortSettings</code>		

Pin	In or out	Type	Description
Return Value	Out	Boolean	True if the storage reported success.

17.6.9 Load Comfort Settings

Loads the scales from the comfort storage. The current scales stay when nothing is stored.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::LoadComfortSettings</code>		

Pin	In or out	Type	Description
Return Value	Out	Boolean	True if stored settings were found.

17.6.10 Set Comfort Storage

Routes loading and saving to your own save system, then loads from it.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::SetComfortStorage</code>		

Pin	In or out	Type	Description
New Storage	In	Feel Comfort Storage (interface)	An object implementing Feel Comfort Storage. Leave empty to use the project's storage.

17.6.11 Get Effective Force Feedback Scale

How much of the game's controller vibration reaches the player right now: the player's Haptics and Master comfort combined, or 1 when Apply Comfort To Engine Force Feedback is off in the project settings. 0 means the controller stays still, whatever plays.

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel > Comfort.		
Target	Feel Comfort Subsystem, from Get Feel Comfort.		
C++	<code>UFeelComfortSubsystem::GetEffectiveForceFeedbackScale</code>		

Pin	In or out	Type	Description
Return Value	Out	Float	Scale from 0 to 1 applied to the game's controller vibration.

17.6.12 Feel Comfort Preset asset

A custom comfort preset players can apply.

Create one in the Content Browser as a Data Asset of the class Feel Comfort Preset, then apply it with **Apply Custom Comfort Preset**.

Property	Default	Description
Display Name	empty	Name shown to players, for example in a settings menu.
Scales	every scale 1	Scales this preset applies.

17.6.13 Load Comfort Scales

Loads a player's comfort scales.

To keep comfort settings in your own save system, make a class that implements the Feel Comfort Storage interface and implement this function and Save Comfort Scales. Then pick the class in **Comfort Storage Class** in the project settings, or pass an object of it to **Set Comfort Storage**.

Edition	Lite and Pro	Target	Your storage class.
Category	Feel > Comfort.		
Node	Function of the Feel Comfort Storage interface; you implement it in your own class.		
C++	<code>IFeelComfortStorage::LoadComfortScales</code>		

Pin	In or out	Type	Description
Local Player	In	Local Player	The player whose settings to load. Can be empty outside a game.
Out Scales	Out	Feel Comfort Scales	Receives the stored scales.
Return Value	Out	Boolean	False when nothing is stored.

17.6.14 Save Comfort Scales

Stores a player's comfort scales.

Edition	Lite and Pro	Target	Your storage class.
Category	Feel > Comfort.		
Node	Function of the Feel Comfort Storage interface; you implement it in your own class.		
C++	<code>IFeelComfortStorage::SaveComfortScales</code>		

Pin	In or out	Type	Description
Local Player	In	Local Player	The player whose settings to store. Can be empty outside a game.
Scales	In	Feel Comfort Scales	The scales to store.
Return Value	Out	Boolean	False when storing failed.

17.7 Feel Switch

Set Feel Enabled, **Toggle Feel** and **Is Feel Enabled** work with or without a Feel Switch actor in the level. They change and read the same value as the console variable `feel.Enabled` (see 19.1 `feel.Enabled`).

17.7.1 Set Feel Enabled

Turns all FeelKit feedback off or on, for comparing a game with and without it. Off stops every playing recipe at once (slowed time returns to normal) and nothing new plays until it is turned on

again. Only FeelKit is affected: the game's own camera shakes, sounds and effects keep working. Same switch as the console variable `feel.Enabled`; applies to the whole game on this machine and is not saved.

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Switch.		
C++	<code>UFeelBlueprintLibrary::SetFeelEnabled</code>		

Pin	In or out	Type	Description
Enabled	In	Boolean	True to turn FeelKit on, false to turn it off.

17.7.2 Toggle Feel

Turns FeelKit off when it is on, and on when it is off (see Set Feel Enabled).

Edition	Lite and Pro	Node	Callable node with execution pins.
Category	Feel > Switch.		
C++	<code>UFeelBlueprintLibrary::ToggleFeel</code>		

Pin	In or out	Type	Description
Return Value	Out	Boolean	True when FeelKit is on after the switch.

17.7.3 Is Feel Enabled

Whether FeelKit feedback is on (see Set Feel Enabled).

Edition	Lite and Pro	Node	Pure node, no execution pins.
Category	Feel > Switch.		
C++	<code>UFeelBlueprintLibrary::IsFeelEnabled</code>		

Pin	In or out	Type	Description
Return Value	Out	Boolean	True when recipes play.

17.7.4 Feel Switch actor

Lets players turn FeelKit off and on while playing, to see and feel the difference. Place one in a level: it shows a start card that explains the switch, keeps a small ON/OFF badge in a corner of the screen, and switches when a player presses one of the Switch Keys (keyboard or controller). No input assets or Blueprint changes are needed: the keys are read from each local player's own key presses, next to

whatever input setup the game uses. Off stops only FeelKit (see Set Feel Enabled); the game's own effects keep working. The Comfort Menu Keys open the player's comfort menu, so players can also try the comfort settings while playing.

Place one Feel Switch in a level. A second one in the same level logs a warning and does nothing. The switch applies to the whole game on this machine, so every Play In Editor window switches together, and the choice is not saved.

Property	Default	Description
Feel Switch		
Switch Keys	Tab, Gamepad Special Left	Keys and controller buttons that turn the feel off and on. The first keyboard key and the first controller button are named on the start card and the badge.
Start Enabled	On	Whether FeelKit is on when the level starts. The choice is not saved between plays.
Feel Switch > Display		
Show Built in Display	On	Show the start card and the corner badge. Turn off to build your own on-screen display with On Feel Switched.
Show Start Card	On	Show a card when the level starts that explains how to switch. It fades after Start Card Seconds or on any key or button. Editable only when Show Built in Display is on.
Start Card Title	"FeelKit"	Heading of the start card. Editable only when Show Built in Display is on and Show Start Card is on.
Start Card Text	"Everything you see, hear and feel on top of the game comes from FeelKit."	One or two lines under the heading, for example what this level shows. Leave empty for none. Editable only when Show Built in Display is on and Show Start Card is on.
Start Card Seconds	8 s	Seconds (real time) the start card stays before it fades, unless a key or button is pressed first. At least 1. Editable only when Show Built in Display is on and Show Start Card is on.
Controls	empty	The level's controls, shown in a small panel under the badge for as long as the level runs. Leave empty for no panel. Editable only when Show Built in Display is on.
Feel Switch > Comfort Menu		
Comfort Menu Keys	O, Gamepad Special Right	Keys and controller buttons that open the player's comfort menu (Show Feel Comfort Menu) and close it again. They are added to the controls panel. Leave empty for none.
Comfort Menu Class	None	Menu to open. Empty uses Comfort Menu Class from Project Settings > Plugins > FeelKit.
Pause in Comfort Menu	On	Pause the game while the comfort menu is open. The menu's previews still play.
Comfort Menu Control Name	"Comfort settings"	Name of the comfort menu row in the controls panel.

Each entry of **Controls**:

Field	Default	Description
Action	empty	What the input does, such as "Attack".
Keys	empty	The keys and buttons, such as "Left mouse / RB".

Event **On Feel Switched** with **Feel Enabled** (Boolean): Called whenever FeelKit is turned off or on, from a Switch Key, a Blueprint node or the console variable feel.Enabled.

Switch

Turns FeelKit off when it is on, and on when it is off. Same as pressing a Switch Key.

Edition	Lite and Pro	Node	Callable node with execution pins.
Target	Feel Switch actor.	Category	Feel Switch.
C++	<code>AFeelSwitch::Switch</code>		

17.8 Animation notifies

Add these to an animation's **Notifies** track (right-click the track, **Add Notify** or **Add Notify State**). They play when the animation reaches them in the game, and also in the animation editor's preview, so actor effects can be tuned while scrubbing the animation.

17.8.1 Play Feel notify PRO

Plays a recipe at this point of an animation. Also plays in the animation editor preview, so the recipe's actor effects can be tuned while scrubbing the animation.

Property	Default	Description
Recipe	None	Recipe to play.
Target	Mesh	What the recipe plays on.
Intensity	1	Intensity of the play. At least 0.
Parameters	empty	Parameter values passed with the play.
Context Tags	empty	Context tags passed with the play. Feel Maps use them to pick recipe variants.

17.8.2 Send Feel Event notify PRO

Sends a Feel Event at this point of an animation; Feel Maps decide which recipe plays.

Property	Default	Description
Event	None	Event to send.

Property	Default	Description
Target	Mesh	What the recipe plays on.
Intensity	1	Intensity of the play. At least 0.
Parameters	empty	Parameter values passed with the play.
Context Tags	empty	Context tags passed with the play. Feel Maps use them to pick recipe variants.

17.8.3 Set Feel Value notify PRO

Sets or adds to accumulator values at this point of an animation (Project Settings > Plugins > FeelKit > Accumulators). Lets an animation tell recipes what kind of move is happening, for example how heavy a swing is, without any Blueprint: a recipe parameter that reads the accumulator picks the value up when the move's recipe plays.

Property	Default	Description
Values	empty	Accumulators and the values they are set to.
Mode	Set	Whether the values replace or add to the accumulators.
Scope	Owning Actor	Whose values are set.

17.8.4 Play Feel (Window) notify state PRO

Plays a recipe for the length of a notify window. At the end of the window a sustained recipe is released (it plays its ending) and any other recipe is stopped, unless it is set to finish on its own.

Property	Default	Description
Recipe	None	Recipe to play. Recipes with a sustain region keep looping until the window ends.
Target	Mesh	What the recipe plays on.
Intensity	1	Intensity of the play. At least 0.
Parameters	empty	Parameter values passed with the play.
Context Tags	empty	Context tags passed with the play. Feel Maps use them to pick recipe variants.
Let Non Sustained Recipe Finish	Off	When the window ends on a recipe without sustain, let it finish on its own instead of stopping it.

17.9 Feel Trigger component

17.9.1 Feel Trigger PRO

Plays recipes or sends Feel Events when common events happen to its owner, with no Blueprint wiring. Also holds Feel Maps that Send Feel Event checks first when this actor is the target.

The component ticks only when one of its entries uses Jumped, Air Jumped or Launched, because those have no engine event to bind to. The other events are bound when play begins.

Property	Default	Description
Triggers	empty	What to play for each event. Several entries may use the same event.
Feel Maps	empty	Feel Maps checked before the project's maps when a Feel Event targets this actor.

Each entry of **Triggers**:

Field	Default	Description
Event	Take Any Damage	Actor event that fires this entry.
Recipe	None	Recipe to play. When empty, Feel Event is sent instead.
Feel Event	None	Event sent through the Feel Maps when no recipe is set, so maps choose the recipe. Editable only when Recipe is empty.
Value Parameter	None	Recipe parameter that receives the event value (damage amount, landing speed or hit impulse). Map that parameter on tracks to scale them by it. Leave empty to ignore the value.
Intensity	1	Intensity of the play. At least 0.
Target Skeletal Mesh	On	Play on the owner's first skeletal mesh component instead of the owner actor, so actor effects such as scale apply to the visible mesh of characters rather than their collision capsule.
Other Actor as Instigator	On	Pass the other actor of the event (damage causer, hit or overlapping actor) as the play's instigator. When the entry plays on the other actor, the owner becomes the instigator instead.
Play On	Owner	Where the recipe plays: on the owner, or on the other actor of the event.
Only for Players	Off	Fire only when the other actor of the event is a pawn controlled by a player (or that player's controller), so enemies, props or projectiles do not set it off.
Context Tags	empty	Context tags added to every play of this entry.

Fire Event PRO

Fires every entry for an event, as if it happened. Useful for events the component does not bind itself.

Edition	Pro only	Node	Callable node with execution pins.
Target	Feel Trigger component.	Category	Feel.
C++	<code>UFeelTriggerComponent::FireEvent</code>		

Pin	In or out	Type	Description
Event	In	Feel Trigger Event	Which entries to fire: those set to this event.
Event Value	In	Float	The event's value, passed to each entry's Value Parameter (for example a fall speed or damage).
Other Actor	In	Actor	The other actor of the event, used by Other Actor As Instigator and Play On Other Actor. Can be empty.

Values of **Event**, with the event value each one passes:

- **Take Any Damage:** The owner receives damage (any damage type). Event value: the damage amount.
- **Landed:** The owner, a Character, lands after falling. Event value: the downward speed at landing, in cm/s.
- **Component Hit:** The owner's root component reports a blocking hit. Event value: the length of the normal impulse.
- **Begin Overlap:** Another actor starts overlapping the owner. Event value: 0.
- **End Overlap:** Another actor stops overlapping the owner. Event value: 0.
- **Jumped:** The owner, a Character, jumps from the ground (including a late jump just after walking off a ledge). Event value: the jump number, 1.
- **Air Jumped:** The owner, a Character, jumps again while already in the air (double or triple jump). Event value: the jump number, 2 or more.
- **Launched:** The owner, a Character, is launched: thrown by gameplay rather than by its own movement, such as a wall jump, a jump pad or a knockback (Launch Character). Event value: the launch speed in cm/s. The play's Direction is the launch direction.

17.10 Feel Input component

17.10.1 Feel Input PRO

Plays FeelKit recipes from Enhanced Input actions without Blueprint wiring. Add it to a pawn or a player controller and list the actions to respond to. Plays target the owning actor. Bindings attach to

the owner's input component once it exists and are attached again when the pawn is possessed by a new controller.

The input actions must be in an input mapping context that the player has added.

Property	Default	Description
Bindings	empty	Actions this component responds to.

Each entry of **Bindings**:

Field	Default	Description
Action	None	The input action to listen to. It must be in an input mapping context the player has added.
Play On	Started	When to play: Started (the input begins), Triggered (each time the action's triggers fire; every frame while held for a plain press), Completed or Canceled.
Recipe	None	Recipe to play. When empty, Event is sent instead.
Event	None	Feel Event to send when no recipe is set.
Intensity	1	Intensity of the play. At least 0.
Scale Intensity by Value	Off	Multiplies the intensity by the action value's magnitude, clamped to 0 to 1 (for example how far a trigger is pressed).
Value Parameter	None	Recipe parameter that receives the action value's magnitude. Empty sends none.
End when Input Ends	Off	Ends the play when the input ends (Completed or Canceled), for feedback that lasts while the input is held, such as a charge-up. A sustained recipe is released and plays its ending; other recipes stop with a blend out.
Once Per Press	Off	Plays at most once until the input ends, even when Play On fires repeatedly (such as Triggered while held).

Handle Input PRO

Responds to one binding as if its action fired. Used by the input bindings; callable to simulate input.

Edition	Pro only	Node	Callable node with execution pins.
Target	Feel Input component.	Category	Feel > Input.
C++	<code>UFeelInputComponent::HandleInput</code>		

Pin	In or out	Type	Default	Description
Binding Index	In	Integer		Index of the binding in Bindings, starting at 0.
Trigger Event	In	Trigger Event		The input event to respond to, such as Started or Completed. Only bindings set to play on it respond.

Pin	In or out	Type	Default	Description
Action Magnitude	In	Float	1	Strength of the input, 0 to 1 for most actions. Scales the intensity when the binding uses Scale Intensity By Value.

17.11 Feel Replication component

17.11.1 Feel Replication PRO

Plays recipes and sends Feel Events on several machines in a networked game. Add it to a replicated actor, such as a character or a weapon. Called on the server, the request goes to the machines the mode selects. Called on the client that owns the actor, the client plays at once (no wait for the server) and the server forwards the play to everyone else. Called on any other client, the play stays local. Only the request is sent; each machine evaluates the recipe with its own comfort settings. Plays are cosmetic and unreliable: under heavy packet loss a remote machine may miss one.

Add it to a replicated actor, such as a character or a weapon. The component is replicated by default.

Play Feel Networked PRO

Plays a recipe on the machines chosen by Mode.

Edition	Pro only	Node	Callable node with execution pins.
Target	Feel Replication component.	Category	Feel > Network.
C++	<code>UFeelReplicationComponent::PlayFeelNetworked</code>		

Pin	In or out	Type	Default	Description
Recipe	In	Feel Recipe		Recipe to play.
Target	In	Feel Target		What it plays on. Actors and components must exist on every machine (replicated or placed in the level).
Intensity	In	Float		Strength of the play, as in Play Feel.
Context	In	Feel Play Context		Parameters, instigator, direction, location, normal and tags of the play. Can be left unconnected.
Mode	In	Feel Net Mode	Everyone	Which machines play it.

Pin	In or out	Type	Default	Description
Relevancy Distance	In	Float	0	Machines whose local players are all farther than this from the target (measured from each player's character or camera, whichever is closer) skip the play. A player's own character is always in range. 0 means no limit. Shown under the node's advanced arrow.
Return Value	Out	Feel Handle		Handle of the play on this machine, invalid when this machine does not play it.

Send Feel Event Networked PRO

Sends a Feel Event on the machines chosen by Mode. Each machine picks the recipe from its Feel Maps.

Edition	Pro only	Node	Callable node with execution pins.
Target	Feel Replication component.	Category	Feel > Network.
C++	<code>UFeelReplicationComponent::SendFeelEventNetworked</code>		

Pin	In or out	Type	Default	Description
Event	In	Gameplay Tag		Event tag.
Target	In	Feel Target		What it plays on. Actors and components must exist on every machine.
Intensity	In	Float		Strength of the play.
Context	In	Feel Play Context		Parameters, instigator, direction, location, normal and tags of the play. Can be left unconnected.
Mode	In	Feel Net Mode	Everyone	Which machines play it.
Relevancy Distance	In	Float	0	Machines whose local players are all farther than this from the target (measured from each player's character or camera, whichever is closer) skip the play. A player's own character is always in range. 0 means no limit. Shown under the node's advanced arrow.
Return Value	Out	Feel Handle		Handle of the play on this machine, invalid when this machine does not play it or no map entry matched.

Values of **Mode**:

- **Everyone:** Every machine plays it.
- **Owner Only:** Only the machine that owns this component's actor plays it, for feedback meant for one player. The actor must be owned by a player (its pawn, controller or something they own).

- **Skip Owner:** Every machine except the owner's plays it, for feedback the owner already played locally. In a single-player game nothing plays.

17.12 Gameplay Cue notifies (GAS add-on)

These classes are in the GAS add-on, the separate plugin FeelKit GAS with the module FeelGAS. It is not active until you copy it into your project. To use one in Blueprint, create a Blueprint class with the class below as its parent and set its **Gameplay Cue Tag** as for any other gameplay cue notify.

17.12.1 Feel Gameplay Cue Notify PRO

Gameplay Cue notify that plays a FeelKit recipe or sends a Feel Event when the cue is executed or added. For one-shot feedback such as hits and pickups. For feedback that lasts while a gameplay effect is active, use Feel Gameplay Cue Notify (Actor). Set the Gameplay Cue Tag as for any static cue notify.

Property	Default	Description
Recipe	None	Recipe to play. When empty, Event is sent instead.
Event	None	Feel Event to send when no recipe is set, so Feel Maps choose the recipe. Empty uses the cue's own tag.
Scale Intensity by Magnitude	Off	Multiplies the play's intensity by the cue's normalized magnitude (0 to 1).
Intensity	1	Intensity of the play, before the magnitude scale. At least 0.
Raw Magnitude Parameter	None	Recipe parameter that receives the cue's raw magnitude. Empty sends none.
Normalized Magnitude Parameter	None	Recipe parameter that receives the cue's normalized magnitude. Empty sends none.
Effect Causer as Instigator	Off	Use the effect causer (such as a weapon or projectile) as the play's instigator instead of the cue's instigator.

17.12.2 Feel Gameplay Cue Notify (Actor) PRO

Gameplay Cue notify actor that plays a FeelKit recipe while its gameplay cue is active: it starts when the cue is added and ends when the cue is removed (see Stop On Remove). Use a recipe with a sustain region for feedback that loops for as long as the effect lasts.

Property	Default	Description
Recipe	None	Recipe to play. When empty, Event is sent instead.
Event	None	Feel Event to send when no recipe is set, so Feel Maps choose the recipe. Empty uses the cue's own tag.
Scale Intensity by Magnitude	Off	Multiplies the play's intensity by the cue's normalized magnitude (0 to 1).

Property	Default	Description
Intensity	1	Intensity of the play, before the magnitude scale. At least 0.
Raw Magnitude Parameter	None	Recipe parameter that receives the cue's raw magnitude. Empty sends none.
Normalized Magnitude Parameter	None	Recipe parameter that receives the cue's normalized magnitude. Empty sends none.
Effect Causer as Instigator	Off	Use the effect causer (such as a weapon or projectile) as the play's instigator instead of the cue's instigator.
Stop on Remove	Off	When the cue is removed: on, the play stops with a blend out; off, a sustained recipe is released and plays its ending, and any other recipe plays to its end.

17.13 Editor scripting

These functions are in the category **FeelKit > Editor Scripting**, for Editor Utility Blueprints and Python in the editor. They are not available in packaged games. In Python they are called as `unreal.FeelEditorScripting.import_recipe_from_json_file(...)` and so on.

17.13.1 Import Recipe from Json File PRO

Replaces a recipe's content with a recipe JSON file (the same as Import from JSON in the Content Browser) and saves it.

Edition	Pro only	Node	Callable node with execution pins.
Category	FeelKit > Editor Scripting.		
C++	<code>UFeelEditorScripting::ImportRecipeFromJsonFile</code>		

Pin	In or out	Type	Description
Recipe	In	Feel Recipe	Recipe to fill.
File Path	In	String	Path of the .json file.
Out Message	Out	String	What happened, including asset references that are not in this project.
Return Value	Out	Boolean	True when the file was imported.

17.13.2 Create Recipe from Json File PRO

Creates a recipe asset (or reuses the one already there) and fills it from a recipe JSON file, then saves it.

Edition	Pro only	Node	Callable node with execution pins.
Category	FeelKit > Editor Scripting.		
C++	<code>UFeelEditorScripting::CreateRecipeFromJsonFile</code>		

Pin	In or out	Type	Description
Package Path	In	String	Folder, for example /FeelKit/Demos/ActionRPG.
Asset Name	In	String	Name of the recipe asset.
File Path	In	String	Path of the .json file.
Out Message	Out	String	What happened.
Return Value	Out	Feel Recipe	The recipe, or none when it could not be created or imported.

17.13.3 Add Feel Map to Project Settings PRO

Adds a Feel Map to Project Settings > Plugins > FeelKit > Feel Maps (if it is not there yet) and saves the project settings.

Edition	Pro only	Node	Callable node with execution pins.
Category	FeelKit > Editor Scripting.		
C++	<code>UFeelEditorScripting::AddFeelMapToProjectSettings</code>		

Pin	In or out	Type	Description
Feel Map	In	Feel Map	The Feel Map asset to add.
Return Value	Out	Boolean	True when the map is in the list afterwards.

17.13.4 Set Accumulator in Project Settings PRO

Adds an accumulator to Project Settings > Plugins > FeelKit > Accumulators, or updates the one with the same name, and saves the project settings.

Edition	Pro only	Node	Callable node with execution pins.
Category	FeelKit > Editor Scripting.		
C++	<code>UFeelEditorScripting::SetAccumulatorInProjectSettings</code>		

Pin	In or out	Type	Description
Name	In	Name	Accumulator name, as used by Add To Feel Accumulator and recipe parameters.

Pin	In or out	Type	Description
Max Value	In	Float	Highest value it can reach.
Decay Per Second	In	Float	How much it loses per second once it starts decaying.
Decay Delay	In	Float	Seconds after the last add before it starts decaying.
Return Value	Out	Boolean	True when the accumulator is in the list afterwards.

17.14 C++ access

FeelKit's runtime module is FeelCore. Add it to the dependencies of your game module in its `.Build.cs` file; add FeelEnhancedInput to use the Feel Input component from C++, and FeelGAS for the GAS add-on.

```
PublicDependencyModuleNames.AddRange(new string[] { "Core", "CoreUObject", "Engine",
"FeelCore" });
```

Plays go through the world's Feel Subsystem:

```
#include "FeelSubsystem.h"

if (UFeelSubsystem* Feel = UFeelSubsystem::Get(this))
{
    HitHandle = Feel->PlayFeel(HitRecipe, FFeelTarget::FromActor(this));
}
```

Here `HitRecipe` is a `UFeelRecipe` property of the class and `HitHandle` an `FFeelHandle` member. `FFeelTarget` has the static functions `FromActor`, `FromComponent`, `AtLocation`, `FromLocalPlayerCamera` and `FromWidget`, which match the Make Feel Target nodes. The nodes in the **Feel** categories are static functions of `UFeelBlueprintLibrary` and can be called from C++ as well. The table lists the functions of `UFeelSubsystem` for C++ code.

Declaration	What it does	Edition
<code>static UFeelSubsystem* Get(const UObject* WorldContextObject)</code>	Subsystem of the world of <code>WorldContextObject</code> , or null.	Lite and Pro
<code>FFeelHandle PlayFeel(UFeelRecipe* Recipe, const FFeelTarget& Target, float Intensity = 1.0f, const FFeelPlayContext& Context = FFeelPlayContext())</code>	Starts a recipe. Returns an invalid handle when it cannot start (null recipe, cooldown, max concurrent, <code>feel.Enabled 0</code> , dedicated server). Context carries optional parameter values, a second actor and details for this play.	Lite and Pro
<code>FFeelHandle SendFeelEvent(const FGameplayTag& Event, const FFeelTarget& Target, float Intensity = 1.0f, const FFeelPlayContext& Context = FFeelPlayContext())</code>	Plays the recipe a Feel Map assigns to an event, chosen by the event tag and the context tags. Maps on a Feel Trigger component of the target actor are checked before the project's maps. Returns an invalid handle when no row matches.	Pro
<code>void StopFeel(FFeelHandle Handle, bool bBlendOut)</code>	Stops one instance, optionally fading it out over the project's blend-out time.	Lite and Pro

Declaration	What it does	Edition
<code>void ReleaseFeel(FFeelHandle Handle)</code>	Ends the sustain loop of a sustained recipe so it plays to the end. Recipes without sustain are unaffected.	Pro
<code>void StopAllFeel(const AActor* Target = nullptr)</code>	Stops every instance, or only those playing on Target when it is set.	Lite and Pro
<code>bool SetFeelParameter(FFeelHandle Handle, FName ParameterName, float Value)</code>	Changes a parameter of a playing recipe. Takes effect on the next frame. Returns false when the play has ended.	Pro
<code>bool IsPlaying(FFeelHandle Handle) const</code>	True while the instance of Handle is playing (including while blending out).	Lite and Pro
<code>int32 GetNumActiveInstances() const</code>	Number of instances that have not finished.	Lite and Pro
<code>void AddToAccumulator(FName AccumulatorName, float Amount, const AActor* Actor = nullptr)</code>	Adds to an accumulator for an actor, or the global value when Actor is null. Clamped to the accumulator's max.	Pro
<code>void SetAccumulator(FName AccumulatorName, float Value, const AActor* Actor = nullptr)</code>	Sets an accumulator for an actor, or the global value when Actor is null.	Pro
<code>float GetAccumulator(FName AccumulatorName, const AActor* Actor = nullptr) const</code>	Current accumulator value for an actor, or the global value when Actor is null. 0 when never set.	Pro
<code>FOOnFeelStarted OnFeelStarted (FFeelHandle Handle, UFeelRecipe* Recipe)</code>	Broadcast when a recipe starts playing.	Pro
<code>FOOnFeelFinished OnFeelFinished (FFeelHandle Handle, UFeelRecipe* Recipe, bool bInterrupted)</code>	Broadcast when a recipe stops: bInterrupted is false when it played to the end.	Pro

18 Project settings and editor preferences

IN THIS CHAPTER

- 18.1 Where the settings are kept
- 18.2 Camera
- 18.3 Playback
- 18.4 Events
- 18.5 Comfort
- 18.6 Comfort scale fields and preset values
- 18.7 Comfort storage
- 18.8 Editor preferences

18.1 Where the settings are kept

Project settings are under **Edit > Project Settings > Plugins > FeelKit**. They are saved in the project's `Config/DefaultGame.ini`, in the section `[/Script/FeelCore.FeelSettings]`, so they are shared by everyone who works on the project and are packaged with the game.

Editor preferences are under **Edit > Editor Preferences > Plugins > FeelKit**. They belong to one user of one project and are saved in `Saved/Config/WindowsEditor/EditorPerProjectUserSettings.ini` on Windows.

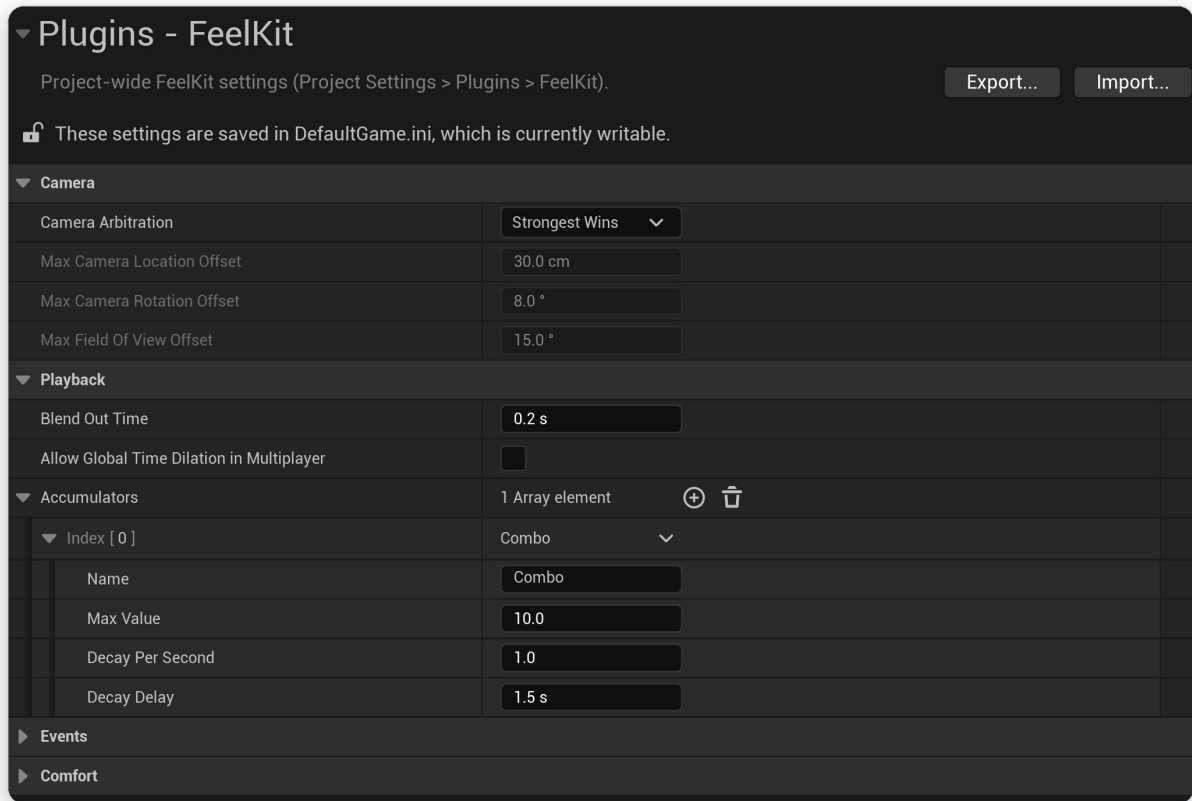


Figure 18.1 Project Settings, Plugins, FeelKit: the Camera and Playback categories, with one accumulator (Combo) added as an example

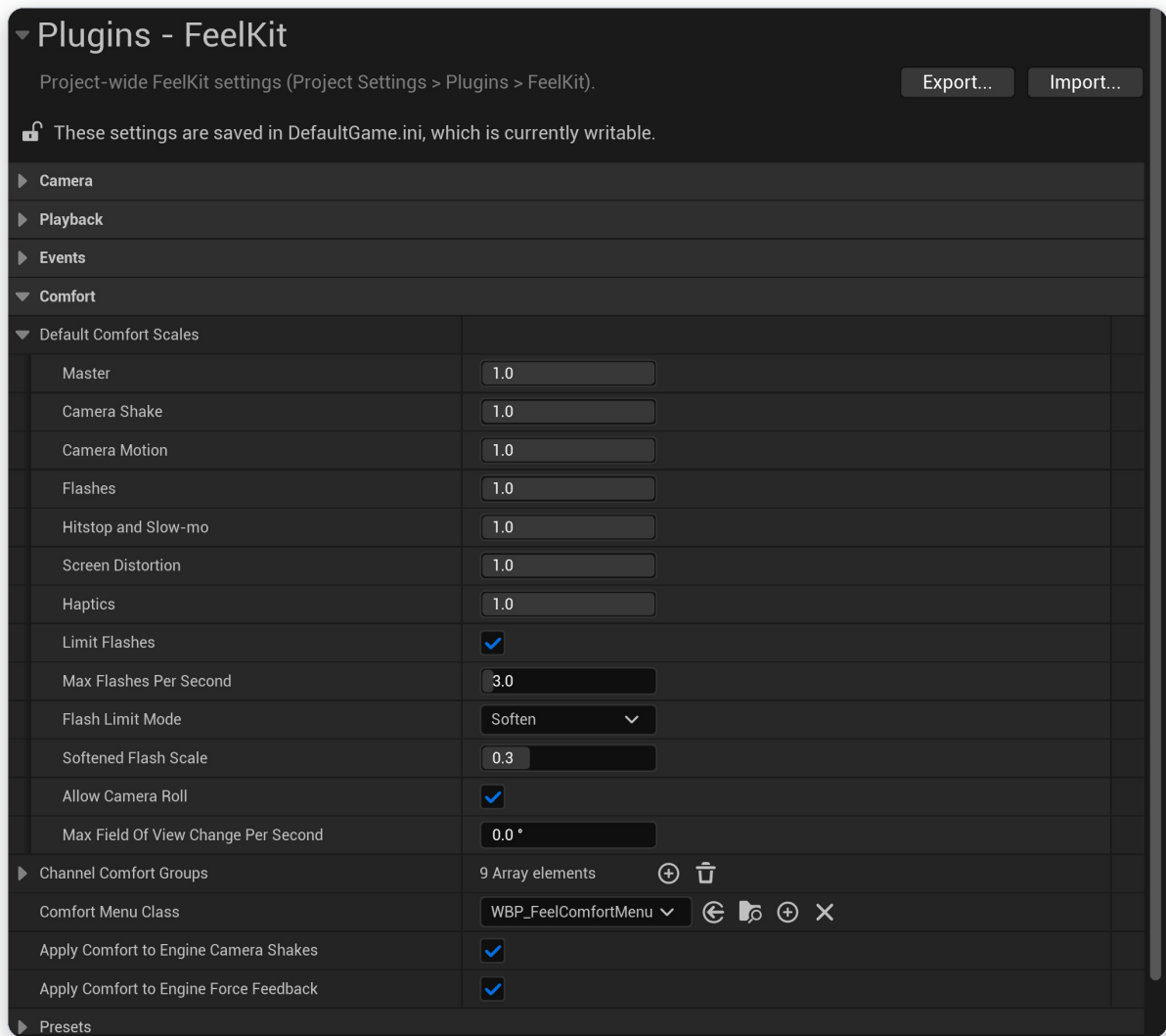


Figure 18.2 Project Settings, Plugins, FeelKit: the Comfort category with the default comfort scales

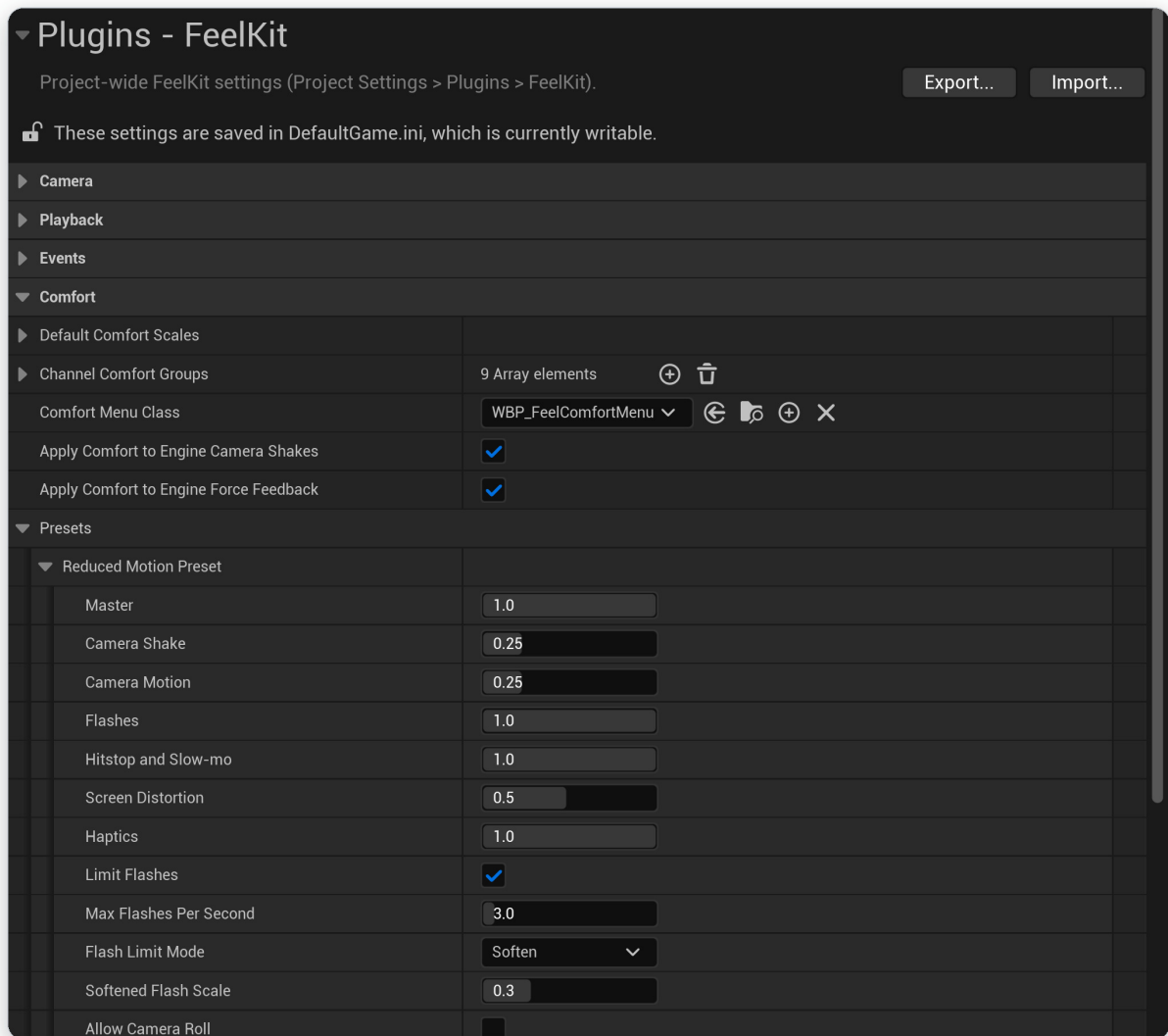


Figure 18.3 Project Settings, Plugins, FeelKit: the comfort presets, with the Reduced Motion preset expanded

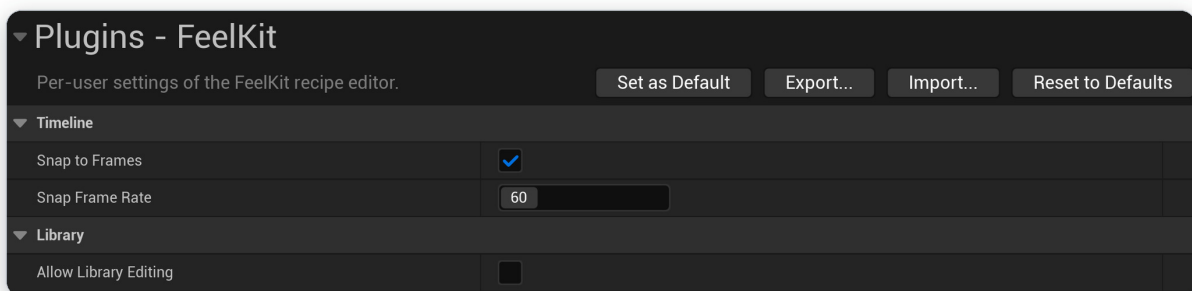


Figure 18.4 Editor Preferences, Plugins, FeelKit

18.2 Camera

These settings decide how the camera output of separate plays combines for one player. Tracks inside one play always add up. With **Strongest Wins**, the strongest location offset, rotation offset

and field of view change each win on their own. With **Additive Capped**, the plays add up and the sum is limited by the three caps.

Property	Default	Description
Camera Arbitration	Strongest Wins	How overlapping camera effects combine.
Max Camera Location Offset	30 cm	Largest combined camera location offset in additive mode. At least 0. Editable only when Camera Arbitration is Additive Capped .
Max Camera Rotation Offset	8°	Largest combined camera rotation offset per axis in additive mode. At least 0. Editable only when Camera Arbitration is Additive Capped .
Max Field Of View Offset	15°	Largest combined field of view change in additive mode. At least 0. Editable only when Camera Arbitration is Additive Capped .

18.3 Playback

Property	Default	Description
Blend Out Time	0.2 s	Fade-out length when a recipe is stopped with blend out. At least 0.
Allow Global Time Dilation in Multiplayer	Off	PRO In networked games (client, listen server), world time is shared by everyone, so by default a Global Hitstop or global Slow-mo slows only the actors of the play (its target) on this machine instead of the whole world. Turn on to change world time anyway: on a listen server that slows every connected player, and on a client the server's time can override it. Single-player games always use world time.
Accumulators	empty	PRO Named values that build up with repeated additions and decay over time. Recipe parameters can read them.

Each entry of **Accumulators**:

Field	Default	Description
Name	None	Name used to add to the value and to read it from recipe parameters.
Max Value	1	Highest value. Additions above it are clamped. At least 0.
Decay Per Second	1	How fast the value falls toward zero, in units per second. 0 keeps it until the game changes it. At least 0.
Decay Delay	0 s	Seconds after the last addition before the value starts to fall. At least 0.

18.4 Events PRO

Property	Default	Description
Feel Maps	empty	Feel Maps used by Send Feel Event for every target. A Feel Trigger component on the target can add maps that are checked first.

18.5 Comfort

Players start with **Default Comfort Scales** until they change their settings or saved settings are loaded. **Channel Comfort Groups** decides which group scales each channel; a mapping covers the channel and every tag below it, and when several mappings match, the most specific tag wins.

Property	Default	Description
Comfort		
Default Comfort Scales	every scale 1 (see the comfort scale tables)	Comfort scales new players start with. Also the Default preset.
Channel Comfort Groups	nine entries (see the table below)	Which comfort group scales each channel. Child tags are included; the most specific tag wins.
Comfort Menu Class	<code>FSoftClassPath(TEXT("/FeelKit/UI/WBP_FeelComfortMenu.WBP_FeelComfortMenu_C"))</code>	Menu opened by Show Feel Comfort Menu when the node is given no menu class. To restyle the menu, copy <code>WBP_FeelComfortMenu</code> into your project (Copy to Project) and select the copy here.
Apply Comfort to Engine Camera Shakes	On	Applies each player's Camera Shake comfort scale (and Master) to camera shakes the game plays through the engine, not only to FeelKit recipes, so one comfort setting reduces every shake.
Apply Comfort to Engine Force Feedback	On	Applies each player's Haptics comfort scale (and Master) to all controller vibration through the player controller's force feedback scale, not only to FeelKit recipes.
Comfort > Presets		
Reduced Motion Preset	see the comfort scale tables	Scales applied by the Reduced Motion preset.
Reduced Flashing Preset	see the comfort scale tables	Scales applied by the Reduced Flashing preset.
No Haptics Preset	see the comfort scale tables	Scales applied by the No Haptics preset.

Each entry of **Channel Comfort Groups**:

Field	Default	Description
Channel	None	Channel tag, for example <code>Feel.Camera.Shake</code> . Child tags are included.
Group	None	Comfort group that scales this channel.

Default **Channel Comfort Groups**:

Channel	Comfort group
<code>Feel.Camera.Shake</code>	Camera Shake
<code>Feel.Camera.Motion</code>	Camera Motion
<code>Feel.Screen.Flash</code>	Flashes
<code>Feel.Screen.Distortion</code>	Screen Distortion
<code>Feel.Screen.Color</code>	Screen Distortion
<code>Feel.Actor.Light</code>	Flashes
<code>Feel.Time.Hitstop</code>	Hitstop and Slow-mo
<code>Feel.Time.SlowMO</code>	Hitstop and Slow-mo
<code>Feel.Haptics</code>	Haptics

Channels with no entry are scaled by Master only: `Feel.Actor.Material`, `Feel.Actor.Transform`, `Feel.Audio`, `Feel.Audio.Mix`, `Feel.Meta.Event`, `Feel.Meta.Recipe`, `Feel.Screen.Fade`, `Feel.Spawn`, `Feel.UI`.

18.6 Comfort scale fields and preset values

Fields of a set of comfort scales (used by **Default Comfort Scales**, the three presets and custom Feel Comfort Preset assets):

Property	Default	Description
Comfort		
Master	1	Scales every effect. Range 0 to 1.
Camera Shake	1	Scales camera shakes. Range 0 to 1.
Camera Motion	1	Scales camera punches, FOV kicks, roll and zoom. Range 0 to 1.
Flashes	1	Scales full-screen flashes. Range 0 to 1.
Hitstop and Slow-mo	1	Scales hitstops and slow motion. Range 0 to 1.
Screen Distortion	1	Scales vignette, chromatic aberration and other distortion. Range 0 to 1.

Property	Default	Description
Haptics	1	Scales controller vibration. Range 0 to 1.
Flash Limiter		
Limit Flashes	On	Limits how many flashes can start in a short time. Counts FeelKit tracks on channels mapped to the Flashes group. A mitigation helper, not a photosensitivity certification: test your game with an analysis tool.
Max Flashes Per Second	3	Flashes allowed per second before the limiter acts. Values below 1 allow one flash per 1 / value seconds. Range 0.1 to 30. Editable only when Limit Flashes is on.
Flash Limit Mode	Soften	What happens to flashes above the allowed rate. Editable only when Limit Flashes is on.
Softened Flash Scale	0.3	Strength of softened flashes. Range 0 to 1. Editable only when Limit Flashes is on and Flash Limit Mode is Soften .
Motion		
Allow Camera Roll	On	Allows FeelKit to roll the camera. Off removes camera roll from every recipe.
Max Field Of View Change Per Second	0°/s	Fastest field of view change FeelKit may cause, in degrees per second. 0 is unlimited. At least 0.

Values of the presets as FeelKit ships them. Each preset replaces all fields at once, so a field a preset does not change keeps the value shown in the Default column.

Field	Default	Reduced Motion	Reduced Flashing	No Haptics
Master	1	1	1	1
Camera Shake	1	0.25	1	1
Camera Motion	1	0.25	1	1
Flashes	1	1	0.2	1
Hitstop and Slow-mo	1	1	1	1
Screen Distortion	1	0.5	1	1
Haptics	1	1	1	0
Limit Flashes	On	On	On	On
Max Flashes Per Second	3	3	1	3
Flash Limit Mode	Soften	Soften	Suppress	Soften
Softened Flash Scale	0.3	0.3	0.3	0.3
Allow Camera Roll	On	Off	On	On
Max Field Of View Change Per Second	0°/s	40°/s	0°/s	0°/s

18.7 Comfort storage

Property	Default	Description
Auto Save Comfort	On	Save automatically whenever a player's comfort settings change.
Comfort Save Slot Prefix	FeelComfort	Save slot prefix for the default storage. The local player index is appended.
Comfort Storage Class	None	Storage for comfort settings, implementing Feel Comfort Storage. Empty uses one save game slot per local player.

18.8 Editor preferences

Property	Default	Description
Timeline		
Snap to Frames	On	Snap timeline edits to whole frames.
Snap Frame Rate	60	Frame rate used for snapping. Range 1 to 240.
Library		
Allow Library Editing	Off	Allows editing and saving the recipes that ship with FeelKit (/FeelKit/Library). They are read-only by default, because the plugin is shared by every project on this engine: copy a library recipe into your project instead (right-click in the Content Browser, FeelKit, Recipe from Template). Turn this on only to author the library itself.

19 Console commands

IN THIS CHAPTER

- 19.1** feel.Enabled
- 19.2** feel.GlobalScale
- 19.3** stat Feel
- 19.4** showdebug feel

Enter these in the console of Play In Editor or of a development build; Shipping builds have no console. In Blueprint, the node **Execute Console Command** runs the same commands.

Command	Default	What it does	Edition
<code>feel.Enabled</code>	1	Enables FeelKit playback. 0 stops all playing recipes and blocks new ones.	Lite and Pro
<code>feel.GlobalScale</code>	1	Multiplies the intensity of every FeelKit recipe. 1 is normal, 0 silences everything without stopping it.	Pro
<code>stat Feel</code>		Shows FeelKit's own stat counters (see below).	Pro
<code>showdebug feel</code>		Draws what FeelKit is doing on the game view (see below).	Pro

19.1 feel.Enabled

Set to 0 to stop every playing recipe at once and to block new plays; time slowed by FeelKit returns to normal. Set to 1 to turn playback back on. The value is not saved. The Feel Switch actor and the nodes **Set Feel Enabled** and **Toggle Feel** change the same value (see [17.7 Feel Switch](#)). Only FeelKit is affected: the game's own camera shakes, sounds and effects keep working.

19.2 feel.GlobalScale PRO

Multiplies the intensity of every play on top of the call intensity. At 0 nothing shows, but plays continue and come back at full strength when the value goes back up. Values above 1 make every recipe stronger, and negative values count as 0. There is no Blueprint node for it; use **Execute Console Command** with, for example, `feel.GlobalScale 0.5`.

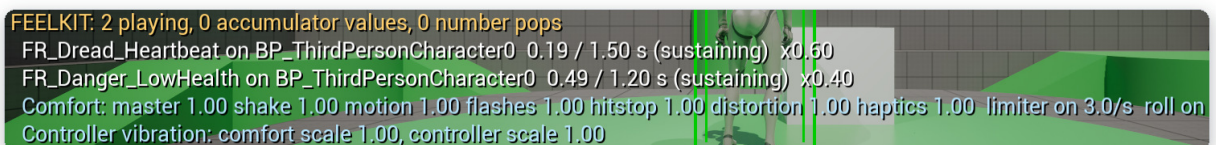
19.3 stat Feel PRO

`stat Feel` shows two counters: **Feel Tick** and **Feel Instances**. Feel Tick is the time FeelKit's update takes, and Feel Instances the number of plays it is running. FeelKit updates only while something plays or still has to be restored, so Feel Tick shows no time while FeelKit is idle.

19.4 showdebug feel PRO

`showdebug feel` draws FeelKit's state over the game view, where Unreal draws its other showdebug pages. Enter the same command again to hide it. The lines are:

- A header with the number of playing recipes, accumulator values and number pops.
- One line per playing recipe: its name, its target, its time and length in seconds, "(sustaining)" while it loops in its sustain region, its intensity and the values of its parameters.
- One line per accumulator value, global or for one actor.
- The comfort scales of the viewing player, with the flash limiter and camera roll settings.
- **Controller vibration:** the comfort scale for vibration and the scale on the player controller. When the controller's scale is 0 the line turns red and ends with NOTHING REACHES THE CONTROLLER.



```
FEELKIT: 2 playing, 0 accumulator values, 0 number pops
FR_Dread_Heartbeat on BP_ThirdPersonCharacter0 0.19 / 1.50 s (sustaining) x0.60
FR_Danger_LowHealth on BP_ThirdPersonCharacter0 0.49 / 1.20 s (sustaining) x0.40
Comfort: master 1.00 shake 1.00 motion 1.00 flashes 1.00 hitstop 1.00 distortion 1.00 haptics 1.00 limiter on 3.0/s roll on
Controller vibration: comfort scale 1.00, controller scale 1.00
```

Figure 19.1 The showdebug feel lines during play, with two recipes playing

20 Writing your own steps in Blueprint and C++

IN THIS CHAPTER

- 20.1 How a step works
- 20.2 A step in Blueprint
- 20.3 A step in C++
- 20.4 Channels and comfort

The 37 built-in steps cover most feedback, and the Blueprint Event step calls any Blueprint code at the start and end of a track. When a game needs an effect of its own that behaves like the built-in ones, with its own settings in the track's Details and its own entry in the **+ Track** menu, it can add a step class. A step written in Blueprint can act when its track starts and stops; a step written in C++ can also produce continuous output, frame by frame, like the built-in camera and screen steps.

Custom steps work in both editions.

20.1 How a step works

A step is a small object that lives inside a track. FeelKit calls it in three ways:

Call	When	Use it for
Evaluate (C++ only)	Every frame while the track plays, in the game and in the preview.	Continuous output: camera offsets, screen flashes, scale, time dilation, post process, material parameters, controller vibration.
On Start	Once, when the track begins.	Side effects: spawning something, playing a sound, sending an event.

Call	When	Use it for
On Stop	Once, when the track ends or its play is stopped. Interrupted is true when the play was stopped early.	Undoing what On Start did, or a closing side effect.

Evaluate computes; FeelKit applies. Evaluate never touches the camera, the screen or an actor. It hands its contribution to an output sink, and FeelKit combines the contributions of every track and every play, applies the player's comfort and the rules for overlapping effects (2.2 [Channels, and what happens when effects overlap](#)), and delivers the result. The same code feeds the editor preview, so a custom step previews exactly like a built-in one.

Evaluate must be pure. Its output may depend only on its context: the time since the track started, the track's length and progress, the intensity, the seed, and the play's directions. It must not remember anything between frames. That is what lets the preview scrub a track forward and backward and show the same frame each time, and what makes a replayed play match the game (10.2 [Replaying a recent play in the editor](#)).

20.2 A step in Blueprint

1. In the Content Browser, choose **Add > Blueprint Class**, open **All Classes**, and pick **Feel Step** as the parent class.
2. Add variables for the step's settings and tick **Instance Editable** on each: they appear in the track's **Details**, per track.
3. Override the functions the step needs, from **Functions > Override** in the **My Blueprint** panel:

Function	Purpose
On Start	Runs when the track begins. Context gives the world, the target, the instigator, the play context with its parameters, the local player controller, the recipe, the track's intensity and length, and the target's location.
On Stop	Runs when the track ends; Interrupted says whether the play was stopped early.
Get Default Channel	Returns the channel a new track with this step gets, such as <code>Feel.Audio</code> . The channel decides the comfort group (8.1 Comfort groups, and how the scales apply).
Supports Preview	Returns whether the recipe editor's preview runs the step. Return false for a step whose side effects only make sense in the game, such as spawning gameplay actors: the preview then skips it and marks the track No preview .

The step appears in the **+ Track** menu under its Blueprint's name, and saving a recipe checks it like any other.

A Blueprint step cannot override Evaluate, so it cannot move the camera or fade the screen frame by frame. For that, write the step in C++, or combine a Blueprint step with built-in tracks in the same recipe.

20.3 A step in C++

A C++ step subclasses `UFeelStep` and overrides `Evaluate`. The module that holds it needs `FeelCore` and `GameplayTags` in its dependencies:

```
PublicDependencyModuleNames.AddRange(new string[] {
    "Core", "CoreUObject", "Engine", "FeelCore", "GameplayTags" });
```

This step moves the camera up and down a set number of times over the track, scaled by the track's intensity. Replace `MYGAME_API` with your module's API macro.

```
// MyFeelStep_CameraBob.h
#pragma once

#include "CoreMinimal.h"
#include "FeelStep.h"
#include "MyFeelStep_CameraBob.generated.h"

/** Moves the camera up and down a set number of times over the track. */
UCLASS(meta = (DisplayName = "Camera Bob"))
class MYGAME_API UMyFeelStep_CameraBob : public UFeelStep
{
    GENERATED_BODY()

public:
    /** Height of each bob. */
    UPROPERTY(EditAnywhere, Category = "Bob", meta = (Units = "Centimeters"))
    float Height = 4.0f;

    /** Number of up and down movements over the length of the track. */
    UPROPERTY(EditAnywhere, Category = "Bob", meta = (ClampMin = "1"))
    int32 Bobs = 2;

    virtual void Evaluate(const FFeelStepEvalContext& Context,
        IFeelOutputSink& Sink) const override;
    virtual FGameplayTag GetDefaultChannel_Implementation() const override;

#if WITH_EDITOR
    virtual void ValidateStep(TArray<FText>& OutErrors,
        TArray<FText>& OutWarnings) const override;
#endif
};
```

```
// MyFeelStep_CameraBob.cpp
#include "MyFeelStep_CameraBob.h"
#include "FeelOutputSink.h"
#include "FeelTags.h"

void UMyFeelStep_CameraBob::Evaluate(const FFeelStepEvalContext& Context,
    IFeelOutputSink& Sink) const
{
    // Depends only on the context, so the editor can scrub the track in any direction.
    const float Wave = FMath::Sin(Context.Alpha * UE_TWO_PI * Bobs);
    const FVector Offset(0.0, 0.0, Height * Wave * Context.Intensity);
    Sink.AddCameraOffset(Offset, FRotator::ZeroRotator);
}

FGameplayTag UMyFeelStep_CameraBob::GetDefaultChannel_Implementation() const
{
    return FeelTags::Camera_Motion;
}

#if WITH_EDITOR
void UMyFeelStep_CameraBob::ValidateStep(TArray<FText>& OutErrors,
    TArray<FText>& OutWarnings) const
{
    if (Height <= 0.0f)
    {
        OutWarnings.Add(NSLOCTEXT("MyGame", "BobHeight",
            "Camera Bob has no height, so it does nothing."));
    }
}
#endif
```

This code was compiled against FeelKit on Unreal Engine 5.6 with no warnings. After a rebuild, **Camera Bob** is in the **+ Track** menu, previews in the recipe editor, and plays in the game with the player's Camera Motion comfort applied.

The context. `FFeelStepEvalContext` holds `LocalTime` (seconds since the track started), `Duration`, `Alpha` (0 at the start, 1 at the end), `Intensity` (the play's intensity times the track's intensity curve and the comfort scale), `Seed`, and the play's `Direction`, `ViewDirection` and `ViewDirectionFromLocation`. Derive any randomness from `Seed`, never from a random number generator, so that each frame can be recomputed.

The sink. `IFeelOutputSink` receives the contribution. Its functions:

Function	Contribution
<code>AddCameraOffset</code>	Camera location offset (X forward, Y right, Z up, in cm) and rotation (degrees).
<code>AddFieldOfViewOffset</code>	Field of view change in degrees.
<code>AddScreenFlash</code> , <code>AddScreenFade</code> , <code>AddScreenTint</code>	Full-screen color with an alpha or weight from 0 to 1.
<code>AddPostProcess</code>	A post-process parameter (vignette, chromatic aberration, saturation) blended in with a weight.
<code>AddPostProcessMaterial</code>	A post-process material with a weight, and the scalar parameter that receives it.
<code>AddTimeDilation</code>	A time dilation for the world or the target, with a priority.

Function	Contribution
<code>AddTargetScale</code> , <code>AddTargetTransform</code>	Relative scale, location and rotation of the target.
<code>AddMaterialParameter</code> , <code>AddOverlayFlash</code> , <code>AddLight</code>	Material parameters, an overlay flash and light changes on the target.
<code>AddForceFeedback</code>	Strength of each controller motor, from 0 to 1.
<code>AddSoundClassAdjust</code>	Volume, pitch and low-pass filter of a sound class.
<code>AddWidgetTransform</code> , <code>AddWidgetColor</code>	Render transform and tint of a widget target.

Other overrides.

Override	Default	When to change it
<code>OnStart_Implementation</code> , <code>OnStop_Implementation</code>	Nothing	Side effects, as in Blueprint.
<code>SupportsPreview_Implementation</code>	true	Return false when the preview cannot show the step.
<code>UsesConstantIntensityByDefault</code>	false	Return true for steps that shape their own envelope, such as a spring; new tracks then start with a flat intensity curve instead of a fade.
<code>RequiresDuration</code>	true	Return false for steps that work as instant tracks, such as a sound; saving then accepts a length of 0.
<code>ValidateStep</code>	Nothing	Report problems with the step's settings when a recipe is saved (5.12 Validation messages on save).

20.4 Channels and comfort

A track's channel decides which comfort group scales it, and which effects it competes with. Give a custom step the channel of the built-in steps it resembles: `Feel.Camera.Motion` for a camera move, `Feel.Screen.Flash` for a flash, and so on. A step on a channel that no comfort group maps plays at the player's **Master** scale only ([8.1 Comfort groups, and how the scales apply](#)). A game can add its own channel tags under `Feel` and map them to a group in the project settings.

21 Recipe JSON files and editor scripting

IN THIS CHAPTER

- 21.1 Exporting and importing in the Content Browser
- 21.2 The file format
- 21.3 Editor scripting

A recipe can be written out as a JSON text file and read back in. Text is easy to review in version control, to share on a forum or in a bug report, to edit in bulk with a script, and to move between projects. FeelKit Pro also offers a few editor scripting functions for setting up recipes and project settings from Python or Editor Utility Blueprints.

21.1 Exporting and importing in the Content Browser

Exporting. Select one or more recipes in the Content Browser, right-click, and choose **Export to JSON...** FeelKit asks where to save each recipe and names the file after it. A notification confirms each file.

Importing. Right-click one recipe and choose **Import from JSON...**, then pick a file. The recipe's contents are replaced by the file's: its settings, parameters and tracks. The import is one undo step, so **Edit > Undo** puts the recipe back as it was. To import into a new recipe, create an empty one first ([5.1 Creating and opening recipes](#)).

- Asset references in the file, such as sounds, materials and curves, are looked up by their paths in this project. A reference the project does not have is left empty, the rest of the recipe still imports, and the notification lists what is missing.
- Library recipes are read-only, so they cannot be import targets; copy one into the project first ([9.3 Copying a library recipe into your project](#)).

- A file that is not valid JSON, or not a FeelKit recipe, changes nothing. A file written by a newer version of FeelKit, with a newer recipe format, is refused with a message saying so.

21.2 The file format

A recipe file is JSON with a short header and the recipe's properties:

```
{
  "format": "FeelKitRecipe",
  "schemaVersion": 2,
  "name": "FR_Denial_Locked",
  "recipe": {
    "tracks": [
      {
        "step": {
          "_ClassName": "/Script/FeelCore.FeelStep_CameraPunch",
          "frequency": 16.0,
          "damping": 10.0
        },
        "startTime": 0.0,
        "duration": 0.3,
        "channel": { "tagName": "Feel.Camera.Motion" }
      }
    ]
  }
}
```

- **format** is always `FeelKitRecipe`, and **schemaVersion** the recipe format of the FeelKit that wrote it.
- **recipe** holds the recipe's properties under their names in lower camel case: `tracks`, `cooldown`, `maxConcurrent`, `parameters` and so on, as they appear in [Chapter 16, Step reference](#).
- Each track's **step** names its class in `_ClassName`, followed by that step's settings. Steps nested inside other steps, such as the options of a Random Choice, are written the same way.
- Asset references are written as asset paths, such as `/FeelKit/Samples/Sounds/S_FK_Denied.S_FK_Denied`.

A property left out of the file keeps its default value on import, so a hand-written file needs only what differs from the defaults. The recipes of the FeelKit library ship in this format, in the plugin's `Library` folder, and make good starting points.

21.3 Editor scripting

These functions are in the category **FeelKit | Editor Scripting**. They run in the editor only, from Python, from Editor Utility Blueprints, or from C++ editor code.

Function	What it does
Create Recipe from Json File	Creates a recipe asset in a folder, imports a JSON file into it and saves it. Returns the recipe and a message saying what happened, including asset references that are not in the project.

Function	What it does
Import Recipe from Json File	Imports a JSON file into an existing recipe and saves it. Returns a message saying what was imported, whether the recipe was saved, and which asset references are not in the project.
Add Feel Map to Project Settings	Adds a Feel Map to Project Settings > Plugins > FeelKit > Feel Maps , if it is not there yet (6.5 Events and Feel Maps).
Set Accumulator in Project Settings	Adds an accumulator to the project settings, or updates the one with that name, with its maximum, decay per second and decay delay (7.3 Accumulators: effects that build up with repeated plays).

In Python, the functions are on `unreal.FeelEditorScripting`. Values that Blueprint returns through output pins come back from Python as return values:

```
import unreal

# Create a recipe from a file: returns the recipe and a message.
recipe, message = unreal.FeelEditorScripting.create_recipe_from_json_file(
    "/Game/Feel", "FR_MyLocked", "C:/Recipes/FR_Denial_Locked.json")
unreal.log(message)

# Import into an existing recipe: returns the message, or None when the import failed.
message = unreal.FeelEditorScripting.import_recipe_from_json_file(recipe,
    "C:/Recipes/FR_Denial_Locked.json")

# Project settings.
unreal.FeelEditorScripting.set_accumulator_in_project_settings("Combo", 10.0, 1.0, 1.5)
```

These calls were run against FeelKit Pro on Unreal Engine 5.6. As with any editor script that saves assets, keep the project under version control or take a copy before running a script over many recipes.

22 Credits and licenses

IN THIS CHAPTER

- 22.1 FeelKit's license
- 22.2 Sample sounds
- 22.3 Materials
- 22.4 Demo projects
- 22.5 Your own project

FeelKit's code, recipes, materials and comfort menu are made for FeelKit and covered by its Fab license. The sample sounds come from other authors, all under Creative Commons CC0 1.0 (public domain): they can be used, changed and shipped in a commercial game, and no credit is required. They are credited here anyway, and in [Credits.md](#) in the plugin folder.

22.1 FeelKit's license

Both editions are distributed on Fab under Fab's Standard License. FeelKit Lite is free; FeelKit Pro is sold with Personal and Professional licenses. The license terms are Fab's, linked from the listings. FeelKit contains no third-party code and no fonts.

22.2 Sample sounds

FeelKit Pro ships 43 sample sounds; FeelKit Lite ships the 7 marked **Lite** below, which its library recipes use. Where a sound was renamed, the name in its source pack is given in brackets. License text: <https://creativecommons.org/publicdomain/zero/1.0/>

Library sounds

Asset in FeelKit	Source pack	Author	In Lite
S_FK_Hit_Light	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney	Lite

Asset in FeelKit	Source pack	Author	In Lite
S_FK_Hit_Heavy	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney	Lite
S_FK_Hit_Crit	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney	Lite
S_FK_Impact_Bullet	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney	
S_FK_Land_Thud	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney	Lite
S_FK_Explosion	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney	
S_FK_Charge_Loop	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney	
S_FK_Whoosh	RPG Audio (https://kenney.nl/assets/rpg-audio)	Kenney	
S_FK_Pickup	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney	Lite
S_FK_LevelUp	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney	
S_FK_Heartbeat	RPG Audio (https://kenney.nl/assets/rpg-audio)	Kenney	
S_FK_Alarm	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney	Lite
S_FK_Denied	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney	Lite
S_FK_UI_Hover	UI Audio (https://kenney.nl/assets/ui-audio)	Kenney	
S_FK_UI_Click	UI Audio (https://kenney.nl/assets/ui-audio)	Kenney	

Body sounds (demo recipes)

Asset in FeelKit	Source pack	Author
S_FK_Body_Hit (impactPunch_medium_001)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney
S_FK_Body_Fall (impactSoft_heavy_003)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney

Movement sounds (Platformer demo kit)

Asset in FeelKit	Source pack	Author
S_FK_Jump_Whoosh (swish-12)	Swishes Sound Pack (https://opengameart.org/content/swishes-sound-pack)	artisticdude
S_FK_Dash_Whoosh (swish-7)	Swishes Sound Pack (https://opengameart.org/content/swishes-sound-pack)	artisticdude
S_FK_Jump_Scuff (footstep_concrete_000)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney
S_FK_Land_Step (footstep_concrete_003)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney
S_FK_Land_Soft (impactSoft_medium_001)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney
S_FK_Land_Heavy (impactSoft_heavy_001)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney

Asset in FeelKit	Source pack	Author
S_FK_Wall_Thump (impactSoft_medium_002)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney
S_FK_Air_Lift (maximize_009)	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney

Shooter sounds (Shooter demo kit)

Asset in FeelKit	Source pack	Author
S_FK_Shot_Pistol (laserRetro_001)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Shot_Rifle (laserRetro_000)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Shot_Thump (laserSmall_004)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Shot_Launcher (laserLarge_001)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Explosion_Crunch (explosionCrunch_000)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Explosion_Low (lowFrequency_explosion_001)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Impact_Thud (impactMetal_004)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Impact_Ping (impactMetal_light_001)	Impact Sounds (https://kenney.nl/assets/impact-sounds)	Kenney
S_FK_Hit_Tick (click_002)	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney
S_FK_Kill_Confirm (confirmation_001)	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney
S_FK_Dry_Click (switch_004)	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney
S_FK_Weapon_Switch (metalLatch)	RPG Audio (https://kenney.nl/assets/rpg-audio)	Kenney

Horror sounds (Horror demo kit)

Asset in FeelKit	Source pack	Author
S_FK_Dread_Drone (spaceEngineLow_002)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Scare_Boom (lowFrequency_explosion_000)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Light_Crackle (glitch_001)	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney
S_FK_Light_Buzz (forceField_001)	Sci-fi Sounds (https://kenney.nl/assets/sci-fi-sounds)	Kenney
S_FK_Door_Creak (creak3)	RPG Audio (https://kenney.nl/assets/rpg-audio)	Kenney

HUD sounds (Shooter demo kit)

Asset in FeelKit	Source pack	Author
S_FK_UI_Tick (pluck_001)	Interface Sounds (https://kenney.nl/assets/interface-sounds)	Kenney

The heartbeat of the Horror demo is the library's [S_FK_Heartbeat](#), listed above. [ATT_FK_World](#), the sound attenuation used for sounds placed in the world, is made for FeelKit.

22.3 Materials

Made for FeelKit and covered by its license. Part of FeelKit Pro.

Asset	Used by
M_FK_Decal_Scorch	Spawn Decal (library and Shooter demo recipes)
M_FK_HitFlash	Hit Flash (demo recipes)
M_FK_PP_Pulse	A ready-made material for the Post Process Material Pulse step

22.4 Demo projects

The playable demos and their source projects are built on Epic's Unreal Engine templates, whose content is covered by Epic's license for Unreal Engine. The Action/RPG demo's own sounds and its guard shield material are listed in the [Credits.md](#) of that demo project.

22.5 Your own project

Nothing here has to stay in a game. A library recipe copied into a project points at these samples, and its sounds and materials can be swapped for the game's own without changing anything else.

23 Version history

The version of an installed FeelKit is shown in **Edit > Plugins**, on its entry. Both editions share one version number.

23.1 1.0.0

First release, for Unreal Engine 5.6, 5.7 and 5.8, built and tested on Windows.

FeelKit Lite

The recipe editor, 12 steps, **Play Feel** and its target nodes, the full comfort layer with its presets, flash limiter and comfort menu, the Feel Switch, and 11 library recipes.

FeelKit Pro

Everything in Lite, plus 25 more steps, parameters, accumulators and sustained recipes, Feel Maps and events, animation notifies, the Feel Trigger, Feel Input and Feel Replication components, the GAS add-on, the Recipe Browser with 38 library recipes, the FeelKit Debugger, recent plays, Capture GIF, sound waveforms and tracks from sound, JSON import and export, the Comfort Audit, and the recipes of the demo levels.

Appendices



Appendix A: Glossary

Term	Meaning
Accumulator	A named value, defined in the project settings, that builds up when the game adds to it and falls back toward 0 over time. Recipe parameters can read it. See 2.5 Parameters, accumulators and sustained recipes, in brief.
Additive Capped	Camera arbitration mode in which the camera output of separate plays adds up and the sum is limited by caps. See 18.2 Camera.
Applies To	Track field that decides whether the track's actor effects go to the play's target or to its instigator. See 2.4 Targets: what a recipe plays on.
Blend out	Fading a play out over Blend Out Time (0.2 s by default) when it is stopped with Stop Feel and Blend Out on.
Channel	Gameplay tag under Feel1 on every track, such as Feel1.Camera.Shake . It decides the track's comfort group and its color on the timeline. See 2.2 Channels, and what happens when effects overlap.
Comfort group	One of the six groups a player can scale separately: Camera Shake, Camera Motion, Flashes, Hitstop and Slow-mo, Screen Distortion and Haptics. See 2.7 Comfort on one page.
Comfort preset	A full set of comfort scales applied at once: the three built-in presets or a Feel Comfort Preset asset.
Comfort scale	A player's setting for Master or one comfort group, from 0 (off) to 1 (full strength).
Context tags	Gameplay tags passed with a play that describe its circumstances. Feel Maps use them to choose between rows.
Cooldown	Recipe field: seconds before the recipe can play again on the same target.
Default channel	The channel a new track of a step gets. Listed for each step in chapter 16, Step reference.
Default Intensity	Recipe field that multiplies every play of the recipe.
Essential track	A track that carries information the player needs. When its group is turned off, its substitute step plays, or it keeps its essential floor.
Essential Floor	Track field: the lowest comfort scale an essential track without a substitute can drop to. It does not apply on Camera Shake and Camera Motion.

Term	Meaning
Feel Event	A gameplay tag sent with Send Feel Event that says what happened. A Feel Map picks the recipe. See 2.6 Events and Feel Maps, in brief.
Feel Input	Component that plays recipes or sends Feel Events from Enhanced Input actions. See 17.10 Feel Input component.
Feel Map	Asset whose rows assign recipes to Feel Events. See 17.5.2 Feel Map asset.
Feel Replication	Component that plays recipes and sends Feel Events on several machines in a networked game. See 17.11 Feel Replication component.
Feel Switch	Actor that lets players turn FeelKit off and on while playing, to compare. See 17.7.4 Feel Switch actor.
Feel Trigger	Component that plays recipes or sends Feel Events when common events happen to its owner, such as landing or taking damage. See 17.9 Feel Trigger component.
Flash limiter	Part of each player's comfort settings that limits how many flash tracks may start per second.
Handle	The value a play node returns, used to stop the play, release it or change its parameters. It stays safe to keep after the play ends.
Instant track	A track with a length of 0, for steps that act once, such as Play Sound.
Instigator	A second actor passed with a play, besides the target, such as the attacker of a hit. See 2.4 Targets: what a recipe plays on.
Intensity	How strongly a track plays at a moment: call intensity, recipe default, intensity curve, parameter mappings, random intensity and comfort multiplied together. See 2.3 Intensity: how strong each effect plays.
Intensity curve	Track field: the intensity over the track, from its start to its end.
Library recipe	A ready-made recipe that ships with FeelKit under /FeelKit/Library. It is read-only; copy it into your project to change it.
Max Concurrent	Recipe field: how many plays of the recipe can run at once on one target. 0 means no limit.
Motion shape	The form of a punch-like motion over a track: Spring, Kick or Smooth. See 16.4 Shared motion settings.
No preview	Label on tracks whose step the editor preview cannot show, such as hitstops and controller vibration. They still play in the game.
Parameter	A named number a recipe accepts each time it plays, such as Damage. See 2.5 Parameters, accumulators and sustained recipes, in brief.
Parameter mapping	Track field: a curve that turns a parameter into an intensity multiplier for the track.
Play	One running instance of a recipe, started by a play node and identified by its handle.
Play context	The optional information passed with a play: parameter values, instigator, direction, location, normal and context tags. See 17.3.1 Feel Play Context structure.
Play in PIE	Recipe editor button that plays the open recipe on the player's pawn in a running Play In Editor session.
Priority	Field of the time steps. When time requests overlap on one clock, the highest priority wins.

Term	Meaning
Real time	Time that hitstops and slow motion do not slow. Recipes are timed in real time.
Recipe	Asset that describes how one moment feels, as timed tracks of steps. See 2.1 Recipes, tracks and steps.
Release	Ending the sustain loop of a sustained play so that it plays the rest of the recipe.
Seed	Track field that fixes the noise and random choices of the step, so the same seed gives the same result.
Step	The effect a track plays, such as Camera Punch or Play Sound. See chapter 16, Step reference.
Strongest Wins	The default camera arbitration mode: for location, rotation and field of view, the strongest play wins. See 18.2 Camera.
Substitute step	Track field: the step an essential track plays instead when the player turns its comfort group off.
Sustain	A looping region of a recipe that repeats until the play is released. See 2.5 Parameters, accumulators and sustained recipes, in brief.
Target	What a recipe plays on: an actor, a component, a location, a player's camera or a widget. See 2.4 Targets: what a recipe plays on.
Track	One timed step inside a recipe, with its own start time, length, channel and intensity curve.